

MMALS_Prototype_1_1_RC2b_Strict_No_Leakage_Policy_Selection_Harness

May 26, 2026

1 MMALS Prototype v1.1-RC2b — Strict No-Leakage Policy-Selection Harness

Mycelium-Inspired Mutualistic Adaptive Learning Systems

Status: strict benchmark harness, not a new backbone architecture.

This notebook reruns the MMALS benchmark loop and selects a deployable policy **inside the experiment**, using validation memory only, before final-test interpretation.

Historical fixed candidate:

MMALS v1.0-RC1b = `paired_context_gap_selected`

RC2b strict selected candidate:

`paired_rc2b_validation_selected_policy`

Candidate policy family:

`proto no-repair / context-only global head / true CE-KL guarded global head / RC1b guarded context`

Scientific question:

Can MMALS select the deployable readout/head policy from internal validation memory, without final-test leakage, while remaining replay-beating, stable, and auditable?

This is the strict successor to the fast v1.1-RC2 retrospective audit.

1.1 1. Why RC2b exists

The v1.1-MS1 and v1.1-RC2 audit results showed that MMALS is replay-beating across Fashion-MNIST, MNIST, RotatedMNIST and PermutedMNIST, but the fixed RC1b guarded head is not always the best deployable policy.

The RC2 audit was useful but retrospective: it consumed existing scorecards. RC2b is stricter.

RC2b reruns the benchmark and performs policy selection during the experiment:

1. train the same proto-first MMALS backbone;
2. split memory into repair, context-selection and policy-validation partitions;
3. build candidate deployable policies;
4. score candidates on policy-validation memory only;

5. select one policy before final-test evaluation;
6. evaluate the selected policy on final test;
7. save traces, selection audit, scorecards, manifests and ZIP evidence.

A pass here would justify treating RC2 as a real validation-selected deployable candidate. It is still not certification, production readiness, boundary-free streaming, open-world learning, or Level 4 domain qualification.

1.2 0. Optional install cell for Colab

Most Colab runtimes already include the required packages. Uncomment only if needed.

```
[37]: # !pip -q install torch torchvision pandas numpy matplotlib
```

1.3 1. Imports and reproducibility

This block imports the scientific Python stack, selects CPU/GPU, and defines reproducibility helpers.

```
[38]: import math
import copy
import random
import gc
import time
import shutil
import sys
import os
import json
import hashlib
import uuid
import platform
import contextlib
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, List, Tuple, Optional, Any

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from IPython.display import display

# Robust torchvision import. Some runtimes have torch/torchvision NMS mismatch.
try:
    from torchvision import datasets, transforms
```

```

except RuntimeError as e:
    if "torchvision::nms" not in str(e):
        raise
    for name in list(sys.modules):
        if name.startswith("torchvision"):
            del sys.modules[name]
    try:
        lib = torch.library.Library("torchvision", "DEF")
        lib.define("nms(Tensor dets, Tensor scores, float iou_threshold) -> Tensor")
    except Exception:
        pass
    from torchvision import datasets, transforms

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("DEVICE:", DEVICE)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def clear_memory():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

def make_loader(ds, batch_size=128, shuffle=True):
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
        num_workers=0, pin_memory=False)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def total_parameters(model):
    return sum(p.numel() for p in model.parameters())

def model_size_mb(model, bytes_per_param=4, trainable_only=True):
    n = count_parameters(model) if trainable_only else total_parameters(model)
    return n * bytes_per_param / (1024 ** 2)

```

DEVICE: cpu

1.4 2. Configuration

Default mode is `smoke` for first execution checks. Switch to `robust` after smoke passes. This notebook keeps the existing options `MNIST`, `FashionMNIST`, and `RotatedMNIST`, and adds the new `PermutedMNIST` option without removing the previous ones.

```
[39]: RUN_MODE = "robust" # first validate execution; set to "robust" after smoke
      ↪ passes
      DATASET_NAME = "FashionMNIST" # "MNIST", "FashionMNIST", "RotatedMNIST", or
      ↪ "PermutedMNIST"
      DATA_ROOT = "./data"
      NOTEBOOK_BASENAME =
      ↪ "MMALS_Prototype_1_1_RC2b_Strict_No_Leakage_Policy_Selection_Harness_Colab"
      NOTEBOOK_FILENAME = f"{NOTEBOOK_BASENAME}.ipynb"
      RESULTS_DIR = Path("./mmals_v11_rc2b_strict_policy_selection_results")
      RESULTS_DIR.mkdir(parents=True, exist_ok=True)

      if RUN_MODE == "smoke":
          TRAIN_PER_CLASS = 200
          TEST_PER_CLASS = 100
          SEEDS = [0]
          EPOCHS_PER_TASK = 2
          MEMORY_PER_TASK = 96
          MEMORY_PER_CLASS = 96
          BATCH_SIZE = 128
          LABEL_NOISE_RATE = 0.05
          INPUT_NOISE_STD = 0.03
          CONTEXT_NOISE_RATE = 0.15
          PROBE_EPOCHS = 10
          PROBE_MAX_PER_TASK = 600
          READOUT_TRAIN_EPOCHS = 2
          READOUT_TRAIN_STEPS = 10
      elif RUN_MODE == "evidence":
          TRAIN_PER_CLASS = 1200
          TEST_PER_CLASS = 400
          SEEDS = [0, 1, 2]
          EPOCHS_PER_TASK = 8
          MEMORY_PER_TASK = 256
          MEMORY_PER_CLASS = 256
          BATCH_SIZE = 128
          LABEL_NOISE_RATE = 0.10
          INPUT_NOISE_STD = 0.05
          CONTEXT_NOISE_RATE = 0.20
          PROBE_EPOCHS = 25
          PROBE_MAX_PER_TASK = 2000
          READOUT_TRAIN_EPOCHS = 5
          READOUT_TRAIN_STEPS = 50
      elif RUN_MODE == "robust":
```

```

TRAIN_PER_CLASS = 1200
TEST_PER_CLASS = 400
SEEDS = list(range(5))
EPOCHS_PER_TASK = 8
MEMORY_PER_TASK = 256
MEMORY_PER_CLASS = 256
BATCH_SIZE = 128
LABEL_NOISE_RATE = 0.10
INPUT_NOISE_STD = 0.05
CONTEXT_NOISE_RATE = 0.20
PROBE_EPOCHS = 25
PROBE_MAX_PER_TASK = 2000
READOUT_TRAIN_EPOCHS = 5
READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "benchmark":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = list(range(5))
    EPOCHS_PER_TASK = 8
    MEMORY_PER_TASK = 256
    MEMORY_PER_CLASS = 256
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 25
    PROBE_MAX_PER_TASK = 2000
    READOUT_TRAIN_EPOCHS = 5
    READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "paper":
    TRAIN_PER_CLASS = 3000
    TEST_PER_CLASS = 1000
    SEEDS = list(range(10))
    EPOCHS_PER_TASK = 15
    MEMORY_PER_TASK = 512
    MEMORY_PER_CLASS = 512
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 50
    PROBE_MAX_PER_TASK = 5000
    READOUT_TRAIN_EPOCHS = 10
    READOUT_TRAIN_STEPS = 120
else:
    raise ValueError("RUN_MODE must be smoke, evidence, robust, benchmark, or_
↳paper")

```

```

PAIR_MODE = "overlap_chain"
N_TASKS = 5
ROTATION_ANGLES = [-30, -15, 0, 15, 30]
PERMUTATION_SEEDS = [100, 101, 102, 103, 104]
N_CONTEXTS = N_TASKS
N_HOSTS = 5
N_CLASSES = 6
INPUT_DIM = 28 * 28
HIDDEN_DIM = 256
LATENT_DIM = 64
ROUTE_HIDDEN_DIM = 128
TASK_EMB_DIM = 16
CONTEXT_FEAT_DIM = 32

LR = 1e-3
PROBE_LR = 1e-2
READOUT_LR = 3e-3
TEMPERATURE = 0.7
DISTILL_TEMPERATURE = 2.0
PROBE_DISTILL_TEMPERATURE = 2.0
CONTEXT_TEMPERATURE = 0.45
FRECHET_TEMPERATURE = 0.20
LATENT_FRECHET_TEMPERATURE = 0.35
CENTER_TEMPERATURE = 0.20

# Context calibration inherited from v0.14.x.
CONTEXT_SUP_START = 1.00
CONTEXT_SUP_END = 0.15
CONTEXT_SUP_GAMMA = 1.25
CONTEXT_LABEL_SMOOTHING = 0.03
LAMBDA_CONTEXT_CENTER = 0.35
LAMBDA_CONTEXT_ENTROPY = 0.010
LAMBDA_CONTEXT_MARGIN = 0.020
CONTEXT_MARGIN_TARGET = 0.35
LAMBDA_CONTEXT_REPLAY_CE = 0.50
LAMBDA_CONTEXT_REPLAY_CENTER = 0.20
LAMBDA_CONTEXT_TEACHER_KL = 0.30
LAMBDA_CONTEXT_FEATURE_DRIFT = 0.10
CONTEXT_DISTILL_TEMPERATURE = 2.0
CONTEXT_REPLAY_BATCH = 64
CONTEXT_TEMP_GRID = [0.25, 0.35, 0.45, 0.60, 0.80, 1.00, 1.25, 1.50, 2.00]
TEMP_CALIB_MAX_ITEMS = 128

# Prototype-first context posterior.
BATCH_BLEND_SINGLE = 0.30
BATCH_BLEND_LATENT = 0.55

```

```

BATCH_BLEND_FEATURE = 0.15
PROTO_FIRST_BLEND_SINGLE = 0.05
PROTO_FIRST_BLEND_LATENT = 0.75
PROTO_FIRST_BLEND_FEATURE = 0.20

# Replay and functional-memory controls.
BALANCED_REPLAY_WEIGHT = 1.00
BALANCED_REPLAY_BATCH = 128
BALANCED_REPLAY_USE_EVAL_CONTEXT = True
LAMBDA_ENTROPY = 0.002
LAMBDA_METABOLIC = 1e-4
PROTOTYPE_MAX_ITEMS = 256

# Readout/head audit controls.
HEAD_REPAIR_BATCH = 128
HEAD_MARGIN_TARGET = 0.25
HEAD_MARGIN_LAMBDA = 0.30
HARD_CLASSES = [2]
REFERENCE_CLASS_FOR_MARGIN = 4
READOUT_KL_WEIGHT = 0.35
READOUT_TEMP = 0.25
COSINE_SCALE_INIT = 12.0
VALIDATION_SPLIT_FRACTION = 0.25
VALIDATION_MAX_ITEMS_PER_CLASS = 128
VALIDATION_SCORE_CLASS2_WEIGHT = 0.20
VALIDATION_SCORE_MINCLASS_WEIGHT = 0.20
VALIDATION_SCORE_DAMAGE_WEIGHT = 1.00
VALIDATION_ACCEPTANCE_TOL = 0.005

# RC2b strict no-leakage policy-selection controls.
RC2B_METHOD = "paired_rc2b_validation_selected_policy"
RC2B_STRICT_NO_FINAL_TEST_SELECTION = True
RC2B_EVALUATE_ALL_CANDIDATES_AFTER_SELECTION = True
RC2B_POLICY_REPAIR_FRACTION = 0.50
RC2B_CONTEXT_SELECTION_FRACTION = 0.25
RC2B_POLICY_VALIDATION_FRACTION = 0.25
RC2B_SELECTION_TIE_TOL = 0.002
RC2B_COMPLEXITY_PENALTY = 0.001
RC2B_CLASS2_FLOOR = 0.80
RC2B_CLASS5_FLOOR = 0.90
RC2B_MIN_CLASS_FLOOR = 0.80
RC2B_VALIDATION_SCORE_CLASS5_WEIGHT = 0.05
RC2B_VALIDATION_SCORE_FLOOR_PENALTY = 2.00
RC2B_CANDIDATE_VARIANTS = [
    "proto_global_no_repair",
    "context_blend_selected_global",
    "context_temp_blend_selected_global",

```

```

    "proto_global_head_ce_kl_guard_035",
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
]

# Patch missing inherited RC1b context-selection constants.
# Run this once, then rerun the failed "Run experiment" cell.

CONTEXT_TEMP_SELECTION_GRID = [0.35, 0.60, 0.80, 1.00, 1.25, 1.50]
CONTEXT_BLEND_SELECTION_GRID = [
    ("base_proto_first", 0.05, 0.75, 0.20),
    ("latent_only", 0.00, 1.00, 0.00),
    ("feature_only", 0.00, 0.00, 1.00),
    ("latent_dominant", 0.05, 0.90, 0.05),
    ("single_mid", 0.30, 0.55, 0.15),
    ("balanced_all", 0.34, 0.33, 0.33),
]

CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT = 0.20
CONTEXT_SELECTION_SCORE_CLASS2_WEIGHT = 0.10
CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT = 0.10
CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY = 0.02

# Backward-compatible names used by inherited v0.14.x helper functions.
HEAD_REFRESH_EPOCHS = READOUT_TRAIN_EPOCHS
HEAD_REFRESH_STEPS = READOUT_TRAIN_STEPS
HEAD_REFRESH_LR = READOUT_LR

# Optional old helper compatibility constants.
PROBE_DISTILL_EPOCHS = 2
PROBE_DISTILL_LR = 2e-3
BIAS_CALIB_STEPS = 8
BIAS_CALIB_LR = 5e-2
PROBE_DISTILL_ALPHA_CE = 0.50
PROBE_DISTILL_ALPHA_KL = 0.50
BIAS_CALIB_REG = 1e-3

LOCAL_ADAPTER_KL_GRID = [0.05, 0.15, 0.35, 0.60]
LOCAL_ADAPTER_MARGIN_WEIGHT = HEAD_MARGIN_LAMBDA

SELECTED_LOCAL_HEAD_VARIANTS = [
    "no_repair_global_head",
    "kl_only_true_noop",
    "global_head_ce_kl_guard_035",
    "local_proto_cosine",
    "local_proto_euclidean",
    "local_trainable_proto_ce_kl_035",

```

```

    "local_diag_adapter_ce_only",
    "local_diag_adapter_ce_kl_005",
    "local_diag_adapter_ce_kl_015",
    "local_diag_adapter_ce_kl_035",
    "local_diag_adapter_ce_kl_060",
    "local_diag_adapter_ce_kl_margin_035",
    "local_adapter_validation_selected",
]

# Keep the historical variants for control rows and final candidate comparison.
SELECTED_CONTEXT_HEAD_VARIANTS = [
    "proto_global_no_repair",
    "proto_kl_only_true_noop",
    "proto_global_head_ce_kl_guard_035",
    "oracle_global_no_repair_reference",
    "oracle_global_head_ce_kl_guard_035_reference",
    "context_blend_selected_global",
    "context_temp_blend_selected_global",
    "context_blend_selected_head_guard_035",
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
]

# Comparative baselines for the benchmark harness.
if RUN_MODE == "smoke":
    SELECTED_BASELINE_METHODS = [
        "finetune",
        "experience_replay",
        "balanced_replay",
        "joint_upper_bound",
    ]
else:
    SELECTED_BASELINE_METHODS = [
        "finetune",
        "ewc",
        "lwf",
        "experience_replay",
        "balanced_replay",
        "sparse_moe",
        "pnn",
        "joint_upper_bound",
    ]

EWC_LAMBDA = 25.0
LWF_LAMBDA = 1.0
BASELINE_REPLAY_WEIGHT = 1.0
SPARSE_MOE_ENTROPY_WEIGHT = 0.002

```

```

RUN_FINAL_PROBES = True
PROBE_CONTEXT_MODES = ["proto", "oracle"]
HOST_PROBE_MODE = "oracle"

COMMON_MLP = dict(
    model_family="mlp",
    strategy="none",
    balanced_replay_ce=False,
    balanced_replay_weight=0.0,
    output_distill=False,
    route=0.0,
    latent=0.0,
    output=0.0,
    context_calibrated=False,
    context_replay=False,
    temp_calibration=False,
    context_replay_scale=0.0,
    head_margin_repair=False,
    post_head_refresh=False,
    post_probe_distill=False,
    post_bias_calibration=False,
)
COMMON_MMALS = dict(
    model_family="mmals",
    route=0.5,
    latent=1.0,
    output=1.0,
    output_distill=True,
    balanced_replay_ce=True,
    balanced_replay_weight=BALANCED_REPLAY_WEIGHT,
    context_calibrated=False,
    context_replay=False,
    temp_calibration=False,
    context_replay_scale=0.0,
    batch_blend_single=PROTO_FIRST_BLEND_SINGLE,
    batch_blend_latent=PROTO_FIRST_BLEND_LATENT,
    batch_blend_feature=PROTO_FIRST_BLEND_FEATURE,
    head_margin_repair=False,
    post_head_refresh=False,
    post_probe_distill=False,
    post_bias_calibration=False,
)
COMMON_PROTO_FIRST = {
    **COMMON_MMALS,
    "strategy": "proto_first",
    "context_calibrated": True,

```

```

    "context_replay": True,
    "temp_calibration": True,
    "context_replay_scale": 1.0,
    "route": 0.5,
    "latent": 1.0,
    "balanced_replay_ce": True,
    "balanced_replay_weight": BALANCED_REPLAY_WEIGHT,
    "output_distill": True,
    "output": 1.0,
}
METHOD_CONFIGS = {
    "mlp_finetune": {**COMMON_MLP},
    "mlp_ewc": {**COMMON_MLP},
    "mlp_lwf": {**COMMON_MLP},
    "mlp_experience_replay": {**COMMON_MLP},
    "mlp_balanced_replay": {**COMMON_MLP},
    "sparse_moe": {**COMMON_MLP, "model_family": "sparse_moe"},
    "pnn": {**COMMON_MLP, "model_family": "pnn"},
    "mlp_joint_upper_bound": {**COMMON_MLP},
    "mmals_balanced_replay": {
        **COMMON_MMALS,
        "strategy": "task_emb",
        "balanced_replay_ce": True,
        "balanced_replay_weight": BALANCED_REPLAY_WEIGHT,
        "output_distill": True,
        "output": 1.0,
    },
    "mmals_uniform_balanced_replay": {
        **COMMON_MMALS,
        "strategy": "uniform",
    },
    "mmals_proto_first_balanced_replay": {
        **COMMON_PROTO_FIRST,
    },
    "mmals_oracle_context_balanced_replay": {
        **COMMON_MMALS,
        "strategy": "oracle",
    },
}

print("RUN_MODE:", RUN_MODE)
print("DATASET_NAME:", DATASET_NAME)
print("SEEDS:", SEEDS)
print("RC2B_CANDIDATE_VARIANTS:", RC2B_CANDIDATE_VARIANTS)
print("SELECTED_BASELINE_METHODS:", SELECTED_BASELINE_METHODS)
print("v1.1-RC2b strict no-leakage harness: train backbone, select deployable_
↳policy using validation memory only, then evaluate final test.")

```

```
print("ROTATION_ANGLES:", ROTATION_ANGLES)
print("PERMUTATION_SEEDS:", PERMUTATION_SEEDS)
```

```
RUN_MODE: robust
DATASET_NAME: FashionMNIST
SEEDS: [0, 1, 2, 3, 4]
RC2B_CANDIDATE_VARIANTS: ['proto_global_no_repair',
'context_blend_selected_global', 'context_temp_blend_selected_global',
'proto_global_head_ce_kl_guard_035',
'context_temp_blend_selected_head_guard_035', 'context_gap_selected']
SELECTED_BASELINE_METHODS: ['finetune', 'ewc', 'lwf', 'experience_replay',
'balanced_replay', 'sparse_moe', 'pnn', 'joint_upper_bound']
v1.1-RC2b strict no-leakage harness: train backbone, select deployable policy
using validation memory only, then evaluate final test.
ROTATION_ANGLES: [-30, -15, 0, 15, 30]
PERMUTATION_SEEDS: [100, 101, 102, 103, 104]
```

1.5 2b. Persistence and OTEL-style traceability layer

This cell creates a persistent artifact directory. In Colab it can mount Google Drive and write results there; outside Colab it falls back to a local directory.

The trace format is **OTEL-style**, not a full external OTLP export by default. It writes JSONL events and span-like rows that can later be ingested by Jaeger, Grafana Tempo, Data-dog, OpenSearch, or an internal STRAT-Q observability pipeline.

```
[40]: # -----
# v1.1-RC2b Strict No-Leakage Policy-Selection Harness
# -----
# Set to False if you do not want Colab to mount Google Drive.
PERSIST_TO_GOOGLE_DRIVE = True
GOOGLE_DRIVE_PROJECT_DIR = "MMALS/v1_1_RC2b_strict_policy_selection"

RUN_TIMESTAMP_UTC = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
RUN_ID =
    ↪f"MMALS_v11_RC2b_STRICT_{DATASET_NAME}_{RUN_MODE}_{RUN_TIMESTAMP_UTC}_seeds{len(SEEDS)}"
TRACE_ID = hashlib.sha256(RUN_ID.encode("utf-8")).hexdigest()[:16]

# Notebook/PDF evidence-export names. The helper cell later stores a
    ↪run-summary PDF
# and optionally a full notebook PDF in RESULTS_DIR/notebook_exports.
NOTEBOOK_BASENAME = globals().get("NOTEBOOK_BASENAME",
    ↪f"MMALS_Prototype_1_0_RC1b_RotatedMNIST_Overlap_Cross_Benchmark_Colab")
NOTEBOOK_FILENAME = globals().get("NOTEBOOK_FILENAME", f"{NOTEBOOK_BASENAME}.
    ↪ipynb")

DRIVE_AVAILABLE = False
if PERSIST_TO_GOOGLE_DRIVE:
```

```

try:
    from google.colab import drive # type: ignore
    drive.mount("/content/drive", force_remount=False)
    DRIVE_AVAILABLE = True
except Exception as e:
    print("Google Drive mount skipped; falling back to local persistence.")
    print("Reason:", repr(e))

if DRIVE_AVAILABLE:
    PERSISTENCE_ROOT = Path("/content/drive/MyDrive") / GOOGLE_DRIVE_PROJECT_DIR
else:
    PERSISTENCE_ROOT = Path("./mmals_v11_rc2b_strict_policy_selection_results")

RESULTS_DIR = PERSISTENCE_ROOT / RUN_ID
TRACE_DIR = RESULTS_DIR / "traces"
AUDIT_DIR = RESULTS_DIR / "audit"
PLOTS_DIR = RESULTS_DIR / "plots"
RAW_LOG_DIR = RESULTS_DIR / "raw_logs"
NOTEBOOK_EXPORT_DIR = RESULTS_DIR / "notebook_exports"
for d in [RESULTS_DIR, TRACE_DIR, AUDIT_DIR, PLOTS_DIR, RAW_LOG_DIR,
↳NOTEBOOK_EXPORT_DIR]:
    d.mkdir(parents=True, exist_ok=True)

TRACE_JSONL_PATH = TRACE_DIR / f"otel_style_events_{RUN_MODE}.jsonl"
TRACE_SPANS_PATH = TRACE_DIR / f"otel_style_spans_{RUN_MODE}.csv"
RAW_CONSOLE_LOG_PATH = RAW_LOG_DIR / f"console_{RUN_MODE}.log"
MANIFEST_PATH = RESULTS_DIR / f"run_manifest_{RUN_MODE}.json"

TRACE_EVENTS = []
TRACE_SPANS = []
_ORIGINAL_STDOUT = sys.stdout
_ORIGINAL_STDERR = sys.stderr
_LOG_FILE_HANDLE = None
_LOGGING_ACTIVE = False

def safe_jsonable(obj):
    """Convert numpy/torch/Pandas objects to JSON-safe values without dumping
    ↳large tensors."""
    if obj is None:
        return None
    if isinstance(obj, (str, int, float, bool)):
        if isinstance(obj, float) and (math.isnan(obj) or math.isinf(obj)):
            return None
        return obj
    if isinstance(obj, (np.integer,)):
        return int(obj)
    if isinstance(obj, (np.floating,)):

```

```

        v = float(obj)
        return None if math.isnan(v) or math.isinf(v) else v
    if isinstance(obj, (np.ndarray,)):
        if obj.size <= 16:
            return obj.tolist()
        return {"type": "ndarray", "shape": list(obj.shape), "mean": float(np.
↳ nanmean(obj)), "std": float(np.nanstd(obj))}
    if torch.is_tensor(obj):
        t = obj.detach().cpu()
        if t.numel() == 1:
            return safe_jsonable(t.item())
        if t.numel() <= 16:
            return t.tolist()
        tf = t.float()
        return {"type": "tensor", "shape": list(t.shape), "mean": float(tf.
↳ mean()), "std": float(tf.std())}
    if isinstance(obj, dict):
        return {str(k): safe_jsonable(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [safe_jsonable(x) for x in obj]
    return str(obj)

def config_snapshot():
    keys = [
        "RUN_MODE", "DATASET_NAME", "PAIR_MODE", "N_TASKS", "N_CONTEXTS",
↳ "N_HOSTS", "N_CLASSES",
        "TRAIN_PER_CLASS", "TEST_PER_CLASS", "SEEDS", "EPOCHS_PER_TASK",
↳ "MEMORY_PER_TASK", "MEMORY_PER_CLASS",
        "LABEL_NOISE_RATE", "INPUT_NOISE_STD", "CONTEXT_NOISE_RATE",
↳ "LATENT_DIM", "HIDDEN_DIM",
        "READOUT_TRAIN_EPOCHS", "READOUT_TRAIN_STEPS",
↳ "CONTEXT_TEMP_SELECTION_GRID", "CONTEXT_BLEND_SELECTION_GRID",
        "SELECTED_CONTEXT_HEAD_VARIANTS", "SELECTED_BASELINE_METHODS",
↳ "RC2B_CANDIDATE_VARIANTS", "RC2B_METHOD",
↳ "RC2B_STRICT_NO_FINAL_TEST_SELECTION", "NOTEBOOK_FILENAME",
↳ "ROTATION_ANGLES", "PERMUTATION_SEEDS",
    ]
    snap = {}
    for k in keys:
        if k in globals():
            snap[k] = safe_jsonable(globals()[k])
    return snap

def stable_hash(obj) -> str:
    payload = json.dumps(safe_jsonable(obj), sort_keys=True,
↳ ensure_ascii=False).encode("utf-8")

```

```

        return hashlib.sha256(payload).hexdigest()

CONFIG_HASH = stable_hash(config_snapshot())

def log_event(event_type: str, seed=None, task_id=None, method=None,
    ↪variant=None, phase=None, payload=None, level="INFO"):
    event = {
        "trace_id": TRACE_ID,
        "run_id": RUN_ID,
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "level": level,
        "event_type": event_type,
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "seed": safe_jsonable(seed),
        "task_id": safe_jsonable(task_id),
        "method": safe_jsonable(method),
        "variant": safe_jsonable(variant),
        "phase": safe_jsonable(phase),
        "config_hash": CONFIG_HASH,
        "payload": safe_jsonable(payload or {}),
    }
    TRACE_EVENTS.append(event)
    with open(TRACE_JSONL_PATH, "a", encoding="utf-8") as f:
        f.write(json.dumps(event, ensure_ascii=False) + "\n")
    return event

class TeeStream:
    def __init__(self, *streams):
        self.streams = streams
    def write(self, data):
        for s in self.streams:
            try:
                s.write(data)
                s.flush()
            except Exception:
                pass
    def flush(self):
        for s in self.streams:
            try:
                s.flush()
            except Exception:
                pass

def start_persistent_logging():
    global _LOG_FILE_HANDLE, _LOGGING_ACTIVE
    if _LOGGING_ACTIVE:

```

```

        return
    _LOG_FILE_HANDLE = open(RAW_CONSOLE_LOG_PATH, "a", buffering=1,
↪encoding="utf-8")
    sys.stdout = TeeStream(_ORIGINAL_STDOUT, _LOG_FILE_HANDLE)
    sys.stderr = TeeStream(_ORIGINAL_STDERR, _LOG_FILE_HANDLE)
    _LOGGING_ACTIVE = True
    log_event("run_logging_started", payload={"raw_console_log":
↪str(RAW_CONSOLE_LOG_PATH)})

def stop_persistent_logging():
    global _LOG_FILE_HANDLE, _LOGGING_ACTIVE
    if not _LOGGING_ACTIVE:
        return
    log_event("run_logging_stopped", payload={"raw_console_log":
↪str(RAW_CONSOLE_LOG_PATH)})
    sys.stdout = _ORIGINAL_STDOUT
    sys.stderr = _ORIGINAL_STDERR
    try:
        _LOG_FILE_HANDLE.close()
    except Exception:
        pass
    _LOG_FILE_HANDLE = None
    _LOGGING_ACTIVE = False

def file_sha256(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):
            h.update(chunk)
    return h.hexdigest()

def write_manifest(extra=None):
    artifacts = []
    if RESULTS_DIR.exists():
        for p in sorted(RESULTS_DIR.rglob("*")):
            if p.is_file() and p != MANIFEST_PATH:
                try:
                    artifacts.append({
                        "path": str(p.relative_to(RESULTS_DIR)),
                        "size_bytes": int(p.stat().st_size),
                        "sha256": file_sha256(p),
                    })
                except Exception as e:
                    artifacts.append({"path": str(p), "error": repr(e)})
    manifest = {
        "run_id": RUN_ID,
        "trace_id": TRACE_ID,
    }

```

```

        "created_utc": datetime.now(timezone.utc).isoformat(),
        "notebook_version": "MMALS_v1.0_RC1b_Observability_Audit_Patch",
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "device": str(DEVICE),
        "python": sys.version,
        "platform": platform.platform(),
        "config_hash": CONFIG_HASH,
        "config": config_snapshot(),
        "persistence_root": str(PERSISTENCE_ROOT),
        "results_dir": str(RESULTS_DIR),
        "drive_available": DRIVE_AVAILABLE,
        "artifacts": artifacts,
        "extra": safe_jsonable(extra or {}),
    }
    with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
        json.dump(manifest, f, indent=2, ensure_ascii=False)
    return manifest

log_event("run_initialized", payload={"results_dir": str(RESULTS_DIR),
    ↪ "drive_available": DRIVE_AVAILABLE})
print("RUN_ID:", RUN_ID)
print("TRACE_ID:", TRACE_ID)
print("RESULTS_DIR:", RESULTS_DIR)
print("TRACE_JSONL_PATH:", TRACE_JSONL_PATH)
print("RAW_CONSOLE_LOG_PATH:", RAW_CONSOLE_LOG_PATH)
print("NOTEBOOK_EXPORT_DIR:", NOTEBOOK_EXPORT_DIR)

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

RUN_ID: MMALS_v11_RC2b_STRICT_FashionMNIST_robust_20260526T012003Z_seeds5

TRACE_ID: a17620d1d0eb77cc

RESULTS_DIR: /content/drive/MyDrive/MMALS/v1_1_RC2b_strict_policy_selection/MMALS_v11_RC2b_STRICT_FashionMNIST_robust_20260526T012003Z_seeds5

TRACE_JSONL_PATH: /content/drive/MyDrive/MMALS/v1_1_RC2b_strict_policy_selection/MMALS_v11_RC2b_STRICT_FashionMNIST_robust_20260526T012003Z_seeds5/traces/otel_style_events_robust.jsonl

RAW_CONSOLE_LOG_PATH: /content/drive/MyDrive/MMALS/v1_1_RC2b_strict_policy_selection/MMALS_v11_RC2b_STRICT_FashionMNIST_robust_20260526T012003Z_seeds5/raw_logs/console_robust.log

NOTEBOOK_EXPORT_DIR: /content/drive/MyDrive/MMALS/v1_1_RC2b_strict_policy_selection/MMALS_v11_RC2b_STRICT_FashionMNIST_robust_20260526T012003Z_seeds5/notebook_exports

1.6 3. Dataset builder: MNIST-family domain-shift overlap-chain with global labels

The benchmark remains the overlapping chain:

$(0,1) \rightarrow (1,2) \rightarrow (2,3) \rightarrow (3,4) \rightarrow (4,5)$

Labels are global class IDs, not local binary task labels.

Supported DATASET_NAME values:

- MNIST: plain MNIST overlap-chain;
- FashionMNIST: plain FashionMNIST overlap-chain;
- RotatedMNIST: MNIST overlap-chain with deterministic task rotations;
- PermutedMNIST: MNIST overlap-chain with deterministic task-specific pixel permutations.

For DATASET_NAME = "RotatedMNIST", each task receives a deterministic rotation:

Task	Classes	Rotation
0	0 vs 1	-30°
1	1 vs 2	-15°
2	2 vs 3	0°
3	3 vs 4	+15°
4	4 vs 5	+30°

For DATASET_NAME = "PermutedMNIST", each task receives a deterministic pixel permutation:

Task	Classes	Permutation seed
0	0 vs 1	100
1	1 vs 2	101
2	2 vs 3	102
3	3 vs 4	103
4	4 vs 5	104

This keeps the original class-incremental overlap-chain stress while adding explicit domain/context shift.

```
[41]: from torchvision.transforms import functional as TF
from torchvision.transforms import InterpolationMode

_PERMUTATION_CACHE = {}

def get_base_transform(name):
    # Do not flatten here. Rotated-MNIST must rotate [1, 28, 28] tensors first.
    # Permuted-MNIST flattens only after applying a deterministic task
    ↪ permutation.
    return transforms.ToTensor()

def load_base_dataset(name):
    transform = get_base_transform(name)
    if name in ["MNIST", "RotatedMNIST", "PermutedMNIST"]:
        train = datasets.MNIST(DATA_ROOT, train=True, download=True,
        ↪ transform=transform)
```

```

        test = datasets.MNIST(DATA_ROOT, train=False, download=True,
↳transform=transform)
        elif name == "FashionMNIST":
            train = datasets.FashionMNIST(DATA_ROOT, train=True, download=True,
↳transform=transform)
            test = datasets.FashionMNIST(DATA_ROOT, train=False, download=True,
↳transform=transform)
        else:
            raise ValueError("DATASET_NAME must be MNIST, FashionMNIST,
↳RotatedMNIST, or PermutedMNIST")
        return train, test

def get_labels(base_dataset):
    if hasattr(base_dataset, "targets"):
        return np.array(base_dataset.targets)
    raise ValueError("Dataset has no targets attribute")

def task_pairs(pair_mode: str):
    if pair_mode == "disjoint_pairs":
        return [(0, 1), (2, 3), (4, 5), (6, 7), (8, 9)]
    if pair_mode == "overlap_chain":
        return [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5)]
    raise ValueError("Unknown PAIR_MODE")

def seen_classes_until(task_id: int):
    classes = set()
    for a, b in task_pairs(PAIR_MODE)[: task_id + 1]:
        classes.add(a)
        classes.add(b)
    return sorted(c for c in classes if c < N_CLASSES)

def task_rotation_angle(task_id: int, dataset_name: str = DATASET_NAME) ->
↳float:
    if dataset_name == "RotatedMNIST":
        if len(ROTATION_ANGLES) != N_TASKS:
            raise ValueError(f"ROTATION_ANGLES must have length {N_TASKS}, got
↳{len(ROTATION_ANGLES)}")
        return float(ROTATION_ANGLES[task_id])
    return 0.0

def task_permutation_seed(task_id: int, dataset_name: str = DATASET_NAME) ->
↳Optional[int]:
    if dataset_name == "PermutedMNIST":
        if len(PERMUTATION_SEEDS) != N_TASKS:
            raise ValueError(f"PERMUTATION_SEEDS must have length {N_TASKS},
↳got {len(PERMUTATION_SEEDS)}")

```

```

        return int(PERMUTATION_SEEDS[task_id])
    return None

def get_task_permutation(task_id: int, input_dim: int = INPUT_DIM, dataset_name:
    ↪ str = DATASET_NAME):
    seed = task_permutation_seed(task_id, dataset_name)
    if seed is None:
        return None
    key = (dataset_name, task_id, int(input_dim), int(seed))
    if key not in _PERMUTATION_CACHE:
        g = torch.Generator()
        g.manual_seed(seed)
        _PERMUTATION_CACHE[key] = torch.randperm(input_dim, generator=g)
    return _PERMUTATION_CACHE[key]

def apply_task_transform(x, task_id: int, dataset_name: str):
    # x arrives as [1, 28, 28]. Return flattened [784].
    if dataset_name == "RotatedMNIST":
        x = TF.rotate(
            x,
            angle=task_rotation_angle(task_id, dataset_name),
            interpolation=InterpolationMode.BILINEAR,
            fill=0.0,
        )
        return x.view(-1).float()

    if dataset_name == "PermutedMNIST":
        x_flat = x.view(-1).float()
        perm = get_task_permutation(task_id, input_dim=x_flat.numel(), ↪
    ↪ dataset_name=dataset_name)
        return x_flat[perm].float()

    return x.view(-1).float()

def subset_pair_dataset_global(
    base_dataset,
    a,
    b,
    task_id: int,
    dataset_name: str,
    max_per_class=None,
    seed=42,
    label_noise=0.0,
    input_noise_std=0.0,
):
    rng = np.random.default_rng(seed)
    labels = get_labels(base_dataset)

```

```

idx_a = np.where(labels == a)[0]
idx_b = np.where(labels == b)[0]

if max_per_class is not None:
    idx_a = rng.choice(idx_a, size=min(max_per_class, len(idx_a)),
↪replace=False)
    idx_b = rng.choice(idx_b, size=min(max_per_class, len(idx_b)),
↪replace=False)

indices = np.concatenate([idx_a, idx_b])
rng.shuffle(indices)

xs, ys = [], []
for idx in indices:
    x, y = base_dataset[int(idx)]
    y = int(y)
    if y >= N_CLASSES:
        continue

    x = apply_task_transform(x, task_id=task_id, dataset_name=dataset_name)

    if input_noise_std > 0:
        x = torch.clamp(x + torch.randn_like(x) * input_noise_std, 0.0, 1.0)

    xs.append(x)
    ys.append(y)

X = torch.stack(xs).float()
y = torch.tensor(ys).long()

if label_noise > 0:
    flip_mask = torch.rand(len(y)) < label_noise
    y[flip_mask & (y == a)] = b
    y[flip_mask & (y == b)] = a

return TensorDataset(X, y)

def build_stress_tasks(name=DATASET_NAME, seed=42):
    train_base, test_base = load_base_dataset(name)
    pairs = task_pairs(PAIR_MODE)
    tasks = []

    for task_id, (a, b) in enumerate(pairs):
        train_ds = subset_pair_dataset_global(
            train_base, a, b,
            task_id=task_id,
            dataset_name=name,

```

```

        max_per_class=TRAIN_PER_CLASS,
        seed=seed + task_id,
        label_noise=LABEL_NOISE_RATE,
        input_noise_std=INPUT_NOISE_STD,
    )
    test_ds = subset_pair_dataset_global(
        test_base, a, b,
        task_id=task_id,
        dataset_name=name,
        max_per_class=TEST_PER_CLASS,
        seed=seed + 100 + task_id,
        label_noise=0.0,
        input_noise_std=INPUT_NOISE_STD,
    )

    angle = task_rotation_angle(task_id, name)
    perm_seed = task_permutation_seed(task_id, name)
    task_name = f"{a} vs {b}"
    if name == "RotatedMNIST":
        task_name += f" @ {angle:+.0f}deg"
    elif name == "PermutedMNIST":
        task_name += f" @ perm_seed={perm_seed}"

    tasks.append({
        "task_id": task_id,
        "name": task_name,
        "pair": (a, b),
        "rotation_angle": angle,
        "permutation_seed": perm_seed,
        "train": train_ds,
        "test": test_ds,
    })

    return tasks

def sample_context_id(task_id: int, n_contexts: int = N_CONTEXTS, noise_rate:
    ↪float = CONTEXT_NOISE_RATE):
    if random.random() >= noise_rate:
        return task_id
    candidates = [i for i in range(n_contexts) if i != task_id]
    return random.choice(candidates)

tasks_preview = build_stress_tasks(DATASET_NAME, seed=0)
[
    (
        t["task_id"],
        t["name"],

```

```

        t["pair"],
        t["rotation_angle"],
        t.get("permutation_seed"),
        len(t["train"]),
        len(t["test"]),
        seen_classes_until(t["task_id"]),
    )
    for t in tasks_preview
]

```

```

[41]: [(0, '0 vs 1', (0, 1), 0.0, None, 2400, 800, [0, 1]),
      (1, '1 vs 2', (1, 2), 0.0, None, 2400, 800, [0, 1, 2]),
      (2, '2 vs 3', (2, 3), 0.0, None, 2400, 800, [0, 1, 2, 3]),
      (3, '3 vs 4', (3, 4), 0.0, None, 2400, 800, [0, 1, 2, 3, 4]),
      (4, '4 vs 5', (4, 5), 0.0, None, 2400, 800, [0, 1, 2, 3, 4, 5])]

```

1.7 4. Model definitions: MLP baseline and MMALS fungal medium

MMALS keeps hosts, transformations, a context encoder, a router, and a mediated latent state **zf**.

This domain-shift notebook does not change the fungal backbone. It freezes the robustly selected RC1b candidate and tests it under rotation-induced or permutation-induced context/domain shift.

```

[42]: class MLPBaseline(nn.Module):
        def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
            ↪latent_dim=LATENT_DIM, n_classes=N_CLASSES):
            super().__init__()
            self.encoder = nn.Sequential(
                nn.Linear(input_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, latent_dim),
                nn.ReLU(),
            )
            self.global_head = nn.Linear(latent_dim, n_classes)

        def forward(self, x, return_cache=False):
            z = self.encoder(x)
            logits = self.global_head(z)
            if not return_cache:
                return logits
            q = torch.full((x.shape[0], N_CONTEXTS), 1.0 / N_CONTEXTS, device=x.
            ↪device)
            cache = {
                "zf": z,
                "routes": None,
                "transformed": None,
                "host_z": None,
            }

```

```

        "q_context": q,
        "q_single": q,
        "context_logits": torch.zeros((x.shape[0], N_CONTEXTS), device=x.
↪device),
        "context_center_logits": torch.zeros((x.shape[0], N_CONTEXTS), ↪
↪device=x.device),
        "context_feat": torch.zeros((x.shape[0], CONTEXT_FEAT_DIM), ↪
↪device=x.device),
        "active_context_mode": "mlp_none",
    }
    return logits, cache

class MLPBlock(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim),
            nn.ReLU(),
        )
    def forward(self, x):
        return self.net(x)

class ContextEncoder(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=128, ↪
↪feat_dim=CONTEXT_FEAT_DIM, n_contexts=N_CONTEXTS):
        super().__init__()
        self.feature_net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, feat_dim),
            nn.ReLU(),
        )
        self.logit_head = nn.Linear(feat_dim, n_contexts)
    def forward(self, x):
        feat = self.feature_net(x)
        logits = self.logit_head(feat)
        return feat, logits

class MMALSClazzIncremental(nn.Module):
    def __init__(self):
        super().__init__()
        self.n_tasks = N_TASKS
        self.n_contexts = N_CONTEXTS
        self.n_hosts = N_HOSTS
        self.n_classes = N_CLASSES

```

```

        self.temperature = TEMPERATURE
        self.context_temperature = CONTEXT_TEMPERATURE
        self.hosts = nn.ModuleList([MLPBlock(INPUT_DIM, HIDDEN_DIM, LATENT_DIM)
↪for _ in range(N_HOSTS)])
        self.transforms = nn.ModuleList([
            nn.Sequential(nn.Linear(LATENT_DIM, LATENT_DIM), nn.ReLU(), nn.
↪Linear(LATENT_DIM, LATENT_DIM))
            for _ in range(N_HOSTS)
        ])
        self.context_encoder = ContextEncoder()
        self.context_centers = nn.Parameter(torch.randn(N_CONTEXTS,
↪CONTEXT_FEAT_DIM) * 0.05)
        self.context_embedding = nn.Embedding(N_CONTEXTS, TASK_EMB_DIM)
        self.router = nn.Sequential(
            nn.Linear(INPUT_DIM + TASK_EMB_DIM, ROUTE_HIDDEN_DIM),
            nn.ReLU(),
            nn.Linear(ROUTE_HIDDEN_DIM, N_HOSTS),
        )
        self.global_head = nn.Linear(LATENT_DIM, N_CLASSES)

    def transform_from_host_latents(self, host_z):
        transformed = [self.transforms[i](host_z[:, i, :]) for i in range(self.
↪n_hosts)]
        return torch.stack(transformed, dim=1)

    def context_probs_from_x(self, x):
        feat, logits = self.context_encoder(x)
        q = F.softmax(logits / self.context_temperature, dim=-1)
        return feat, logits, q

    def context_center_logits(self, feat):
        feat_n = F.normalize(feat, dim=-1)
        centers_n = F.normalize(self.context_centers, dim=-1)
        return (feat_n @ centers_n.T) / CENTER_TEMPERATURE

    def explicit_context_probs(self, batch_size, context_id, device):
        q = torch.zeros(batch_size, self.n_contexts, device=device)
        q[:, context_id] = 1.0
        return q

    def uniform_context_probs(self, batch_size, device):
        return torch.full((batch_size, self.n_contexts), 1.0 / self.n_contexts,
↪device=device)

    def latent_from_context_probs(self, x, q_context):
        context_emb = q_context @ self.context_embedding.weight
        host_z = torch.stack([host(x) for host in self.hosts], dim=1)

```

```

transformed = self.transform_from_host_latents(host_z)
route_input = torch.cat([x, context_emb], dim=1)
route_logits = self.router(route_input)
routes = F.softmax(route_logits / self.temperature, dim=-1)
zf = (routes.unsqueeze(-1) * transformed).sum(dim=1)
return zf, routes, route_logits, host_z, transformed

def forward(self, x, task_id: int, context_mode: str = "oracle", context_id:
↳ Optional[int] = None,
        batch_context_probs: Optional[torch.Tensor] = None,
↳ return_cache: bool = False):
    b = x.shape[0]
    device = x.device
    feat, ctx_logits, q_single = self.context_probs_from_x(x)
    if context_mode in ["oracle", "noisy"]:
        cid = task_id if context_id is None else context_id
        q_context = self.explicit_context_probs(b, cid, device)
    elif context_mode == "uniform":
        q_context = self.uniform_context_probs(b, device)
    elif context_mode == "inferred_single":
        q_context = q_single
    elif context_mode == "inferred_batch":
        q_context = batch_context_probs if batch_context_probs is not None
↳ else q_single
    else:
        raise ValueError("context_mode must be oracle, noisy, uniform,
↳ inferred_single, or inferred_batch")
    zf, routes, route_logits, host_z, transformed = self.
↳ latent_from_context_probs(x, q_context)
    logits = self.global_head(zf)
    if not return_cache:
        return logits
    return logits, {
        "zf": zf,
        "routes": routes,
        "route_logits": route_logits,
        "host_z": host_z,
        "transformed": transformed,
        "context_feat": feat,
        "context_logits": ctx_logits,
        "context_center_logits": self.context_center_logits(feat),
        "q_context": q_context,
        "q_single": q_single,
    }

def build_model(config):
    if config["model_family"] == "mlp":

```

```

        return MLPBaseline().to(DEVICE)
    if config["model_family"] == "mmals":
        return MMALSClassIncremental().to(DEVICE)
    raise ValueError(f"Unknown model_family: {config['model_family']}")

for family in ["mlp", "mmals"]:
    model_preview = build_model({"model_family": family})
    print(family, "trainable parameters:", count_parameters(model_preview),
    ↪ "total parameters:", total_parameters(model_preview), "approx MB total:",
    ↪ round(model_size_mb(model_preview, trainable_only=False), 3))
    del model_preview
clear_memory()

```

mlp trainable parameters: 217798 total parameters: 217798 approx MB total: 0.831
mmals trainable parameters: 1337216 total parameters: 1337216 approx MB total: 5.101

1.8 5. Context prototypes and prototype-first inference

Prototype-first context inference is preserved from v0.14.x and v1.0c. The RC1b candidate changes only validation-selected temperature/blend context posterior parameters and a guarded global read-out; it does not use explicit task IDs at inference.

```

[43]: def entropy_from_probs(q, eps=1e-8):
    return -(q * torch.log(q + eps)).sum(dim=-1)

def margin_from_probs(q):
    top2 = torch.topk(q, k=2, dim=-1).values
    return top2[:, 0] - top2[:, 1]

def brier_from_probs(q, target: int):
    y = torch.full((q.shape[0],), target, dtype=torch.long, device=q.device)
    onehot = F.one_hot(y, num_classes=q.shape[1]).float()
    return ((q - onehot) ** 2).sum(dim=-1)

def ece_from_probs(q, target: int, n_bins: int = 10):
    y = torch.full((q.shape[0],), target, dtype=torch.long, device=q.device)
    conf, pred = torch.max(q, dim=-1)
    correct = (pred == y).float()
    ece = torch.zeros((), device=q.device)
    for lo in torch.linspace(0, 1, n_bins + 1, device=q.device)[:n_bins]:
        hi = lo + 1.0 / n_bins
        mask = (conf > lo) & (conf <= hi)
        if mask.any():
            ece = ece + mask.float().mean() * torch.abs(conf[mask].mean() -
    ↪ correct[mask].mean())
    return ece

```

```

@torch.no_grad()
def gaussian_stats_from_tensor(Z):
    if Z.shape[0] < 2:
        return None
    return {"mean": Z.mean(dim=0).to(DEVICE), "var": Z.var(dim=0,
↳unbiased=False).clamp_min(1e-6).to(DEVICE), "n": Z.shape[0]}

def diag_frechet_distance_from_stats(batch_values, proto_stats):
    mu = batch_values.mean(dim=0)
    var = batch_values.var(dim=0, unbiased=False).clamp_min(1e-6)
    mu_k = proto_stats["mean"].to(batch_values.device)
    var_k = proto_stats["var"].to(batch_values.device)
    return (mu - mu_k).pow(2).sum() + (torch.sqrt(var) - torch.sqrt(var_k)).
↳pow(2).sum()

def frechet_posterior_from_distances(distances, temperature):
    return F.softmax(-distances / temperature, dim=0)

def distance_margin_and_top(distances):
    sorted_d = torch.sort(distances).values
    margin = sorted_d[1] - sorted_d[0] if len(sorted_d) >= 2 else torch.
↳tensor(float("nan"), device=distances.device)
    top_context = torch.argmax(distances)
    return margin, top_context

@torch.no_grad()
def features_and_latents_for_dataset(model, dataset, context_id: int,
↳max_items=PROTOTYPE_MAX_ITEMS, batch_size=BATCH_SIZE):
    model.eval()
    X, _ = dataset.tensors
    if max_items is not None and len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        X = X[idx]
    feats, latents = [], []
    loader = make_loader(TensorDataset(X, torch.zeros(len(X), dtype=torch.
↳long)), batch_size=batch_size, shuffle=False)
    for xb, _ in loader:
        xb = xb.to(DEVICE)
        feat, _, _ = model.context_probs_from_x(xb)
        q_oracle = model.explicit_context_probs(xb.shape[0], context_id, xb.
↳device)
        zf, _, _, _, _ = model.latent_from_context_probs(xb, q_oracle)
        feats.append(feat.detach().cpu())
        latents.append(zf.detach().cpu())
    return torch.cat(feats, dim=0), torch.cat(latents, dim=0)

```

```

@torch.no_grad()
def refresh_context_prototypes(model, memory_buffers: Dict[int, TensorDataset]):
    if not isinstance(model, MMALSClazzIncremental):
        return {}
    prototypes = {}
    for context_id, mem_ds in memory_buffers.items():
        feats, latents = features_and_latents_for_dataset(model, mem_ds,
↪context_id=context_id)
        feat_stats = gaussian_stats_from_tensor(feats)
        latent_stats = gaussian_stats_from_tensor(latents)
        if feat_stats is not None and latent_stats is not None:
            prototypes[context_id] = {"feature": feat_stats, "latent":
↪latent_stats}
    return prototypes

def infer_batch_context_probs(model, x, prototypes, config=None):
    config = config or {}
    blend_single = config.get("batch_blend_single", BATCH_BLEND_SINGLE)
    blend_latent = config.get("batch_blend_latent", BATCH_BLEND_LATENT)
    blend_feature = config.get("batch_blend_feature", BATCH_BLEND_FEATURE)
    context_feat, _, q_single = model.context_probs_from_x(x)
    if not prototypes:
        return q_single, {
            "frechet_margin": torch.full((x.shape[0],), float("nan"), device=x.
↪device),
            "latent_frechet_margin": torch.full((x.shape[0],), float("nan"),
↪device=x.device),
            "feature_frechet_margin": torch.full((x.shape[0],), float("nan"),
↪device=x.device),
            "prototype_weight": torch.full((x.shape[0],), 0.0, device=x.device),
        }
    zf_prelim, _, _, _ = model.latent_from_context_probs(x, q_single)
    feature_distances = torch.full((model.n_contexts,), 1e6, device=x.device)
    latent_distances = torch.full((model.n_contexts,), 1e6, device=x.device)
    for cid, proto in prototypes.items():
        feature_distances[cid] = diag_frechet_distance_from_stats(context_feat,
↪proto["feature"])
        latent_distances[cid] = diag_frechet_distance_from_stats(zf_prelim,
↪proto["latent"])
        q_feature = frechet_posterior_from_distances(feature_distances,
↪FRECHET_TEMPERATURE)
        q_latent = frechet_posterior_from_distances(latent_distances,
↪LATENT_FRECHET_TEMPERATURE)
        proto_mass = max(blend_latent + blend_feature, 1e-8)
        q_proto = (blend_feature * q_feature + blend_latent * q_latent) / proto_mass

```

```

q_batch = q_proto.unsqueeze(0).expand(x.shape[0], -1)
q = blend_single * q_single + (1.0 - blend_single) * q_batch
q = q / q.sum(dim=-1, keepdim=True).clamp_min(1e-8)
f_margin, _ = distance_margin_and_top(feature_distances)
l_margin, _ = distance_margin_and_top(latent_distances)
return q, {
    "frechet_margin": l_margin.expand(x.shape[0]),
    "latent_frechet_margin": l_margin.expand(x.shape[0]),
    "feature_frechet_margin": f_margin.expand(x.shape[0]),
    "prototype_weight": torch.full((x.shape[0],), 1.0 - blend_single,
device=x.device),
}

```

1.9 6. Routing helpers

This preserves uniform, oracle and proto-first routes. The robust validation harness uses the same zf extraction protocol for every cloned variant.

```

[44]: def q_uniform_like(model, x):
    return model.uniform_context_probs(x.shape[0], x.device)

def normalize_probs(q, eps=1e-8):
    return q / q.sum(dim=-1, keepdim=True).clamp_min(eps)

def logits_from_context_probs(model, x, q_context):
    zf, routes, route_logits, host_z, transformed = model.
latent_from_context_probs(x, q_context)
    logits = model.global_head(zf)
    cache = {
        "zf": zf,
        "routes": routes,
        "route_logits": route_logits,
        "host_z": host_z,
        "transformed": transformed,
        "q_context": q_context,
    }
    return logits, cache

def method_context_mode_and_probs(model, x, task_id, config, prototypes=None,
train=False, noisy_context_id=None, y=None):
    if config["model_family"] == "mlp":
        return "mlp_none", None, None, {}
    strategy = config["strategy"]
    if strategy == "oracle":
        return "oracle", None, task_id, {}
    if strategy == "noisy":
        cid = noisy_context_id if noisy_context_id is not None else task_id

```

```

        return "noisy", None, cid, {}
    if strategy == "uniform":
        return "uniform", None, None, {}
    if strategy == "inferred_single":
        return "inferred_single", None, None, {}
    if strategy in ["inferred_batch", "proto_first"]:
        if train:
            return "inferred_single", None, None, {}
        q_batch, batch_info = infer_batch_context_probs(model, x, prototypes or {}, config=config)
        return "inferred_batch", q_batch, None, batch_info
    raise ValueError(f"Unknown strategy: {strategy}")

def forward_by_method(model, x, task_id, config, prototypes=None, train=False, noisy_context_id=None, y=None, return_cache=True):
    if config["model_family"] == "mlp":
        logits, cache = model(x, return_cache=True)
        return (logits, cache) if return_cache else logits
    mode, batch_probs, context_id, batch_info = method_context_mode_and_probs(
        model, x, task_id, config, prototypes=prototypes, train=train,
        noisy_context_id=noisy_context_id, y=y
    )
    logits, cache = model(
        x,
        task_id,
        context_mode=mode,
        context_id=context_id,
        batch_context_probs=batch_probs,
        return_cache=True,
    )
    cache.update(batch_info)
    cache["active_context_mode"] = mode
    return (logits, cache) if return_cache else logits

def probe_config_from_mode(context_mode: str):
    if context_mode == "uniform":
        return {**COMMON_PROTO_FIRST, "strategy": "uniform"}
    if context_mode == "proto":
        return {**COMMON_PROTO_FIRST, "strategy": "proto_first"}
    if context_mode == "oracle":
        return {**COMMON_MMALS, "strategy": "oracle"}
    raise ValueError("context_mode must be uniform, proto, or oracle")

```

1.10 7. Training losses, balanced replay and functional memory

This block retains the v0.14.x continual-learning losses. v1.0-RC1b candidate-freeze validation does not change the base training objective.

```

[45]: def entropy_loss(routes, eps=1e-8):
    if routes is None:
        return torch.zeros((), device=DEVICE)
    return -(routes * torch.log(routes + eps)).sum(dim=1).mean()

def metabolic_loss(transformed, routes):
    if transformed is None or routes is None:
        return torch.zeros((), device=DEVICE)
    return (routes.unsqueeze(-1) * transformed.pow(2)).sum(dim=(1, 2)).mean()

def kl_distill_loss(logits_current, logits_teacher,
    ↪temperature=DISTILL_TEMPERATURE):
    log_p_current = F.log_softmax(logits_current / temperature, dim=-1)
    p_teacher = F.softmax(logits_teacher / temperature, dim=-1)
    return (temperature ** 2) * F.kl_div(log_p_current, p_teacher,
    ↪reduction="batchmean")

def clone_teacher(model):
    teacher = copy.deepcopy(model).to(DEVICE)
    teacher.eval()
    for p in teacher.parameters():
        p.requires_grad_(False)
    return teacher

def build_memory_subset(dataset: TensorDataset, max_items: int, seed: int):
    X, y = dataset.tensors
    rng = np.random.default_rng(seed)
    idx = rng.choice(np.arange(len(X)), size=min(max_items, len(X)),
    ↪replace=False)
    return TensorDataset(X[idx].clone(), y[idx].clone())

def sample_from_tensor_dataset(dataset: TensorDataset, batch_size: int):
    X, y = dataset.tensors
    idx = torch.randint(0, len(X), (min(batch_size, len(X)),))
    return X[idx].to(DEVICE), y[idx].to(DEVICE)

def update_class_memory(class_memory: Dict[int, Dict[str, torch.Tensor]],
    ↪dataset: TensorDataset, task_id: int, max_per_class: int, seed: int):
    X, y = dataset.tensors
    rng = np.random.default_rng(seed)
    new_memory = {k: {kk: vv.clone() for kk, vv in v.items()}} for k, v in
    ↪class_memory.items()
    for cls in sorted(torch.unique(y).tolist()):
        cls = int(cls)
        idx = torch.where(y == cls)[0]
        if len(idx) == 0:
            continue

```

```

X_new = X[idx].clone()
y_new = y[idx].clone()
t_new = torch.full((len(idx),), task_id, dtype=torch.long)
if cls in new_memory:
    X_cat = torch.cat([new_memory[cls]["X"], X_new], dim=0)
    y_cat = torch.cat([new_memory[cls]["y"], y_new], dim=0)
    t_cat = torch.cat([new_memory[cls]["task_id"], t_new], dim=0)
else:
    X_cat, y_cat, t_cat = X_new, y_new, t_new
if len(X_cat) > max_per_class:
    keep = rng.choice(np.arange(len(X_cat)), size=max_per_class,
↪replace=False)
    keep = torch.tensor(keep, dtype=torch.long)
    X_cat, y_cat, t_cat = X_cat[keep], y_cat[keep], t_cat[keep]
    new_memory[cls] = {"X": X_cat, "y": y_cat, "task_id": t_cat}
return new_memory

def sample_balanced_class_memory(class_memory: Dict[int, Dict[str, torch.
↪Tensor]], batch_size: int):
    active_classes = sorted([int(c) for c, data in class_memory.items() if
↪len(data["y"]) > 0])
    if not active_classes:
        return None, None, None
    per_class = max(1, math.ceil(batch_size / len(active_classes)))
    xs, ys, ts = [], [], []
    for cls in active_classes:
        data = class_memory[cls]
        n = len(data["y"])
        k = min(per_class, n)
        idx = torch.randint(0, n, (k,))
        xs.append(data["X"][idx])
        ys.append(data["y"][idx])
        ts.append(data["task_id"][idx])
    X = torch.cat(xs, dim=0)
    y = torch.cat(ys, dim=0)
    t = torch.cat(ts, dim=0)
    perm = torch.randperm(len(y))[: min(batch_size, len(y))]
    return X[perm].to(DEVICE), y[perm].to(DEVICE), t[perm].to(DEVICE)

def balanced_replay_ce_loss(model, class_memory, config, prototypes=None,
↪batch_size=BALANCED_REPLAY_BATCH):
    zero = torch.zeros((), device=DEVICE)
    if not config.get("balanced_replay_ce", False) or not class_memory:
        return zero, {"balanced_replay_ce_raw": 0.0, "balanced_replay_classes":
↪0, "balanced_replay_items": 0}
    xb, yb, tb = sample_balanced_class_memory(class_memory,
↪batch_size=batch_size)

```

```

    if xb is None:
        return zero, {"balanced_replay_ce_raw": 0.0, "balanced_replay_classes": 0, "balanced_replay_items": 0}
    total_loss = zero
    n_groups = 0
    for src_task_id in torch.unique(tb).detach().cpu().tolist():
        src_task_id = int(src_task_id)
        mask = (tb == src_task_id)
        if not mask.any():
            continue
        logits, _ = forward_by_method(
            model,
            xb[mask],
            src_task_id,
            config,
            prototypes=prototypes,
            train=not BALANCED_REPLAY_USE_EVAL_CONTEXT,
            y=yb[mask],
            return_cache=True,
        )
        total_loss = total_loss + F.cross_entropy(logits, yb[mask])
        n_groups += 1
    raw = total_loss / max(n_groups, 1)
    weight = float(config.get("balanced_replay_weight", BALANCED_REPLAY_WEIGHT))
    return weight * raw, {
        "balanced_replay_ce_raw": float(raw.detach().cpu()),
        "balanced_replay_classes": len(class_memory),
        "balanced_replay_items": int(sum(len(v["y"]) for v in class_memory.
values())),
    }

def context_supervision_weight(config, task_id: int, epoch: int):
    if not config.get("context_calibrated", False):
        return 0.0
    total_steps = max(N_TASKS * EPOCHS_PER_TASK - 1, 1)
    step = task_id * EPOCHS_PER_TASK + epoch
    progress = min(max(step / total_steps, 0.0), 1.0)
    return CONTEXT_SUP_END + (CONTEXT_SUP_START - CONTEXT_SUP_END) * ((1.0 -
progress) ** CONTEXT_SUP_GAMMA)

def context_calibration_loss(model, cache, task_id: int, config, epoch: int):
    zero = torch.zeros((), device=DEVICE)
    if not config.get("context_calibrated", False) or config["model_family"] != "mmals":
        return zero, {"ctx_weight": 0.0, "ctx_ce": 0.0}
    target = torch.full((cache["context_logits"].shape[0],), task_id,
dtype=torch.long, device=DEVICE)

```

```

    weight = context_supervision_weight(config, task_id, epoch)
    ce = F.cross_entropy(cache["context_logits"], target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
    center_ce = F.cross_entropy(model.
↪context_center_logits(cache["context_feat"]), target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
    entropy = entropy_from_probs(cache["q_single"]).mean()
    margin_penalty = F.relu(CONTEXT_MARGIN_TARGET -
↪margin_from_probs(cache["q_single"])).mean()
    loss = weight * (ce + LAMBDA_CONTEXT_CENTER * center_ce) +
↪LAMBDA_CONTEXT_ENTROPY * entropy + LAMBDA_CONTEXT_MARGIN * margin_penalty
    return loss, {"ctx_weight": float(weight), "ctx_ce": float(ce.detach().
↪cpu())}

def context_replay_loss(model, teacher, memory_buffers, config,
↪batch_size=CONTEXT_REPLAY_BATCH):
    zero = torch.zeros((), device=DEVICE)
    if config["model_family"] != "mmals" or not config.get("context_replay",
↪False) or not memory_buffers:
        return zero, {"ctx_replay_raw": 0.0, "ctx_replay_scale": config.
↪get("context_replay_scale", 0.0)}
    ce_total = zero
    center_total = zero
    kl_total = zero
    feat_total = zero
    n_terms = 0
    for old_context_id, mem_ds in memory_buffers.items():
        xb, _ = sample_from_tensor_dataset(mem_ds, batch_size=batch_size)
        feat_cur, logits_cur, _ = model.context_probs_from_x(xb)
        target = torch.full((xb.shape[0],), old_context_id, dtype=torch.long,
↪device=DEVICE)
        ce_total = ce_total + F.cross_entropy(logits_cur, target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        center_total = center_total + F.cross_entropy(model.
↪context_center_logits(feat_cur), target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        if teacher is not None and isinstance(teacher, MMALSClazzIncremental):
            with torch.no_grad():
                feat_t, logits_t, _ = teacher.context_probs_from_x(xb)
                kl_total = kl_total + kl_distill_loss(logits_cur, logits_t,
↪temperature=CONTEXT_DISTILL_TEMPERATURE)
        feat_total = feat_total + F.mse_loss(feat_cur, feat_t)
    n_terms += 1
    ce_loss = ce_total / max(n_terms, 1)
    center_loss = center_total / max(n_terms, 1)

```

```

    kl_loss = kl_total / max(n_terms, 1) if teacher is not None and
    ↪ isinstance(teacher, MMALSClazzIncremental) else zero
    feat_loss = feat_total / max(n_terms, 1) if teacher is not None and
    ↪ isinstance(teacher, MMALSClazzIncremental) else zero
    raw = LAMBDA_CONTEXT_REPLAY_CE * ce_loss + LAMBDA_CONTEXT_REPLAY_CENTER *
    ↪ center_loss + LAMBDA_CONTEXT_TEACHER_KL * kl_loss +
    ↪ LAMBDA_CONTEXT_FEATURE_DRIFT * feat_loss
    scale = float(config.get("context_replay_scale", 1.0))
    return scale * raw, {"ctx_replay_raw": float(raw.detach().cpu()),
    ↪ "ctx_replay_scale": scale}

def old_memory_losses(model, teacher, memory_buffers, config, batch_size,
    ↪ prototypes=None):
    zero = torch.zeros((), device=DEVICE)
    if teacher is None or not memory_buffers:
        return zero, {"route": 0.0, "latent": 0.0, "output": 0.0}
    route_total = zero
    latent_total = zero
    output_total = zero
    n_terms = 0
    for old_task_id, mem_ds in memory_buffers.items():
        xb, yb = sample_from_tensor_dataset(mem_ds, batch_size=batch_size)
        logits_cur, cache_cur = forward_by_method(model, xb, old_task_id,
    ↪ config, prototypes=prototypes, train=False, y=yb, return_cache=True)
        with torch.no_grad():
            logits_t, cache_t = forward_by_method(teacher, xb, old_task_id,
    ↪ config, prototypes=prototypes, train=False, y=yb, return_cache=True)
            if config["model_family"] == "mmals":
                route_total = route_total + F.mse_loss(cache_cur["routes"],
    ↪ cache_t["routes"])
                latent_total = latent_total + F.mse_loss(cache_cur["zf"],
    ↪ cache_t["zf"])
            if config.get("output_distill", False) or config.get("output", 0.0) > 0:
                output_total = output_total + kl_distill_loss(logits_cur, logits_t)
            n_terms += 1
    route_loss = route_total / max(n_terms, 1)
    latent_loss = latent_total / max(n_terms, 1)
    output_loss = output_total / max(n_terms, 1)
    loss = config.get("route", 0.0) * route_loss + config.get("latent", 0.0) *
    ↪ latent_loss + config.get("output", 0.0) * output_loss
    if config["model_family"] == "mlp" and config.get("output_distill", False):
        loss = output_loss
    return loss, {"route": float(route_loss.detach().cpu()), "latent":
    ↪ float(latent_loss.detach().cpu()), "output": float(output_loss.detach().
    ↪ cpu())}

```

```

@torch.no_grad()
def calibrate_context_temperature(model, memory_buffers, config,
    ↪candidate_temps=CONTEXT_TEMP_GRID, max_items=TEMP_CALIB_MAX_ITEMS):
    if config["model_family"] != "mmals" or not config.get("temp_calibration",
    ↪False) or not memory_buffers:
        return float(getattr(model, "context_temperature", float("nan")))
    saved_temp = float(model.context_temperature)
    logits_all, targets_all = [], []
    model.eval()
    for cid, mem_ds in memory_buffers.items():
        X, _ = mem_ds.tensors
        if len(X) > max_items:
            X = X[torch.randperm(len(X))[:max_items]]
            loader = make_loader(TensorDataset(X, torch.zeros(len(X), dtype=torch.
    ↪long)), batch_size=BATCH_SIZE, shuffle=False)
            for xb, _ in loader:
                xb = xb.to(DEVICE)
                _, logits = model.context_encoder(xb)
                logits_all.append(logits.detach())
                targets_all.append(torch.full((xb.shape[0],), cid, dtype=torch.
    ↪long, device=DEVICE))
            if not logits_all:
                return saved_temp
            logits_all = torch.cat(logits_all, dim=0)
            targets_all = torch.cat(targets_all, dim=0)
            best_temp, best_loss = saved_temp, float("inf")
            for temp in candidate_temps:
                loss = F.cross_entropy(logits_all / temp, targets_all).item()
                if loss < best_loss:
                    best_loss = loss
                    best_temp = float(temp)
            model.context_temperature = best_temp
            return best_temp

# -----
# v0.14.8 head-repair utilities
# -----

def head_margin_repair_loss(logits, y, target_margin=HEAD_MARGIN_TARGET,
    ↪hard_classes=HARD_CLASSES, ref_class=REFERENCE_CLASS_FOR_MARGIN):
    """Penalize hard-class logits when they fall below the reference class
    ↪logit.

    This is intentionally head-focused: it does not change routes or latent
    ↪states.
    """

```

```

if logits is None or y is None:
    return torch.zeros(), device=DEVICE)
losses = []
for cls in hard_classes:
    if cls >= logits.shape[1] or ref_class >= logits.shape[1]:
        continue
    mask = (y == cls)
    if mask.any():
        margin = logits[mask, cls] - logits[mask, ref_class]
        losses.append(F.relu(target_margin - margin).mean())
if not losses:
    return torch.zeros(), device=logits.device)
return torch.stack(losses).mean()

def class_memory_to_tensor_dataset(class_memory: Dict[int, Dict[str, torch.
    ↪Tensor]]):
    if not class_memory:
        return None
    xs, ys, ts = [], [], []
    for cls, data in sorted(class_memory.items()):
        if len(data["y"]) == 0:
            continue
        xs.append(data["X"])
        ys.append(data["y"])
        ts.append(data["task_id"])
    if not xs:
        return None
    return TensorDataset(torch.cat(xs, dim=0), torch.cat(ys, dim=0), torch.
    ↪cat(ts, dim=0))

@torch.no_grad()
def extract_zf_from_batch_by_task(model, xb, tb, config, prototypes=None):
    """Extract frozen z_f for a mixed replay batch containing items from
    ↪several source tasks."""
    zf_out = torch.zeros((xb.shape[0], LATENT_DIM), device=xb.device)
    for src_task_id in torch.unique(tb).detach().cpu().tolist():
        src_task_id = int(src_task_id)
        mask = (tb == src_task_id)
        if not mask.any():
            continue
        _, cache = forward_by_method(
            model,
            xb[mask],
            src_task_id,
            config,

```

```

        prototypes=prototypes,
        train=False,
        y=None,
        return_cache=True,
    )
    zf_out[mask] = cache["zf"].detach()
    return zf_out

def sample_repair_batch(class_memory, batch_size=HEAD_REPAIR_BATCH):
    xb, yb, tb = sample_balanced_class_memory(class_memory,
    ↪batch_size=batch_size)
    return xb, yb, tb

def train_head_refresh_phase(model, class_memory, config, prototypes=None,
    ↪steps=HEAD_REFRESH_STEPS, epochs=HEAD_REFRESH_EPOCHS, lr=HEAD_REFRESH_LR):
    """Freeze the fungal backbone and update only the deployed global head,
    ↪using balanced memory."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    model.eval()
    opt = torch.optim.Adam(model.global_head.parameters(), lr=lr)
    for epoch in range(epochs):
        sums = {"head_refresh_loss": 0.0, "head_refresh_ce": 0.0,
    ↪"head_refresh_margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(class_memory)
            if xb is None:
                continue
            with torch.no_grad():
                zf = extract_zf_from_batch_by_task(model, xb, tb, config,
    ↪prototypes=prototypes)
            opt.zero_grad(set_to_none=True)
            logits = model.global_head(zf)
            ce = F.cross_entropy(logits, yb)
            margin = head_margin_repair_loss(logits, yb)
            loss = ce + HEAD_MARGIN_LAMBDA * margin
            loss.backward()
            opt.step()
            sums["head_refresh_loss"] += float(loss.detach().cpu())
            sums["head_refresh_ce"] += float(ce.detach().cpu())
            sums["head_refresh_margin"] += float(margin.detach().cpu())
            n += 1
        if n > 0:

```

```

        rows.append({k: v / n for k, v in sums.items()} | {"repair_epoch":
↪epoch, "repair_type": "head_refresh"})
    return rows

def build_memory_feature_dataset(model, class_memory, config, prototypes=None,
↪max_items=None):
    mem_ds = class_memory_to_tensor_dataset(class_memory)
    if mem_ds is None:
        return None
    X, y, t = mem_ds.tensors
    if max_items is not None and len(y) > max_items:
        idx = torch.randperm(len(y))[:max_items]
        X, y, t = X[idx], y[idx], t[idx]
    feats, labels = [], []
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
    model.eval()
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        with torch.no_grad():
            zf = extract_zf_from_batch_by_task(model, xb, tb, config,
↪prototypes=prototypes)
            feats.append(zf.detach().cpu())
            labels.append(yb.detach().cpu())
    return TensorDataset(torch.cat(feats, dim=0).float(), torch.cat(labels,
↪dim=0).long())

def train_temporary_zf_probe(feature_ds, seed=0, epochs=PROBE_EPOCHS):
    if feature_ds is None:
        return None, []
    set_seed(seed + 81000)
    probe = LinearProbe(input_dim=LATENT_DIM).to(DEVICE)
    opt = torch.optim.Adam(probe.parameters(), lr=PROBE_LR)
    loader = make_loader(feature_ds, batch_size=BATCH_SIZE, shuffle=True)
    rows = []
    for epoch in range(epochs):
        total, n = 0.0, 0
        probe.train()
        for zf, yb in loader:
            zf, yb = zf.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(probe(zf), yb)
            loss.backward()
            opt.step()
            total += float(loss.detach().cpu())

```

```

        n += 1
        rows.append({"repair_epoch": epoch, "repair_type": "temporary_probe",
↪ "probe_loss": total / max(n, 1)})
        probe.eval()
        for p in probe.parameters():
            p.requires_grad_(False)
        return probe, rows

def train_probe_distilled_head_phase(model, class_memory, config,
↪ prototypes=None, seed=0, steps=HEAD_REFRESH_STEPS,
↪ epochs=PROBE_DISTILL_EPOCHS, lr=PROBE_DISTILL_LR):
    """Train a temporary linear probe on frozen z_f and distill it into the
↪ deployed global head."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    feature_ds = build_memory_feature_dataset(model, class_memory, config,
↪ prototypes=prototypes)
    probe, probe_rows = train_temporary_zf_probe(feature_ds, seed=seed,
↪ epochs=max(5, PROBE_EPOCHS // 2))
    rows.extend(probe_rows)
    if probe is None:
        return rows
    model.eval()
    opt = torch.optim.Adam(model.global_head.parameters(), lr=lr)
    for epoch in range(epochs):
        sums = {"probe_distill_loss": 0.0, "probe_distill_ce": 0.0,
↪ "probe_distill_kl": 0.0, "probe_distill_margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(class_memory)
            if xb is None:
                continue
            with torch.no_grad():
                zf = extract_zf_from_batch_by_task(model, xb, tb, config,
↪ prototypes=prototypes)
                probe_logits = probe(zf)
                opt.zero_grad(set_to_none=True)
                logits = model.global_head(zf)
                ce = F.cross_entropy(logits, yb)
                kl = kl_distill_loss(logits, probe_logits,
↪ temperature=PROBE_DISTILL_TEMPERATURE)
                margin = head_margin_repair_loss(logits, yb)
                loss = PROBE_DISTILL_ALPHA_CE * ce + PROBE_DISTILL_ALPHA_KL * kl +
↪ HEAD_MARGIN_LAMBDA * margin

```

```

        loss.backward()
        opt.step()
        sums["probe_distill_loss"] += float(loss.detach().cpu())
        sums["probe_distill_ce"] += float(ce.detach().cpu())
        sums["probe_distill_kl"] += float(kl.detach().cpu())
        sums["probe_distill_margin"] += float(margin.detach().cpu())
        n += 1
    if n > 0:
        rows.append({k: v / n for k, v in sums.items()} | {"repair_epoch": epoch, "repair_type": "probe_distill"})
    return rows

def train_bias_calibration_phase(model, class_memory, config, prototypes=None, steps=BIAS_CALIB_STEPS, lr=BIAS_CALIB_LR):
    """Learn a tiny post-hoc class-bias correction on balanced memory and fold it into the global head bias."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    delta = nn.Parameter(torch.zeros(N_CLASSES, device=DEVICE))
    opt = torch.optim.Adam([delta], lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch(class_memory)
        if xb is None:
            continue
        with torch.no_grad():
            zf = extract_zf_from_batch_by_task(model, xb, tb, config, prototypes=prototypes)
            base_logits = model.global_head(zf).detach()
            opt.zero_grad(set_to_none=True)
            logits = base_logits + delta.unsqueeze(0)
            ce = F.cross_entropy(logits, yb)
            margin = head_margin_repair_loss(logits, yb)
            reg = BIAS_CALIB_REG * delta.pow(2).mean()
            loss = ce + HEAD_MARGIN_LAMBDA * margin + reg
            loss.backward()
            opt.step()
            rows.append({
                "repair_epoch": step,
                "repair_type": "bias_calibration",
                "bias_calib_loss": float(loss.detach().cpu()),
                "bias_calib_ce": float(ce.detach().cpu()),
                "bias_calib_margin": float(margin.detach().cpu()),
            })
    with torch.no_grad():
        if model.global_head.bias is not None:

```

```

        model.global_head.bias.add_(delta.detach())
    return rows

def maybe_run_head_repair_phases(model, class_memory, config, prototypes=None,
    ↪seed=0):
    rows = []
    if config["model_family"] != "mmals":
        return rows
    if config.get("post_head_refresh", False):
        rows.extend(train_head_refresh_phase(model, class_memory, config,
    ↪prototypes=prototypes))
    if config.get("post_probe_distill", False):
        rows.extend(train_probe_distilled_head_phase(model, class_memory,
    ↪config, prototypes=prototypes, seed=seed))
    if config.get("post_bias_calibration", False):
        rows.extend(train_bias_calibration_phase(model, class_memory, config,
    ↪prototypes=prototypes))
    return rows

```

1.11 8. Readout and v1.0-RC1b context-posterior utilities

The local readout classes from v1.0a/v1.0b are kept as reusable infrastructure, but the RC1b validation target is **not** a local-head branch.

The object of validation is the inferred-context posterior used to extract z_f , plus a true CE+KL guarded global readout. Deployable variants remain task-free. Oracle variants are diagnostic references only.

```

[46]: def clone_tensor_dataset(ds: TensorDataset):
    tensors = tuple(t.clone().detach().cpu() for t in ds.tensors)
    return TensorDataset(*tensors)

def clone_memory_buffers(memory_buffers: Dict[int, TensorDataset]):
    return {int(k): clone_tensor_dataset(v) for k, v in memory_buffers.items()}

def clone_class_memory(class_memory):
    return {int(k): {kk: vv.clone().detach().cpu() for kk, vv in v.items()} for
    ↪k, v in class_memory.items()}

def clone_prototypes(prototypes):
    out = {}
    for cid, proto in prototypes.items():
        out[int(cid)] = {}
        for kind, stats in proto.items():
            out[int(cid)][kind] = {}
            for sk, sv in stats.items():

```

```

        if torch.is_tensor(sv):
            out[int(cid)][kind][sk] = sv.clone().detach().cpu()
        else:
            out[int(cid)][kind][sk] = copy.deepcopy(sv)
    return out

def cpu_state_dict(model):
    return {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}

def restore_base_model_from_checkpoint(checkpoint, config):
    model = build_model(config)
    model.load_state_dict(checkpoint["model_state"])
    if hasattr(model, "context_temperature"):
        model.context_temperature = float(checkpoint.get("context_temperature",
CONTEXT_TEMPERATURE))
    model.to(DEVICE)
    return model

def split_class_memory_for_readout_validation(class_memory,
val_fraction=VALIDATION_SPLIT_FRACTION, seed=0):
    if not class_memory:
        return {}, {}
    rng = np.random.default_rng(seed)
    repair_memory, validation_memory = {}, {}
    for cls, data in class_memory.items():
        n = len(data["y"])
        if n == 0:
            continue
        idx = np.arange(n)
        rng.shuffle(idx)
        n_val = int(round(n * val_fraction))
        n_val = min(max(n_val, 1 if n >= 4 else 0), max(n - 1, 0))
        val_idx = torch.tensor(idx[:n_val], dtype=torch.long)
        train_idx = torch.tensor(idx[n_val:], dtype=torch.long)
        if len(train_idx) > 0:
            repair_memory[int(cls)] = {k: v[train_idx].clone() for k, v in data.
items()}
        if len(val_idx) > 0:
            validation_memory[int(cls)] = {k: v[val_idx].clone() for k, v in
data.items()}
    if not repair_memory:
        repair_memory = clone_class_memory(class_memory)
    if not validation_memory:
        validation_memory = clone_class_memory(class_memory)
    return repair_memory, validation_memory

@torch.no_grad()

```

```

def collect_zf_memory(model, class_memory, config, prototypes=None,
    ↪max_items=None):
    mem_ds = class_memory_to_tensor_dataset(class_memory)
    if mem_ds is None:
        return None
    X, y, t = mem_ds.tensors
    if max_items is not None and len(y) > max_items:
        idx = torch.randperm(len(y))[:max_items]
        X, y, t = X[idx], y[idx], t[idx]
    Zs, Ys, Ts, Qs = [], [], [], []
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
    ↪shuffle=False)
    model.eval()
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(
                model, xb[mask], src_task_id, config,
                prototypes=prototypes, train=False, y=yb[mask],
    ↪return_cache=True
            )
            Zs.append(cache["zf"].detach().cpu())
            Ys.append(yb[mask].detach().cpu())
            Ts.append(tb[mask].detach().cpu())
            Qs.append(cache["q_context"].detach().cpu())
    if not Zs:
        return None
    return {
        "Z": torch.cat(Zs, dim=0).float(),
        "y": torch.cat(Ys, dim=0).long(),
        "task_id": torch.cat(Ts, dim=0).long(),
        "q_context": torch.cat(Qs, dim=0).float(),
    }

def compute_class_prototypes(memory_features, n_classes=N_CLASSES, eps=1e-8):
    Z = memory_features["Z"]
    y = memory_features["y"]
    mu = torch.zeros(n_classes, Z.shape[1])
    counts = torch.zeros(n_classes)
    valid = torch.zeros(n_classes, dtype=torch.bool)
    for cls in range(n_classes):
        mask = (y == cls)
        if mask.any():

```

```

        mu[cls] = Z[mask].mean(dim=0)
        counts[cls] = float(mask.sum())
        valid[cls] = True
    return mu, valid, counts

def compute_context_class_prototypes(memory_features, n_contexts=N_CONTEXTS,
    ↪n_classes=N_CLASSES):
    Z = memory_features["Z"]
    y = memory_features["y"]
    t = memory_features["task_id"]
    global_mu, global_valid, _ = compute_class_prototypes(memory_features,
    ↪n_classes=n_classes)
    mu = torch.zeros(n_contexts, n_classes, Z.shape[1])
    valid = torch.zeros(n_contexts, n_classes, dtype=torch.bool)
    for cid in range(n_contexts):
        for cls in range(n_classes):
            mask = (t == cid) & (y == cls)
            if mask.any():
                mu[cid, cls] = Z[mask].mean(dim=0)
                valid[cid, cls] = True
            elif global_valid[cls]:
                mu[cid, cls] = global_mu[cls]
                valid[cid, cls] = True
    return mu, valid, global_mu, global_valid

class GlobalLinearReadout(nn.Module):
    def __init__(self, base_head):
        super().__init__()
        self.linear = copy.deepcopy(base_head)
    def forward(self, zf, q_context=None):
        return self.linear(zf)

class PrototypeCosineReadout(nn.Module):
    def __init__(self, prototypes, valid, scale=COSINE_SCALE_INIT, bias=None):
        super().__init__()
        self.register_buffer("prototypes", F.normalize(prototypes.float(),
    ↪dim=-1))
        self.register_buffer("valid", valid.bool())
        self.scale = nn.Parameter(torch.tensor(float(scale)))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES) if bias is None else
    ↪bias.float())
    def forward(self, zf, q_context=None):
        z = F.normalize(zf, dim=-1)
        logits = self.scale.clamp(1.0, 80.0) * (z @ self.prototypes.T) + self.
    ↪bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

```

```

class PrototypeEuclideanReadout(nn.Module):
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP, bias=None):
        super().__init__()
        self.register_buffer("prototypes", prototypes.float())
        self.register_buffer("valid", valid.bool())
        self.log_temperature = nn.Parameter(torch.log(torch.
→tensor(float(temperature))))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES) if bias is None else
→bias.float())
        def forward(self, zf, q_context=None):
            diff = zf.unsqueeze(1) - self.prototypes.unsqueeze(0)
            dist = diff.pow(2).mean(dim=-1)
            temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
            logits = -dist / temp + self.bias
            logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
            return logits

class TrainablePrototypeEuclideanReadout(nn.Module):
    """Trainable prototype readout without per-dimension adapter.

    This separates the effect of learning class centers/bias/temperature from
→the
    stronger diagonal local adapter.
    """
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP):
        super().__init__()
        self.mu = nn.Parameter(prototypes.float().clone())
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
        self.log_temperature = nn.Parameter(torch.log(torch.
→tensor(float(temperature))))
        self.register_buffer("valid", valid.bool())
        def forward(self, zf, q_context=None):
            diff = zf.unsqueeze(1) - self.mu.unsqueeze(0)
            dist = diff.pow(2).mean(dim=-1)
            temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
            logits = -dist / temp + self.bias
            logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
            return logits

class LocalAdapterPrototypeReadout(nn.Module):
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP):
        super().__init__()
        self.mu = nn.Parameter(prototypes.float().clone())
        self.log_diag = nn.Parameter(torch.zeros_like(prototypes.float()))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))

```

```

        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))
        self.register_buffer("valid", valid.bool())
    def forward(self, zf, q_context=None):
        alpha = torch.exp(self.log_diag.clamp(-2.0, 2.0))
        diff = (zf.unsqueeze(1) - self.mu.unsqueeze(0)) * alpha.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class ContextPrototypeCosineReadout(nn.Module):
    def __init__(self, context_prototypes, context_valid,
↪scale=COSINE_SCALE_INIT):
        super().__init__()
        self.register_buffer("context_prototypes", F.
↪normalize(context_prototypes.float(), dim=-1))
        self.register_buffer("context_valid", context_valid.bool())
        self.scale = nn.Parameter(torch.tensor(float(scale)))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
    def forward(self, zf, q_context=None):
        if q_context is None:
            q_context = torch.full((zf.shape[0], N_CONTEXTS), 1.0 / N_CONTEXTS,
↪device=zf.device)
            z = F.normalize(zf, dim=-1)
            # scores: batch x context x class
            scores = torch.einsum("bd,cyd->bcy", z, self.context_prototypes.to(zf.
↪device))
            scores = scores.masked_fill(~self.context_valid.to(zf.device).
↪unsqueeze(0), -1e9)
            logits = (q_context.unsqueeze(-1) * scores).sum(dim=1)
            logits = self.scale.clamp(1.0, 80.0) * logits + self.bias
            return logits

def build_readout_from_memory(model, class_memory, config, prototypes, variant,
↪seed=0):
    set_seed(seed)
    feats = collect_zf_memory(model, class_memory, config,
↪prototypes=prototypes)
    if feats is None:
        return GlobalLinearReadout(model.global_head), [{"repair_type":
↪"fallback_global_empty_memory"}]
    class_mu, class_valid, counts = compute_class_prototypes(feats)
    rows = [{
        "repair_type": "prototype_build",

```

```

        "variant": variant,
        "memory_items": int(len(feats["y"])),
        "valid_classes": int(class_valid.sum().item()),
    }]
    if variant == "local_proto_cosine":
        return PrototypeCosineReadout(class_mu, class_valid).to(DEVICE), rows
    if variant == "local_proto_euclidean":
        return PrototypeEuclideanReadout(class_mu, class_valid).to(DEVICE), rows
    if variant == "local_trainable_proto_ce_kl_035":
        return TrainablePrototypeEuclideanReadout(class_mu, class_valid).
    to(DEVICE), rows
    if variant.startswith("local_diag_adapter") or variant == "
    "local_adapter_proto_ce_kl_035":
        return LocalAdapterPrototypeReadout(class_mu, class_valid).to(DEVICE),
    rows
    if variant == "context_local_proto_cosine":
        ctx_mu, ctx_valid, _gm, _gv = compute_context_class_prototypes(feats)
        return ContextPrototypeCosineReadout(ctx_mu, ctx_valid).to(DEVICE), rows
    raise ValueError(f"Unknown local readout variant: {variant}")

def sample_repair_batch_v10b(class_memory, batch_size=HEAD_REPAIR_BATCH):
    if not class_memory:
        return None, None, None
    return sample_balanced_class_memory(class_memory, batch_size)

def readout_ce_kl_loss(readout, zf, q_context, yb, pre_global_head=None,
    kl_weight=READOUT_KL_WEIGHT, use_kl=True, margin_weight=0.0):
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, yb)
    kl = logits.sum() * 0.0
    if use_kl and pre_global_head is not None and kl_weight > 0:
        with torch.no_grad():
            pre_logits = pre_global_head(zf)
            kl = kl_distill_loss(logits, pre_logits,
            temperature=PROBE_DISTILL_TEMPERATURE)
    margin = head_margin_repair_loss(logits, yb) if margin_weight > 0 else
    logits.sum() * 0.0
    return ce + kl_weight * kl + margin_weight * margin, {"ce": ce, "kl": kl,
    "margin": margin}, logits

def train_readout_on_memory(readout, model, class_memory, config,
    prototypes=None, pre_global_head=None,
    epochs=READOUT_TRAIN_EPOCHS,
    steps=READOUT_TRAIN_STEPS,
    lr=READOUT_LR, kl_weight=READOUT_KL_WEIGHT,
    use_kl=True,

```

```

margin_weight=0.0, variant="readout_train"):
rows = []
if not class_memory:
    return rows
# Only train readout parameters. The fungal backbone remains frozen.
model.eval()
for p in model.parameters():
    p.requires_grad_(False)
params = [p for p in readout.parameters() if p.requires_grad]
if not params:
    return rows
opt = torch.optim.Adam(params, lr=lr)
for epoch in range(epochs):
    sums = {"loss": 0.0, "ce": 0.0, "kl": 0.0, "margin": 0.0}
    n = 0
    for _ in range(steps):
        xb, yb, tb = sample_repair_batch_v10b(class_memory)
        if xb is None:
            continue
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        zf_list, q_list, y_list = [], [], []
        with torch.no_grad():
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                zf_list.append(cache["zf"].detach())
                q_list.append(cache["q_context"].detach())
                y_list.append(yb[mask].detach())
            if not zf_list:
                continue
            zf = torch.cat(zf_list, dim=0)
            q_context = torch.cat(q_list, dim=0)
            y_mix = torch.cat(y_list, dim=0)
            opt.zero_grad(set_to_none=True)
            loss, details, _ = readout_ce_kl_loss(readout, zf, q_context,
↪y_mix, pre_global_head=pre_global_head, kl_weight=kl_weight, use_kl=use_kl,
↪margin_weight=margin_weight)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
            opt.step()
            sums["loss"] += float(loss.detach().cpu())
            sums["ce"] += float(details["ce"].detach().cpu())
            sums["kl"] += float(details["kl"].detach().cpu())

```

```

        sums["margin"] += float(details["margin"].detach().cpu())
        n += 1
    if n > 0:
        rows.append({
            "repair_epoch": epoch,
            "repair_type": variant,
            "readout_train_loss": sums["loss"] / n,
            "readout_train_ce": sums["ce"] / n,
            "readout_train_kl": sums["kl"] / n,
            "readout_train_margin": sums["margin"] / n,
            "kl_weight": kl_weight,
            "margin_weight": margin_weight,
        })
    return rows

@torch.no_grad()
def proxy_metrics_for_readout(model, readout, class_memory, config,
    ↪ prototypes=None, max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    if not class_memory:
        return {"proxy_acc": np.nan, "proxy_class2": np.nan, "proxy_class5": np.
    ↪ nan, "proxy_min_class": np.nan}
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    model.eval(); readout.eval()
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
    ↪ shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
    ↪ config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                logits = readout(cache["zf"], cache["q_context"])
                seen = seen_classes_until(src_task_id)
                pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
                yy = yb[mask]
                for c in torch.unique(yy).detach().cpu().tolist():

```

```

        c = int(c)
        m = (yy == c)
        total_by_cls[c] += int(m.sum().item())
        correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
    per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↪total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
    total = sum(total_by_cls.values())
    correct = sum(correct_by_cls.values())
    valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
    return {
        "proxy_acc": correct / max(total, 1),
        "proxy_class2": per_cls.get(2, np.nan),
        "proxy_class5": per_cls.get(5, np.nan),
        "proxy_min_class": float(np.min(valid_accs)) if valid_accs else np.nan,
    }

def validation_score(candidate, baseline):
    if candidate is None:
        return -1e9
    acc = candidate.get("proxy_acc", np.nan)
    c2 = candidate.get("proxy_class2", np.nan)
    minc = candidate.get("proxy_min_class", np.nan)
    base_acc = baseline.get("proxy_acc", np.nan)
    if np.isnan(acc):
        return -1e9
    damage = max(0.0, (base_acc if not np.isnan(base_acc) else acc) - acc)
    return float(
        acc
        + VALIDATION_SCORE_CLASS2_WEIGHT * (0.0 if np.isnan(c2) else c2)
        + VALIDATION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(minc) else minc)
        - VALIDATION_SCORE_DAMAGE_WEIGHT * damage
    )

def build_and_train_readout_variant(model, class_memory, config, prototypes,
↪variant, seed=0):
    """Return a readout and logs for one paired local-head variant."""
    set_seed(seed)
    repair_memory, validation_memory =
↪split_class_memory_for_readout_validation(class_memory, seed=seed + 100)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)
    rows = []

    if variant in ["no_repair_global_head", "kl_only_true_noop"]:
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)

```

```

        rows.append({"repair_type": "no_op", "variant": variant})
        return readout, rows

    if variant == "global_head_ce_kl_guard_035":
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True,
↪variant="global_head_ce_kl_guard_035"))
        return readout, rows

    if variant in ["local_proto_cosine", "local_proto_euclidean",
↪"context_local_proto_cosine"]:
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, variant, seed=seed)
        rows.extend(build_rows)
        return readout, rows

    if variant == "local_adapter_proto_ce_kl_035":
        # Backward-compatible alias from v1.0a.
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, "local_diag_adapter_ce_kl_035", seed=seed)
        rows.extend(build_rows)
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True,
↪margin_weight=0.0, variant="local_adapter_proto_ce_kl_035"))
        return readout, rows

    if variant == "local_trainable_proto_ce_kl_035":
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, variant, seed=seed)
        rows.extend(build_rows)
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True,
↪margin_weight=0.0, variant=variant))
        return readout, rows

    if variant.startswith("local_diag_adapter_ce"):
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, variant, seed=seed)
        rows.extend(build_rows)
        if variant == "local_diag_adapter_ce_only":
            kl_weight, use_kl, margin_weight = 0.0, False, 0.0
        elif variant == "local_diag_adapter_ce_kl_005":

```

```

        kl_weight, use_kl, margin_weight = 0.05, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_015":
        kl_weight, use_kl, margin_weight = 0.15, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_035":
        kl_weight, use_kl, margin_weight = 0.35, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_060":
        kl_weight, use_kl, margin_weight = 0.60, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_margin_035":
        kl_weight, use_kl, margin_weight = 0.35, True,
↪LOCAL_ADAPTER_MARGIN_WEIGHT
    else:
        raise ValueError(f"Unknown local diagonal adapter variant:
↪{variant}")
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
                                                kl_weight=kl_weight, use_kl=use_kl,
↪margin_weight=margin_weight, variant=variant))
        return readout, rows

    if variant in ["local_mycorrhizal_selected",
↪"local_adapter_validation_selected"]:
        baseline_readout = GlobalLinearReadout(model.global_head).to(DEVICE)
        baseline_proxy = proxy_metrics_for_readout(model, baseline_readout,
↪validation_memory, config, prototypes=prototypes)
        candidates = [
            "global_head_ce_kl_guard_035",
            "local_proto_euclidean",
            "local_trainable_proto_ce_kl_035",
            "local_diag_adapter_ce_only",
            "local_diag_adapter_ce_kl_005",
            "local_diag_adapter_ce_kl_015",
            "local_diag_adapter_ce_kl_035",
            "local_diag_adapter_ce_kl_060",
            "local_diag_adapter_ce_kl_margin_035",
        ]
        best_readout = baseline_readout
        best_name = "no_repair_global_head"
        best_proxy = baseline_proxy
        best_score = validation_score(baseline_proxy, baseline_proxy)
        rows.append({"repair_type": "selector_baseline", "candidate":
↪best_name, **{f"baseline_{k}": v for k, v in baseline_proxy.items()}},
↪{"validation_score": best_score})
        for cand in candidates:
            cand_readout, cand_rows = build_and_train_readout_variant(model,
↪repair_memory, config, prototypes, cand, seed=seed + 17)

```

```

        proxy = proxy_metrics_for_readout(model, cand_readout,
↪validation_memory, config, prototypes=prototypes)
        score = validation_score(proxy, baseline_proxy)
        rows.extend([**r, "candidate": cand} for r in cand_rows])
        rows.append({
            "repair_type": "selector_candidate",
            "candidate": cand,
            "proxy_acc": proxy.get("proxy_acc", np.nan),
            "proxy_class2": proxy.get("proxy_class2", np.nan),
            "proxy_class5": proxy.get("proxy_class5", np.nan),
            "proxy_min_class": proxy.get("proxy_min_class", np.nan),
            "validation_score": score,
        })
        if score > best_score + VALIDATION_ACCEPTANCE_TOL:
            best_score = score
            best_readout = cand_readout
            best_name = cand
            best_proxy = proxy
        rows.append({
            "repair_type": "selector_selected",
            "selected_candidate": best_name,
            "selected_score": best_score,
            "selected_proxy_acc": best_proxy.get("proxy_acc", np.nan),
            "selected_proxy_class2": best_proxy.get("proxy_class2", np.nan),
            "selected_proxy_class5": best_proxy.get("proxy_class5", np.nan),
            "selected_proxy_min_class": best_proxy.get("proxy_min_class", np.
↪nan),
        })
        return best_readout, rows

    raise ValueError(f"Unknown readout variant: {variant}")

# -----
# v1.0c inferred-context / proto-to-oracle gap closure utilities
# -----

def method_name_for_context_variant(variant: str):
    return f"paired_{variant}"

def make_context_config(base_strategy="proto_first", temp=None,
↪blend_tuple=None):
    """Create an evaluation/training config for a context posterior variant.

    temp is applied to model.context_temperature outside this config.
    blend_tuple is (name, single_weight, latent_weight, feature_weight).
    """

```

```

    if base_strategy == "oracle":
        cfg = copy.
    ↪deepcopy(METHOD_CONFIGS["mmals_oracle_context_balanced_replay"])
        return cfg
    cfg = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    cfg["strategy"] = "proto_first"
    if blend_tuple is not None:
        _name, bs, bl, bf = blend_tuple
        s = max(float(bs) + float(bl) + float(bf), 1e-8)
        cfg["batch_blend_single"] = float(bs) / s
        cfg["batch_blend_latent"] = float(bl) / s
        cfg["batch_blend_feature"] = float(bf) / s
        cfg["context_blend_name"] = str(_name)
    if temp is not None:
        cfg["selected_context_temperature"] = float(temp)
    return cfg

def set_model_context_temperature(model, temp):
    if hasattr(model, "context_temperature") and temp is not None:
        model.context_temperature = float(temp)

@torch.no_grad()
def proxy_metrics_for_context_config(model, class_memory, config,
    ↪prototypes=None, readout=None,
    ↪
    ↪max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    """Score a context-posterior config on memory validation, without final
    ↪test data.

    This combines readout accuracy on memory, context top-1 against stored
    ↪source task,
    context entropy/margin, class-2 accuracy, class-5 preservation and
    ↪min-class robustness.
    """
    if not class_memory:
        return {
            "proxy_acc": np.nan,
            "proxy_context_top1": np.nan,
            "proxy_context_entropy": np.nan,
            "proxy_context_margin": np.nan,
            "proxy_class2": np.nan,
            "proxy_class5": np.nan,
            "proxy_min_class": np.nan,
            "proxy_score": -1e9,

```

```

    }
    if readout is None:
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    model.eval(); readout.eval()
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    correct_total, total = 0, 0
    top1_values, ent_values, margin_values = [], [], []
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(
                    model, xb[mask], src_task_id, config,
                    prototypes=prototypes, train=False, y=yb[mask],
↪return_cache=True
                )
                logits = readout(cache["zf"], cache["q_context"])
                seen = seen_classes_until(src_task_id)
                pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
                yy = yb[mask]
                correct = (pred == yy)
                correct_total += int(correct.sum().item())
                total += int(yy.numel())
                q = cache["q_context"]
                top1_values.append((q.argmax(dim=-1) == src_task_id).float().
↪detach().cpu())
                ent_values.append(entropy_from_probs(q).detach().cpu())
                margin_values.append(margin_from_probs(q).detach().cpu())
                for c in torch.unique(yy).detach().cpu().tolist():
                    c = int(c)
                    m = (yy == c)
                    total_by_cls[c] += int(m.sum().item())
                    correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())

```

```

    per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↪total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
    valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
    acc = correct_total / max(total, 1)
    ctx_top1 = float(torch.cat(top1_values).mean()) if top1_values else np.nan
    ctx_entropy = float(torch.cat(ent_values).mean()) if ent_values else np.nan
    ctx_margin = float(torch.cat(margin_values).mean()) if margin_values else
↪np.nan
    min_class = float(np.min(valid_accs)) if valid_accs else np.nan
    c2 = per_cls.get(2, np.nan)
    c5 = per_cls.get(5, np.nan)
    score = float(
        acc
        + CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT * (0.0 if np.isnan(ctx_top1)
↪else ctx_top1)
        + CONTEXT_SELECTION_SCORE_CLASS2_WEIGHT * (0.0 if np.isnan(c2) else c2)
        + CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(min_class)
↪else min_class)
        - CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY * (0.0 if np.
↪isnan(ctx_entropy) else ctx_entropy)
    )
    return {
        "proxy_acc": acc,
        "proxy_context_top1": ctx_top1,
        "proxy_context_entropy": ctx_entropy,
        "proxy_context_margin": ctx_margin,
        "proxy_class2": c2,
        "proxy_class5": c5,
        "proxy_min_class": min_class,
        "proxy_score": score,
    }

def candidate_context_configs(selection_mode="temp_blend"):
    """Return candidate (name, temp, blend_tuple) triples for context selection.
    ↪"""
    base_blend = ("base_proto_first", PROTO_FIRST_BLEND_SINGLE,
↪PROTO_FIRST_BLEND_LATENT, PROTO_FIRST_BLEND_FEATURE)
    if selection_mode == "temp":
        return [(f"temp_{t}", t, base_blend) for t in
↪CONTEXT_TEMP_SELECTION_GRID]
    if selection_mode == "blend":
        return [(f"blend_{name}", CONTEXT_TEMPERATURE, (name, bs, bl, bf)) for
↪name, bs, bl, bf in CONTEXT_BLEND_SELECTION_GRID]
    if selection_mode == "temp_blend":
        # Cross-product is deliberately compact to keep the audit tractable.

```

```

        temps = [0.60, 0.80, 1.00, 1.25]
        blends = CONTEXT_BLEND_SELECTION_GRID
        return [(f"temp_{t}__blend_{name}", t, (name, bs, bl, bf)) for t in
↪ temps for name, bs, bl, bf in blends]
        raise ValueError("selection_mode must be temp, blend, or temp_blend")

def select_context_config_on_memory(model, class_memory, base_config,
↪ prototypes, selection_mode="temp", seed=0):
    """Select a context posterior config on held-out memory, never final test
↪ data."""
    set_seed(seed)
    _repair_memory, validation_memory =
↪ split_class_memory_for_readout_validation(class_memory, seed=seed + 303)
    saved_temp = float(getattr(model, "context_temperature",
↪ CONTEXT_TEMPERATURE))
    baseline_readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    best = None
    rows = []
    for cand_name, temp, blend in candidate_context_configs(selection_mode):
        set_model_context_temperature(model, temp)
        cfg = make_context_config(base_strategy="proto_first", temp=temp,
↪ blend_tuple=blend)
        proxy = proxy_metrics_for_context_config(model, validation_memory, cfg,
↪ prototypes=prototypes, readout=baseline_readout)
        row = {
            "repair_type": "context_config_candidate",
            "candidate": cand_name,
            "selection_mode": selection_mode,
            "candidate_temp": float(temp),
            "candidate_blend_name": str(blend[0]),
            "candidate_blend_single": float(cfg.get("batch_blend_single", np.
↪ nan)),
            "candidate_blend_latent": float(cfg.get("batch_blend_latent", np.
↪ nan)),
            "candidate_blend_feature": float(cfg.get("batch_blend_feature", np.
↪ nan)),
            **proxy,
        }
        rows.append(row)
        if best is None or proxy["proxy_score"] > best["proxy_score"]:
            best = {**row, "config": cfg, "temp": float(temp), "blend": blend}
    if best is None:
        set_model_context_temperature(model, saved_temp)
        return copy.deepcopy(base_config), [{"repair_type":
↪ "context_config_fallback", "selection_mode": selection_mode}]

```

```

set_model_context_temperature(model, best["temp"])
rows.append({
    "repair_type": "context_config_selected",
    "selection_mode": selection_mode,
    "selected_candidate": best["candidate"],
    "selected_temp": best["temp"],
    "selected_blend_name": best["candidate_blend_name"],
    "selected_blend_single": best["candidate_blend_single"],
    "selected_blend_latent": best["candidate_blend_latent"],
    "selected_blend_feature": best["candidate_blend_feature"],
    "selected_proxy_score": best["proxy_score"],
    "selected_proxy_acc": best["proxy_acc"],
    "selected_proxy_context_top1": best["proxy_context_top1"],
    "selected_proxy_class2": best["proxy_class2"],
    "selected_proxy_min_class": best["proxy_min_class"],
})
return best["config"], rows

```

1.12 9. Evaluation metrics and class diagnostics

The scorecard keeps the v0.14.x/v1.0 diagnostics: final average accuracy, minimum task accuracy, forgetting, class-2/class-4 ambiguity, class-5 preservation, predicted-class-4 bias, readout parameter cost, and probe gaps.

```

[47]: def mask_logits_to_seen_classes(logits, seen_classes):
    mask = torch.full_like(logits, -1e9)
    mask[:, seen_classes] = 0.0
    return logits + mask

@torch.no_grad()
def evaluate_task_with_readout(model, readout, dataset, task_id, config,
    ⇨ prototypes=None, batch_size=BATCH_SIZE, seen_classes=None):
    model.eval(); readout.eval()
    loader = make_loader(dataset, batch_size=batch_size, shuffle=False)
    correct, total = 0, 0
    context_entropies, context_margins, context_top1 = [], [], []
    context_briers, context_eces = [], []
    frechet_margins, latent_frechet_margins, prototype_weights = [], [], []
    class2_margin_24, class5_margin_54 = [], []
    class2_logit_rank, class5_logit_rank = [], []
    if seen_classes is None:
        seen_classes = list(range(N_CLASSES))
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        _, cache = forward_by_method(model, xb, task_id, config,
    ⇨ prototypes=prototypes, train=False, y=yb, return_cache=True)
        logits = readout(cache["zf"], cache["q_context"])

```

```

logits_eval = mask_logits_to_seen_classes(logits, seen_classes)
pred = logits_eval.argmax(dim=-1)
correct += (pred == yb).sum().item()
total += yb.numel()
q = cache["q_context"]
context_entropies.append(entropy_from_probs(q).detach().cpu())
context_margins.append(margin_from_probs(q).detach().cpu())
context_top1.append((q.argmax(dim=-1) == task_id).float().detach().
↪cpu())
context_briers.append(brier_from_probs(q, task_id).detach().cpu())
context_eces.append(torch.tensor([float(ece_from_probs(q, task_id).
↪detach().cpu()))))
    if "frechet_margin" in cache:
        frechet_margins.append(cache["frechet_margin"].detach().cpu())
    if "latent_frechet_margin" in cache:
        latent_frechet_margins.append(cache["latent_frechet_margin"].
↪detach().cpu())
    if "prototype_weight" in cache:
        prototype_weights.append(cache["prototype_weight"].detach().cpu())
    if (yb == 2).any():
        m = (yb == 2)
        class2_margin_24.append((logits_eval[m, 2] - logits_eval[m, 4]).
↪detach().cpu())
        ranks = (logits_eval[m] > logits_eval[m, 2].unsqueeze(-1)).
↪sum(dim=-1) + 1
        class2_logit_rank.append(ranks.float().detach().cpu())
    if (yb == 5).any():
        m = (yb == 5)
        class5_margin_54.append((logits_eval[m, 5] - logits_eval[m, 4]).
↪detach().cpu())
        ranks = (logits_eval[m] > logits_eval[m, 5].unsqueeze(-1)).
↪sum(dim=-1) + 1
        class5_logit_rank.append(ranks.float().detach().cpu())
    return {
        "acc": correct / max(total, 1),
        "context_entropy": float(torch.cat(context_entropies).mean()) if
↪context_entropies else np.nan,
        "context_margin": float(torch.cat(context_margins).mean()) if
↪context_margins else np.nan,
        "context_top1_acc": float(torch.cat(context_top1).mean()) if
↪context_top1 else np.nan,
        "context_brier": float(torch.cat(context_briers).mean()) if
↪context_briers else np.nan,
        "context_ece": float(torch.cat(context_eces).mean()) if context_eces
↪else np.nan,

```

```

        "latent_frechet_margin": float(torch.cat(latent_frechet_margins).
↪mean()) if latent_frechet_margins else np.nan,
        "prototype_weight": float(torch.cat(prototype_weights).mean()) if
↪prototype_weights else 0.0,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_logit_rank).mean()) if
↪class2_logit_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_logit_rank).mean()) if
↪class5_logit_rank else np.nan,
    }

@torch.no_grad()
def evaluate_seen_tasks_with_readout(model, readout, tasks, upto_task, config,
↪prototypes=None, batch_size=BATCH_SIZE):
    seen = seen_classes_until(upto_task)
    return [
        evaluate_task_with_readout(model, readout, tasks[tid]["test"], tid,
↪config, prototypes=prototypes, batch_size=batch_size, seen_classes=seen)
        for tid in range(upto_task + 1)
    ]

def forgetting_from_history(history_df: pd.DataFrame):
    rows = []
    for (method, seed), group in history_df.groupby(["method", "seed"]):
        for after_task in sorted(group.after_task.unique()):
            sub = group[group.after_task == after_task]
            fs = []
            for task_id in sorted(sub.task_id.unique()):
                task_hist = group[(group.task_id == task_id) & (group.
↪after_task <= after_task)]
                best = task_hist.acc.max()
                current = sub[sub.task_id == task_id].acc.iloc[0]
                if task_id < after_task:
                    fs.append(best - current)
            rows.append({"method": method, "seed": seed, "after_task":
↪after_task, "avg_forgetting": float(np.mean(fs)) if fs else 0.0})
    return pd.DataFrame(rows)

@torch.no_grad()
def class_confusion_matrix_with_readout(model, readout, tasks, upto_task,
↪config, prototypes=None, batch_size=BATCH_SIZE):
    model.eval(); readout.eval()
    seen = seen_classes_until(upto_task)

```

```

mat = torch.zeros(N_CLASSES, N_CLASSES, dtype=torch.long)
for task_id in range(upto_task + 1):
    loader = make_loader(tasks[task_id]["test"], batch_size=batch_size,
⇨shuffle=False)
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        _, cache = forward_by_method(model, xb, task_id, config,
⇨prototypes=prototypes, train=False, y=yb, return_cache=True)
        logits = readout(cache["zf"], cache["q_context"])
        logits_eval = mask_logits_to_seen_classes(logits, seen)
        pred = logits_eval.argmax(dim=-1).detach().cpu()
        yb_cpu = yb.detach().cpu()
        for yt, yp in zip(yb_cpu, pred):
            if int(yt) < N_CLASSES and int(yp) < N_CLASSES:
                mat[int(yt), int(yp)] += 1

rows = []
for true_class in range(N_CLASSES):
    denom = mat[true_class].sum().item()
    for pred_class in range(N_CLASSES):
        rows.append({
            "true_class": true_class,
            "pred_class": pred_class,
            "count": int(mat[true_class, pred_class].item()),
            "rate": float(mat[true_class, pred_class].item() / max(denom,
⇨1)),
        })
return pd.DataFrame(rows)

def class_diagnostics_from_confusion(confusion_df: pd.DataFrame):
    if confusion_df is None or confusion_df.empty:
        return pd.DataFrame()
    rows = []
    for (method, seed), sub in confusion_df.groupby(["method", "seed"]):
        diag = {}
        for cls in range(N_CLASSES):
            m = sub[(sub.true_class == cls) & (sub.pred_class == cls)]
            diag[cls] = float(m.rate.iloc[0]) if not m.empty else np.nan
        pred4 = sub[sub.pred_class == 4].groupby("true_class")["rate"].sum()
        pred4_bias = float(pred4.reindex(range(N_CLASSES), fill_value=0.0).
⇨mean())
        rows.append({
            "method": method,
            "seed": seed,
            "old_class_acc": float(np.nanmean([diag.get(0, np.nan), diag.get(1,
⇨np.nan), diag.get(2, np.nan)])),
            "recent_class_acc": float(np.nanmean([diag.get(3, np.nan), diag.
⇨get(4, np.nan), diag.get(5, np.nan)])),

```

```

        "class0_acc": diag.get(0, np.nan),
        "class1_acc": diag.get(1, np.nan),
        "class2_acc": diag.get(2, np.nan),
        "class3_acc": diag.get(3, np.nan),
        "class4_acc": diag.get(4, np.nan),
        "class5_acc": diag.get(5, np.nan),
        "min_class_acc": float(np.nanmin([diag.get(c, np.nan) for c in
↪range(N_CLASSES)])),
        "pred4_bias": pred4_bias,
    })
    return pd.DataFrame(rows)

```

1.13 10. Paired same-backbone training and RC1b validation protocol

The protocol is inherited from v0.14.8e, v0.14.9, v1.0a, v1.0b, v1.0c-control-fix and the v1.0-RC1 robust harness, but the goal is now candidate freeze:

1. Train one proto-first MMALS backbone per seed.
2. Save checkpoints after every task.
3. Clone the same checkpoint.
4. Keep the fungal backbone fixed.
5. Compare no-repair, no-op, true global guard, context-only, rejected original RC1 diagnostic, RC1b context+guard candidate, and oracle references.
6. Evaluate whether the RC1b candidate remains superior or clearly robustness-preferable across several seeds.

This is not a new full-training recipe. It is a corrected freeze/validate harness for the first v1.0 benchmark candidate.

```

[48]: def train_base_proto_first_checkpoints(seed: int):
    """Train one proto-first MMALS backbone and save checkpoints after every_
↪task."""
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    config = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = build_model(config)
    memory_buffers: Dict[int, TensorDataset] = {}
    class_memory: Dict[int, Dict[str, torch.Tensor]] = {}
    context_prototypes: Dict[int, Dict[str, torch.Tensor]] = {}
    teacher: Optional[nn.Module] = None
    loss_rows, checkpoints = [], []
    started = time.time()
    for task_id, task in enumerate(tasks):
        opt = torch.optim.Adam(model.parameters(), lr=LR)
        loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)
        for epoch in range(EPOCHS_PER_TASK):
            model.train()

```

```

        sums = {"loss": 0.0, "task": 0.0, "memory": 0.0, "context": 0.0,
↪ "replay": 0.0, "balanced": 0.0, "fungal": 0.0}
        n_batches = 0
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            noisy_context_id = sample_context_id(task_id)
            opt.zero_grad(set_to_none=True)
            logits, cache = forward_by_method(
                model, xb, task_id, config,
                prototypes=context_prototypes, train=True,
                noisy_context_id=noisy_context_id, y=yb, return_cache=True
            )
            task_loss = F.cross_entropy(logits, yb)
            fungal_loss = LAMBDA_ENTROPY * entropy_loss(cache["routes"]) +
↪ LAMBDA_METABOLIC * metabolic_loss(cache["transformed"], cache["routes"])
            context_loss, _ = context_calibration_loss(model, cache,
↪ task_id, config, epoch)
            replay_loss, _ = context_replay_loss(model, teacher,
↪ memory_buffers, config, batch_size=min(CONTEXT_REPLAY_BATCH,
↪ MEMORY_PER_TASK))
            memory_loss, _ = old_memory_losses(model, teacher,
↪ memory_buffers, config, batch_size=min(BATCH_SIZE, MEMORY_PER_TASK),
↪ prototypes=context_prototypes)
            balanced_loss, _ = balanced_replay_ce_loss(model, class_memory,
↪ config, prototypes=context_prototypes, batch_size=BALANCED_REPLAY_BATCH)
            loss = task_loss + fungal_loss + context_loss + replay_loss +
↪ memory_loss + balanced_loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
            opt.step()
            sums["loss"] += float(loss.detach().cpu())
            sums["task"] += float(task_loss.detach().cpu())
            sums["memory"] += float(memory_loss.detach().cpu())
            sums["context"] += float(context_loss.detach().cpu())
            sums["replay"] += float(replay_loss.detach().cpu())
            sums["balanced"] += float(balanced_loss.detach().cpu())
            sums["fungal"] += float(fungal_loss.detach().cpu())
            n_batches += 1
        loss_rows.append({
            "method": "base_proto_first_training",
            "seed": seed,
            "task_id": task_id,
            "epoch": epoch,
            "loss": sums["loss"] / max(n_batches, 1),
            "task_loss": sums["task"] / max(n_batches, 1),
            "memory_loss": sums["memory"] / max(n_batches, 1),

```

```

        "context_loss": sums["context"] / max(n_batches, 1),
        "context_replay_loss": sums["replay"] / max(n_batches, 1),
        "balanced_replay_loss": sums["balanced"] / max(n_batches, 1),
        "fungal_loss": sums["fungal"] / max(n_batches, 1),
        "context_temperature": float(getattr(model,
↪"context_temperature", np.nan)),
    })
    print(
        f"[base_proto seed={seed}] task={task_id} epoch={epoch+1}/
↪{EPOCHS_PER_TASK} "
        f"loss={sums['loss']/max(n_batches,1):.4f} task={sums['task']/
↪max(n_batches,1):.4f} "
        f"memory={sums['memory']/max(n_batches,1):.4f}
↪balanced={sums['balanced']/max(n_batches,1):.4f}"
    )
    memory_buffers[task_id] = build_memory_subset(task["train"],
↪MEMORY_PER_TASK, seed=seed + 1000 + task_id)
    class_memory = update_class_memory(class_memory, task["train"],
↪task_id, MEMORY_PER_CLASS, seed=seed + 2000 + task_id)
    context_prototypes = refresh_context_prototypes(model, memory_buffers)
    chosen_temp = calibrate_context_temperature(model, memory_buffers,
↪config)
    if config.get("temp_calibration", False):
        print(f"[base_proto seed={seed}] calibrated context temperature
↪after task {task_id}: {chosen_temp:.2f}")
        checkpoints.append({
            "seed": seed,
            "after_task": task_id,
            "model_state": cpu_state_dict(model),
            "context_temperature": float(getattr(model, "context_temperature",
↪CONTEXT_TEMPERATURE)),
            "memory_buffers": clone_memory_buffers(memory_buffers),
            "class_memory": clone_class_memory(class_memory),
            "context_prototypes": clone_prototypes(context_prototypes),
            "replay_memory_items": int(sum(len(v["y"]) for v in class_memory.
↪values()))),
        })
    teacher = clone_teacher(model)
    clear_memory()
    print(f"[base_proto seed={seed}] completed in {time.time() - started:.1f}s")
    return {
        "seed": seed,
        "tasks": tasks,
        "config": config,
        "checkpoints": checkpoints,
        "loss_df": pd.DataFrame(loss_rows),

```

```

}

def method_name_for_variant(variant: str):
    return f"paired_{variant}"

def apply_local_head_variant_to_checkpoint(base_run, checkpoint, variant: str):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    config = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    method_name = method_name_for_variant(variant)
    model = restore_base_model_from_checkpoint(checkpoint, config)
    class_memory = clone_class_memory(checkpoint["class_memory"])
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])
    readout, repair_rows = build_and_train_readout_variant(
        model, class_memory, config, context_prototypes,
        variant=variant, seed=seed * 10_000 + after_task * 101
    )
    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "variant": variant,
            "paired_same_backbone": True,
            "local_mycorrhizal_head": not variant.startswith("global") and
variant not in ["no_repair_global_head", "kl_only_true_noop"],
        })
    evals = evaluate_seen_tasks_with_readout(model, readout, tasks,
upto_task=after_task, config=config, prototypes=context_prototypes)
    history_rows = []
    replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
    readout_trainable = count_parameters(readout)
    readout_total = total_parameters(readout)
    for old_task_id, metrics in enumerate(evals):
        row = {
            "method": method_name,
            "seed": seed,
            "model_family": "mmals_local_readout",
            "after_task": after_task,
            "task_id": old_task_id,
            "task_name": tasks[old_task_id]["name"],
            "seen_classes": str(seen_classes_until(after_task)),
            "trainable_params": count_parameters(model) + readout_trainable,
            "total_params": total_parameters(model) + readout_total,
            "backbone_params": total_parameters(model),

```

```

        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) * 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": not variant.startswith("global") and
variant not in ["no_repair_global_head", "kl_only_true_noop"],
        "uses_global_head_refresh": variant ==
"global_head_ce_kl_guard_035",
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_custom_readout",
        "probe_context_mode": "proto_first",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": variant,
    }
    row.update(metrics)
    history_rows.append(row)
    confusion_rows = []
    if after_task == N_TASKS - 1:
        cm_df = class_confusion_matrix_with_readout(model, readout, tasks,
upto_task=after_task, config=config, prototypes=context_prototypes)
        cm_df["method"] = method_name
        cm_df["seed"] = seed
        cm_df["after_task"] = after_task
        cm_df["backbone_source"] = "same_proto_first_checkpoint"
        cm_df["readout_variant"] = variant
        confusion_rows = cm_df.to_dict("records")
        print(f"[{method_name}] seed={seed} after task {after_task}:
{[round(m['acc'], 3) for m in evals]}")
        return pd.DataFrame(history_rows), pd.DataFrame(confusion_rows), pd.
DataFrame(repair_rows)

def final_probe_artifact_from_base_run(base_run):
    final_cp = base_run["checkpoints"][-1]
    model = restore_base_model_from_checkpoint(final_cp,
METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    return {
        "method": "mmals_proto_first_balanced_replay",
        "seed": int(final_cp["seed"]),
        "model": model,
        "tasks": base_run["tasks"],
    }

```

```

        "config": METHOD_CONFIGS["mmals_proto_first_balanced_replay"],
        "memory_buffers": clone_memory_buffers(final_cp["memory_buffers"]),
        "class_memory": clone_class_memory(final_cp["class_memory"]),
        "context_prototypes": clone_prototypes(final_cp["context_prototypes"]),
        "replay_memory_items": int(final_cp.get("replay_memory_items", 0)),
    }

# -----
# v1.0c paired context-head variant application
# -----

def context_variant_selection_mode(variant: str):
    if "temp_blend" in variant or variant == "context_gap_selected":
        return "temp_blend"
    if "blend_selected" in variant:
        return "blend"
    if "temp_selected" in variant:
        return "temp"
    return None

def is_oracle_context_reference(variant: str):
    return variant.startswith("oracle_")

def variant_uses_guarded_head(variant: str):
    """Return True only for variants that must actually train a guarded global_
    ↪readout.

    v1.0c evidence exposed a control issue: the control named
    `proto_global_head_ce_kl_guard_035` did not contain the exact substring
    `head_guard_035`, so the previous helper treated it as a no-op.

    This control-fix accepts both naming conventions:
    - `head_guard_035` for context-selected guarded variants;
    - `global_head_ce_kl_guard_035` for the true global guarded control and_
    ↪oracle reference.
    """
    return (
        "head_guard_035" in variant
        or "global_head_ce_kl_guard_035" in variant
        or variant == "context_gap_selected"
    )

def build_context_variant_config_and_logs(model, class_memory,
    ↪context_prototypes, variant, seed=0):

```

```

"""Return config/logs for one deployable or reference context variant."""
if is_oracle_context_reference(variant):
    cfg = make_context_config(base_strategy="oracle")
    return cfg, [{"repair_type": "context_reference", "variant": variant,
↪ "uses_oracle_context_reference": True}]

    cfg = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    mode = context_variant_selection_mode(variant)
    if mode is None:
        return cfg, [{"repair_type": "context_no_change", "variant": variant,
↪ "uses_oracle_context_reference": False}]

    selected_cfg, rows = select_context_config_on_memory(model, class_memory,
↪ cfg, context_prototypes, selection_mode=mode, seed=seed)
    for r in rows:
        r.update({"variant": variant, "uses_oracle_context_reference": False})
    return selected_cfg, rows

def apply_context_head_variant_to_checkpoint(base_run, checkpoint, variant:
↪ str):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    method_name = method_name_for_context_variant(variant)

    # Start from the exact same checkpoint.
    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    class_memory = clone_class_memory(checkpoint["class_memory"])
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    # Select / set the context posterior configuration.
    config, config_rows = build_context_variant_config_and_logs(
        model, class_memory, context_prototypes, variant=variant,
        seed=seed * 10_000 + after_task * 313
    )

    # Build readout. v1.0-RC1b primarily uses global readout controls; local
↪ adapters are not the target here.
    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)

    if variant_uses_guarded_head(variant):
        repair_memory, _validation_memory =
↪ split_class_memory_for_readout_validation(class_memory, seed=seed * 10_000 +
↪ after_task * 101)
        pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)

```

```

pre_global_head.eval()
for p in pre_global_head.parameters():
    p.requires_grad_(False)
repair_rows.extend(train_readout_on_memory(
    readout, model, repair_memory, config,
    prototypes=context_prototypes, pre_global_head=pre_global_head,
    kl_weight=0.35, use_kl=True, margin_weight=0.0,
    variant=f"{variant}_global_head_guard"
))

for rr in repair_rows:
    rr.update({
        "method": method_name,
        "seed": seed,
        "after_task": after_task,
        "task_id": after_task,
        "variant": variant,
        "paired_same_backbone": True,
        "context_gap_closure_audit": True,
        "uses_oracle_context_reference": □
    })
    is_oracle_context_reference(variant),
    "uses_context_selection": context_variant_selection_mode(variant) □
    is not None,
    "uses_guarded_global_head": variant_uses_guarded_head(variant),
    "true_guard_control_fixed": variant == □
    "proto_global_head_ce_kl_guard_035",
    })

evals = evaluate_seen_tasks_with_readout(
    model, readout, tasks, upto_task=after_task,
    config=config, prototypes=context_prototypes
)

history_rows = []
replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
readout_trainable = count_parameters(readout)
readout_total = total_parameters(readout)
for old_task_id, metrics in enumerate(evals):
    row = {
        "method": method_name,
        "seed": seed,
        "model_family": "mmals_context_readout",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
    }

```

```

        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) * 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
    )
    if is_oracle_context_reference(variant):
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_context_variant",
        "probe_context_mode": "oracle_reference" if is_oracle_context_reference(variant) else "proto_first_variant",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": variant,
        "context_variant": variant,
        "context_temperature_used": float(getattr(model, "context_temperature", np.nan)),
        "batch_blend_single_used": float(config.get("batch_blend_single", np.nan)),
        "batch_blend_latent_used": float(config.get("batch_blend_latent", np.nan)),
        "batch_blend_feature_used": float(config.get("batch_blend_feature", np.nan))
    }
    row.update(metrics)
    history_rows.append(row)

confusion_rows = []
if after_task == N_TASKS - 1:
    cm_df = class_confusion_matrix_with_readout(
        model, readout, tasks, upto_task=after_task,
        config=config, prototypes=context_prototypes
    )
    cm_df["method"] = method_name
    cm_df["seed"] = seed
    cm_df["after_task"] = after_task

```

```

cm_df["backbone_source"] = "same_proto_first_checkpoint"
cm_df["readout_variant"] = variant
cm_df["context_variant"] = variant
cm_df["uses_oracle_context_reference"] =
↪is_oracle_context_reference(variant)
confusion_rows = cm_df.to_dict("records")

print(f"[{method_name} seed={seed}] after task {after_task}:\n
↪{[round(m['acc'], 3) for m in evals]}")
return pd.DataFrame(history_rows), pd.DataFrame(confusion_rows), pd.
↪DataFrame(repair_rows)

```

1.14 10b. RC2b strict no-leakage policy-selection utilities

These functions implement the strict RC2b rule.

The important rule is:

Candidate policy selection uses only memory-derived repair/context/policy-validation partitions. Final test sets are evaluated only after the policy has already been selected.

For auditability, the notebook can still evaluate all candidate policies after the selection decision. Those candidate final-test metrics are diagnostic only and are not used by the selector.

```

[49]: # -----
# v1.1-RC2b strict no-leakage policy-selection utilities.
# -----

def rc2b_candidate_method_name(variant: str) -> str:
    return method_name_for_context_variant(variant)

def rc2b_policy_family(variant: str) -> str:
    if variant in ["proto_global_no_repair", "proto_kl_only_true_noop"]:
        return "proto_no_repair"
    if variant in ["context_blend_selected_global",
↪"context_temp_blend_selected_global"]:
        return "context_only_global"
    if variant == "proto_global_head_ce_kl_guard_035":
        return "proto_true_global_guard"
    if variant in ["context_temp_blend_selected_head_guard_035",
↪"context_gap_selected"]:
        return "rc1b_guarded_context_gap"
    return "other"

def rc2b_policy_label(variant: str) -> str:
    labels = {

```

```

        "proto_no_repair": "proto-first no-repair / copied global head",
        "context_only_global": "validation-selected context-only global head",
        "proto_true_global_guard": "proto-first true CE/KL guarded global head",
        "rc1b_guarded_context_gap": "RC1b guarded context-gap",
        "other": f"other policy: {variant}",
    }
    return labels.get(rc2b_policy_family(variant), str(variant))

def rc2b_policy_complexity_rank(variant: str) -> int:
    # Lower is simpler. Used only as a tie-breaker after validation score.
    fam = rc2b_policy_family(variant)
    if fam == "proto_no_repair":
        return 0
    if fam == "context_only_global":
        return 1
    if fam == "proto_true_global_guard":
        return 2
    if fam == "rc1b_guarded_context_gap":
        return 3
    return 9

def split_class_memory_three_way(class_memory, seed=0,
                                repair_fraction=RC2B_POLICY_REPAIR_FRACTION,
                                context_fraction=RC2B_CONTEXT_SELECTION_FRACTION):
    """Split class memory into repair, context-selection and policy-validation
    memory.

    This is the core RC2b no-leakage guard:
    - repair_memory trains guarded heads;
    - context_selection_memory selects context temp/blend when needed;
    - policy_validation_memory selects the deployable policy;
    - final test is not touched until after selection.
    """
    rng = np.random.default_rng(seed)
    repair, context_select, policy_val = {}, {}, {}
    for cls, data in class_memory.items():
        X, y, t = data["X"], data["y"], data["task_id"]
        n = len(y)
        if n == 0:
            continue
        idx = np.arange(n)
        rng.shuffle(idx)
        n_repair = max(1, int(round(n * repair_fraction))) if n >= 3 else
        max(1, n - 1)

```

```

        n_context = max(1, int(round(n * context_fraction))) if n - n_repair >= 0
↪2 else max(0, n - n_repair - 1)
        n_repair = min(n_repair, n)
        n_context = min(n_context, max(0, n - n_repair))
        repair_idx = idx[:n_repair]
        context_idx = idx[n_repair:n_repair + n_context]
        policy_idx = idx[n_repair + n_context:]
        if len(policy_idx) == 0:
            # Make sure policy validation exists, even in tiny smoke memory.
            if len(context_idx) > 0:
                policy_idx = context_idx[-1:]
                context_idx = context_idx[:-1]
            elif len(repair_idx) > 1:
                policy_idx = repair_idx[-1:]
                repair_idx = repair_idx[:-1]
        def pack(sub_idx):
            if len(sub_idx) == 0:
                return None
            tid = torch.as_tensor(sub_idx, dtype=torch.long)
            return {"X": X[tid].clone(), "y": y[tid].clone(), "task_id": 0
↪t[tid].clone()}
        r = pack(repair_idx)
        c = pack(context_idx)
        p = pack(policy_idx)
        if r is not None:
            repair[int(cls)] = r
        if c is not None:
            context_select[int(cls)] = c
        if p is not None:
            policy_val[int(cls)] = p
        # Fallback: if context selection is empty because smoke memory is tiny, use
↪policy validation.
        if not context_select:
            context_select = clone_class_memory(policy_val)
        return repair, context_select, policy_val

def rc2b_validation_score(proxy, baseline_proxy, variant: str):
    """Policy-level validation score using memory validation only."""
    base_score = validation_score(proxy, baseline_proxy)
    if base_score <= -1e8:
        return -1e9
    c2 = proxy.get("proxy_class2", np.nan)
    c5 = proxy.get("proxy_class5", np.nan)
    minc = proxy.get("proxy_min_class", np.nan)
    acc = proxy.get("proxy_acc", np.nan)
    penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)

```

```

    floor_penalty = 0.0
    if not np.isnan(c2) and c2 < RC2B_CLASS2_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS2_FLOOR - c2)
    if not np.isnan(c5) and c5 < RC2B_CLASS5_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS5_FLOOR - c5)
    if not np.isnan(minc) and minc < RC2B_MIN_CLASS_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_MIN_CLASS_FLOOR - minc)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if np.isnan(c5)
↪else c5)
    return float(base_score + class5_bonus - penalty - floor_penalty)

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
↪context_selection_memory, policy_validation_memory,
                                         seed=0):
    """Build one candidate policy and score it on policy-validation memory only.
↪"""
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    config, config_rows = build_context_variant_config_and_logs(
        model, context_selection_memory, context_prototypes,
        variant=variant, seed=candidate_seed + 313,
    )

    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)

    if variant_uses_guarded_head(variant):
        repair_rows.extend(train_readout_on_memory(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, pre_global_head=pre_global_head,

```

```

        kl_weight=0.35, use_kl=True, margin_weight=0.0,
        variant=f"{variant}_rc2b_guarded_policy_candidate",
    ))

    proxy = proxy_metrics_for_context_config(
        model, policy_validation_memory, config,
        prototypes=context_prototypes, readout=readout,
        max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
    )

    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": cp_seed,
            "after_task": after_task,
            "task_id": after_task,
            "variant": variant,
            "repair_type": str(rr.get("repair_type", "")),
            "rc2b_strict_candidate": True,
            "rc2b_no_final_test_used_for_selection": True,
            "policy_family": rc2b_policy_family(variant),
            "uses_guarded_global_head": variant_uses_guarded_head(variant),
            "uses_context_selection": context_variant_selection_mode(variant)
        })
        is_oracle_context_reference(variant),
    })

    return {
        "variant": variant,
        "method": method_name,
        "policy_family": rc2b_policy_family(variant),
        "policy_label": rc2b_policy_label(variant),
        "complexity_rank": rc2b_policy_complexity_rank(variant),
        "model": model,
        "readout": readout,
        "config": config,
        "context_prototypes": context_prototypes,
        "proxy": proxy,
        "repair_rows": repair_rows,
    }

def rc2b_history_rows_from_evals(candidate, tasks, checkpoint,
    method_name=None, selected=False):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])

```

```

method = method_name or candidate["method"]
model = candidate["model"]
readout = candidate["readout"]
config = candidate["config"]
context_prototypes = candidate["context_prototypes"]
evals = evaluate_seen_tasks_with_readout(model, readout, tasks,
↳upto_task=after_task, config=config, prototypes=context_prototypes)

history_rows = []
replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
readout_trainable = count_parameters(readout)
readout_total = total_parameters(readout)
for old_task_id, metrics in enumerate(evals):
    row = {
        "method": method,
        "seed": seed,
        "model_family": "mmals_rc2b_policy" if selected else
↳"mmals_rc2b_candidate",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) *
↳4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh":
↳variant_uses_guarded_head(candidate["variant"]),
        "uses_context_selection":
↳context_variant_selection_mode(candidate["variant"]) is not None,
        "uses_oracle_context_reference": False,
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_rc2b_policy",
        "probe_context_mode": "proto_first_rc2b_strict",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": candidate["variant"],
        "context_variant": candidate["variant"],

```

```

        "selected_candidate_method": candidate["method"] if selected else
↪None,
        "rc2b_selected_policy": bool(selected),
        "rc2b_candidate_policy_family": candidate["policy_family"],
        "rc2b_selection_proxy_acc": candidate["proxy"].get("proxy_acc", np.
↪nan),
        "rc2b_selection_score": candidate.get("selection_score", np.nan),
        "rc2b_no_final_test_used_for_selection": True,
        "context_temperature_used": float(getattr(model,
↪"context_temperature", np.nan)),
        "batch_blend_single_used": float(config.get("batch_blend_single",
↪np.nan)),
        "batch_blend_latent_used": float(config.get("batch_blend_latent",
↪np.nan)),
        "batch_blend_feature_used": float(config.get("batch_blend_feature",
↪np.nan)),
    }
    row.update(metrics)
    history_rows.append(row)
    return pd.DataFrame(history_rows)

def rc2b_confusion_rows(candidate, tasks, checkpoint, method_name=None,
↪selected=False):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    if after_task != N_TASKS - 1:
        return pd.DataFrame()
    method = method_name or candidate["method"]
    cm_df = class_confusion_matrix_with_readout(
        candidate["model"], candidate["readout"], tasks,
        upto_task=after_task, config=candidate["config"],
↪prototypes=candidate["context_prototypes"]
    )
    cm_df["method"] = method
    cm_df["seed"] = seed
    cm_df["after_task"] = after_task
    cm_df["backbone_source"] = "same_proto_first_checkpoint"
    cm_df["readout_variant"] = candidate["variant"]
    cm_df["context_variant"] = candidate["variant"]
    cm_df["rc2b_selected_policy"] = bool(selected)
    cm_df["selected_candidate_method"] = candidate["method"] if selected else
↪None
    return cm_df

```

```

def apply_rc2b_strict_policy_to_checkpoint(base_run, checkpoint):
    """Strictly select the deployable policy before final-test evaluation."""
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    class_memory = clone_class_memory(checkpoint["class_memory"])

    repair_memory, context_selection_memory, policy_validation_memory =
↪split_class_memory_three_way(
        class_memory,
        seed=seed * 100_000 + after_task * 10_000 + 777,
    )

    candidates = []
    for variant in RC2B_CANDIDATE_VARIANTS:
        cand = build_rc2b_candidate_from_checkpoint(
            base_run, checkpoint, variant,
            repair_memory=repair_memory,
            context_selection_memory=context_selection_memory,
            policy_validation_memory=policy_validation_memory,
            seed=seed * 10_000 + after_task * 313 +
↪rc2b_policy_complexity_rank(variant),
        )
        candidates.append(cand)

    baseline = next((c for c in candidates if c["variant"] ==
↪"proto_global_no_repair"), candidates[0])
    baseline_proxy = baseline["proxy"]

    selection_rows = []
    for cand in candidates:
        score = rc2b_validation_score(cand["proxy"], baseline_proxy,
↪cand["variant"])
        cand["selection_score"] = score
        p = cand["proxy"]
        selection_rows.append({
            "method": RC2B_METHOD,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "candidate_variant": cand["variant"],
            "candidate_method": cand["method"],
            "policy_family": cand["policy_family"],
            "policy_label": cand["policy_label"],
            "complexity_rank": cand["complexity_rank"],
            "proxy_acc": p.get("proxy_acc", np.nan),
            "proxy_class2": p.get("proxy_class2", np.nan),

```

```

        "proxy_class5": p.get("proxy_class5", np.nan),
        "proxy_min_class": p.get("proxy_min_class", np.nan),
        "proxy_score": score,
        "selection_phase": "candidate_scored_before_final_test",
        "no_final_test_used_for_selection": True,
        "repair_memory_items": int(sum(len(v["y"]) for v in repair_memory.
↪values()))),
        "context_selection_memory_items": int(sum(len(v["y"]) for v in_
↪context_selection_memory.values()))),
        "policy_validation_memory_items": int(sum(len(v["y"]) for v in_
↪policy_validation_memory.values()))),
    })

    # Deterministic tie-break: score descending, then simpler policy, then_
↪proxy accuracy.
    candidates_sorted = sorted(
        candidates,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            -float(c["complexity_rank"]),
            float(c["proxy"].get("proxy_acc", -1e9) if not np.isnan(c["proxy"]).
↪get("proxy_acc", np.nan)) else -1e9),
        ),
        reverse=True,
    )
    best = candidates_sorted[0]
    # If multiple candidates are within the tie tolerance, choose the simpler_
↪policy.
    near_best = [c for c in candidates if c.get("selection_score", -1e9) >=_
↪best.get("selection_score", -1e9) - RC2B_SELECTION_TIE_TOL]
    selected = sorted(near_best, key=lambda c: (c["complexity_rank"],_
↪-float(c["proxy"].get("proxy_acc", -1e9) if not np.isnan(c["proxy"]).
↪get("proxy_acc", np.nan)) else -1e9)))[0]

    for r in selection_rows:
        r["selected_candidate_variant"] = selected["variant"]
        r["selected_candidate_method"] = selected["method"]
        r["selected_policy_family"] = selected["policy_family"]
        r["selected_policy_label"] = selected["policy_label"]
        r["selected_proxy_score"] = selected.get("selection_score", np.nan)
        r["selected_proxy_acc"] = selected["proxy"].get("proxy_acc", np.nan)
        r["is_selected_candidate"] = (r["candidate_variant"] ==_
↪selected["variant"])

    candidate_history, candidate_confusions = [], []
    if RC2B_EVALUATE_ALL_CANDIDATES_AFTER_SELECTION:

```

```

        for cand in candidates:
            h = rc2b_history_rows_from_evals(cand, tasks, checkpoint,
            ↪method_name=cand["method"], selected=False)
            candidate_history.append(h)
            c = rc2b_confusion_rows(cand, tasks, checkpoint,
            ↪method_name=cand["method"], selected=False)
            if not c.empty:
                candidate_confusions.append(c)

        selected_history = rc2b_history_rows_from_evals(selected, tasks,
        ↪checkpoint, method_name=RC2B_METHOD, selected=True)
        selected_confusion = rc2b_confusion_rows(selected, tasks, checkpoint,
        ↪method_name=RC2B_METHOD, selected=True)

        repair_rows = []
        for cand in candidates:
            for rr in cand.get("repair_rows", []):
                rr = dict(rr)
                rr.update({
                    "rc2b_candidate_variant": cand["variant"],
                    "rc2b_selected_candidate_variant": selected["variant"],
                    "rc2b_selected_candidate_method": selected["method"],
                    "rc2b_is_selected_candidate": cand["variant"] ==
            ↪selected["variant"],
                    "rc2b_no_final_test_used_for_selection": True,
                })
                repair_rows.append(rr)

        selection_df = pd.DataFrame(selection_rows)
        candidate_history_df = pd.concat(candidate_history, ignore_index=True) if
        ↪candidate_history else pd.DataFrame()
        candidate_confusion_df = pd.concat(candidate_confusions, ignore_index=True)
        ↪if candidate_confusions else pd.DataFrame()
        selected_confusion_df = selected_confusion if not selected_confusion.empty
        ↪else pd.DataFrame()
        repair_df = pd.DataFrame(repair_rows)

        print(
            f"[RC2b seed={seed}] after task {after_task}: selected
            ↪{selected['variant']} "
            f"proxy_acc={selected['proxy'].get('proxy_acc', np.nan):.4f} "
            f"score={selected.get('selection_score', np.nan):.4f}"
        )
        return selected_history, selected_confusion_df, candidate_history_df,
        ↪candidate_confusion_df, repair_df, selection_df

```

1.15 11. Final frozen representation probes

The probes remain the diagnostic ceiling. If the deployed candidate stays close to the proto-context **zf** probe while the oracle-context probe remains higher, the remaining frontier is context inference / context memory rather than readout micro-repair.

```
[50]: class LinearProbe(nn.Module):
    def __init__(self, input_dim, n_classes=N_CLASSES):
        super().__init__()
        self.linear = nn.Linear(input_dim, n_classes)
    def forward(self, x):
        return self.linear(x)

def freeze_model(model):
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    return model

@torch.no_grad()
def extract_mmals_features(model, dataset, task_id, context_mode, prototypes,
    ↪feature_kind="zf", max_items=None, batch_size=BATCH_SIZE):
    model.eval()
    config = probe_config_from_mode(context_mode)
    X, y = dataset.tensors
    if max_items is not None and len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        X = X[idx]
        y = y[idx]
    feats, labels, tasks = [], [], []
    loader = make_loader(TensorDataset(X, y), batch_size=batch_size,
    ↪shuffle=False)
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        logits, cache = forward_by_method(model, xb, task_id, config,
    ↪prototypes=prototypes, train=False, y=yb, return_cache=True)
        if feature_kind == "zf":
            f = cache["zf"]
        elif feature_kind == "context_feat":
            f = cache["context_feat"]
        elif feature_kind.startswith("host"):
            hid = int(feature_kind.replace("host", ""))
            f = cache["host_z"][:, hid, :]
        elif feature_kind.startswith("transformed"):
            hid = int(feature_kind.replace("transformed", ""))
            f = cache["transformed"][:, hid, :]
        elif feature_kind == "routes":
            f = cache["routes"]
```

```

        else:
            raise ValueError(f"Unknown feature_kind: {feature_kind}")
        feats.append(f.detach().cpu())
        labels.append(yb.detach().cpu())
        tasks.append(torch.full((yb.shape[0],), task_id, dtype=torch.long))
        return torch.cat(feats, dim=0).float(), torch.cat(labels, dim=0).long(),
        ↪torch.cat(tasks, dim=0).long()

def build_probe_dataset(model, tasks, prototypes, context_mode="proto",
    ↪feature_kind="zf", train=True, max_per_task=PROBE_MAX_PER_TASK):
    Xs, ys, ts = [], [], []
    key = "train" if train else "test"
    for task_id, task in enumerate(tasks):
        X, y, t = extract_mmals_features(
            model, task[key], task_id, context_mode=context_mode,
            prototypes=prototypes, feature_kind=feature_kind,
            max_items=max_per_task if train else None,
        )
        Xs.append(X)
        ys.append(y)
        ts.append(t)
    return TensorDataset(torch.cat(Xs, dim=0), torch.cat(ys, dim=0), torch.
    ↪cat(ts, dim=0))

def train_linear_probe(train_ds, input_dim, seed=0, epochs=PROBE_EPOCHS):
    set_seed(seed + 70000)
    probe = LinearProbe(input_dim=input_dim).to(DEVICE)
    opt = torch.optim.Adam(probe.parameters(), lr=PROBE_LR)
    loader = make_loader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
    rows = []
    for epoch in range(epochs):
        probe.train()
        total_loss, n_batches = 0.0, 0
        for xb, yb, _tb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            logits = probe(xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()
            total_loss += float(loss.detach().cpu())
            n_batches += 1
        rows.append({"epoch": epoch, "probe_loss": total_loss / max(n_batches,
    ↪1)})
    return probe, pd.DataFrame(rows)

@torch.no_grad()

```

```

def evaluate_probe(probe, test_ds, method_name, seed, backbone_source,
    ↪feature_kind, context_mode):
    probe.eval()
    X, y, task_ids = test_ds.tensors
    final_task = N_TASKS - 1
    seen = seen_classes_until(final_task)
    rows = []
    mat = torch.zeros(N_CLASSES, N_CLASSES, dtype=torch.long)
    for task_id in range(N_TASKS):
        mask = (task_ids == task_id)
        if not mask.any():
            continue
        ds = TensorDataset(X[mask], y[mask])
        loader = make_loader(ds, batch_size=BATCH_SIZE, shuffle=False)
        correct, total = 0, 0
        class2_margin_24, class5_margin_54 = [], []
        class2_logit_rank, class5_logit_rank = [], []
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits = mask_logits_to_seen_classes(probe(xb), seen)
            pred = logits.argmax(dim=-1)
            correct += (pred == yb).sum().item()
            total += yb.numel()
            if (yb == 2).any():
                m = (yb == 2)
                class2_margin_24.append((logits[m, 2] - logits[m, 4]).detach().
    ↪cpu())

                ranks = (logits[m] > logits[m, 2].unsqueeze(-1)).sum(dim=-1) + 1
                class2_logit_rank.append(ranks.float().detach().cpu())
            if (yb == 5).any():
                m = (yb == 5)
                class5_margin_54.append((logits[m, 5] - logits[m, 4]).detach().
    ↪cpu())

                ranks = (logits[m] > logits[m, 5].unsqueeze(-1)).sum(dim=-1) + 1
                class5_logit_rank.append(ranks.float().detach().cpu())
        for yt, yp in zip(yb.detach().cpu(), pred.detach().cpu()):
            mat[int(yt), int(yp)] += 1
    rows.append({
        "method": method_name,
        "seed": seed,
        "model_family": "probe",
        "after_task": final_task,
        "task_id": task_id,
        "task_name": f"probe_task_{task_id}",
        "seen_classes": str(seen),
        "trainable_params": count_parameters(probe),
        "total_params": total_parameters(probe),

```

```

        "model_size_mb_trainable": model_size_mb(probe,
↪trainable_only=True),
        "model_size_mb_total": model_size_mb(probe, trainable_only=False),
        "uses_balanced_replay": False,
        "uses_output_distill": False,
        "probe_row": True,
        "backbone_source": backbone_source,
        "probe_feature": feature_kind,
        "probe_context_mode": context_mode,
        "replay_memory_items": 0,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "acc": correct / max(total, 1),
        "context_entropy": np.nan,
        "context_margin": np.nan,
        "context_top1_acc": np.nan,
        "context_brier": np.nan,
        "context_ece": np.nan,
        "latent_frechet_margin": np.nan,
        "prototype_weight": np.nan,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_logit_rank).mean()) if
↪class2_logit_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_logit_rank).mean()) if
↪class5_logit_rank else np.nan,
    })
    cm_rows = []
    for true_class in range(N_CLASSES):
        denom = mat[true_class].sum().item()
        for pred_class in range(N_CLASSES):
            cm_rows.append({
                "method": method_name,
                "seed": seed,
                "after_task": final_task,
                "backbone_source": backbone_source,
                "true_class": true_class,
                "pred_class": pred_class,
                "count": int(mat[true_class, pred_class].item()),
                "rate": float(mat[true_class, pred_class].item() / max(denom,
↪1)),
            })
    return pd.DataFrame(rows), pd.DataFrame(cm_rows)

```

```

def short_backbone_name(method: str) -> str:
    mapping = {
        "mmals_uniform_balanced_replay": "uniform",
        "mmals_proto_first_balanced_replay": "proto",
        "mmals_oracle_context_balanced_replay": "oracle",
    }
    return mapping.get(method, method.replace("mmals_", "").
↪replace("_balanced_replay", ""))

def run_representation_audit(artifact: Dict[str, Any]):
    backbone_source = artifact["method"]
    seed = artifact["seed"]
    model = freeze_model(artifact["model"])
    tasks = artifact["tasks"]
    prototypes = artifact["context_prototypes"]
    all_h, all_c, probe_loss_rows = [], [], []
    # zf probes under uniform/proto/oracle contexts.
    for mode in PROBE_CONTEXT_MODES:
        train_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind="zf", train=True)
        test_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind="zf", train=False)
        input_dim = train_ds.tensors[0].shape[1]
        probe, loss_df = train_linear_probe(train_ds, input_dim=input_dim,
↪seed=seed)
        method_name = f"probe_{short_backbone_name(backbone_source)}_zf_{mode}"
        loss_df["method"] = method_name
        loss_df["seed"] = seed
        loss_df["backbone_source"] = backbone_source
        loss_df["probe_feature"] = "zf"
        loss_df["probe_context_mode"] = mode
        h, c = evaluate_probe(probe, test_ds, method_name, seed,
↪backbone_source, "zf", mode)
        all_h.append(h)
        all_c.append(c)
        probe_loss_rows.append(loss_df)
        print(f"[{method_name} seed={seed}] probe final task accs: {[round(x,
↪3) for x in h.acc.tolist()]}")
        # Host/transformed-host probes, limited to transformed host latents for
↪compactness.
        for hid in range(N_HOSTS):
            feature_kind = f"transformed{hid}"
            mode = HOST_PROBE_MODE
            train_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind=feature_kind, train=True)

```

```

        test_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind=feature_kind, train=False)
        input_dim = train_ds.tensors[0].shape[1]
        probe, loss_df = train_linear_probe(train_ds, input_dim=input_dim,
↪seed=seed + hid + 10)
        method_name =
↪f"probe_{short_backbone_name(backbone_source)}_{feature_kind}_{mode}"
        loss_df["method"] = method_name
        loss_df["seed"] = seed
        loss_df["backbone_source"] = backbone_source
        loss_df["probe_feature"] = feature_kind
        loss_df["probe_context_mode"] = mode
        h, c = evaluate_probe(probe, test_ds, method_name, seed,
↪backbone_source, feature_kind, mode)
        all_h.append(h)
        all_c.append(c)
        probe_loss_rows.append(loss_df)
        print(f"[{method_name} seed={seed}] host probe accs: {[round(x, 3) for
↪x in h.acc.tolist()]}")
        return pd.concat(all_h, ignore_index=True), pd.concat(all_c,
↪ignore_index=True), pd.concat(probe_loss_rows, ignore_index=True)

```

1.16 12. Run paired v1.0-RC1b validation audit

This cell performs:

one base MMALS backbone → cloned controls and RC1b candidate variants

The primary controls are:

- paired_proto_global_no_repair: proto-first context, copied global head.
- paired_proto_kl_only_true_noop: deliberately identical no-op. It should match no-repair.
- paired_proto_global_head_ce_kl_guard_035: true corrected global CE+KL guarded-head control.

The deployable context rows are:

- paired_context_blend_selected_global: context blend selection only.
- paired_context_temp_blend_selected_global: context temp+blend selection only.
- paired_context_blend_selected_head_guard_035: rejected original RC1 diagnostic.
- paired_context_temp_blend_selected_head_guard_035: RC1b equivalent diagnostic.
- paired_context_gap_selected: **primary v1.0-RC1b candidate row**.

Oracle rows are references only. They measure the ceiling when the task/context is known; they are not deployable task-free methods.

1.16.1 12.1 Control sanity audit

Before running the expensive experiment, this cell verifies that guarded-head activation is explicit and correct: true global guard and context+guard must activate guard; no-repair, no-op and

context-only must not.

```
[51]: # Control-fix sanity audit: verify that guarded-head activation is explicit and
      ↪correct.
guard_map = {variant: variant_uses_guarded_head(variant) for variant in
      ↪SELECTED_CONTEXT_HEAD_VARIANTS}
print("Guard activation map:")
for k, v in guard_map.items():
    print(f" - {k}: {v}")

assert guard_map["proto_global_head_ce_kl_guard_035"] is True, "Control-fix
      ↪failed: true global guard is inactive."
assert guard_map["proto_global_no_repair"] is False, "No-repair must not
      ↪activate guard."
assert guard_map["proto_kl_only_true_noop"] is False, "No-op must not activate
      ↪guard."
assert guard_map["context_temp_blend_selected_head_guard_035"] is True,
      ↪"Context+guard candidate must activate guard."
assert guard_map["context_temp_blend_selected_global"] is False, "Context-only
      ↪candidate must not activate guard."
print("Control-fix sanity checks passed.")
```

Guard activation map:

- proto_global_no_repair: False
- proto_kl_only_true_noop: False
- proto_global_head_ce_kl_guard_035: True
- oracle_global_no_repair_reference: False
- oracle_global_head_ce_kl_guard_035_reference: True
- context_blend_selected_global: False
- context_temp_blend_selected_global: False
- context_blend_selected_head_guard_035: True
- context_temp_blend_selected_head_guard_035: True
- context_gap_selected: True

Control-fix sanity checks passed.

1.17 12.2 Comparative baseline suite

This section adds standard continual-learning baselines to the same overlapping class-incremental stress protocol.

The MMALS paired audit remains unchanged. Baselines are trained sequentially on the same tasks and evaluated with the same seen-class mask. The joint upper bound is trained on all tasks jointly and is diagnostic, not deployable in a continual-learning setting.

Diagnostic patch. Standard baseline confusion matrices now export normalized **rate** values, matching the MMALS confusion tables. The interpretation section also separates deployable baselines, deployable MMALS rows, oracle references, probes, and the joint upper bound.

```
[52]: class SparseMoEClassifier(nn.Module):
    """Small sparse mixture-of-experts baseline with a learned soft gate."""
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
        ↪latent_dim=LATENT_DIM, n_experts=N_HOSTS, n_classes=N_CLASSES):
        super().__init__()
        self.experts = nn.ModuleList([MLPBlock(input_dim, hidden_dim,
        ↪latent_dim) for _ in range(n_experts)])
        self.gate = nn.Sequential(
            nn.Linear(input_dim, ROUTE_HIDDEN_DIM),
            nn.ReLU(),
            nn.Linear(ROUTE_HIDDEN_DIM, n_experts),
        )
        self.head = nn.Linear(latent_dim, n_classes)

    def forward(self, x, return_gate=False):
        weights = F.softmax(self.gate(x) / TEMPERATURE, dim=-1)
        z_stack = torch.stack([expert(x) for expert in self.experts], dim=1)
        z = (weights.unsqueeze(-1) * z_stack).sum(dim=1)
        logits = self.head(z)
        if return_gate:
            return logits, weights
        return logits

def set_requires_grad(module_or_param, requires_grad: bool):
    """
    Enable or disable gradient updates for a module or a single parameter.
    Used by the PNN baseline to freeze old columns and train only the current
    ↪column.
    """
    if isinstance(module_or_param, nn.Module):
        for p in module_or_param.parameters():
            p.requires_grad_(requires_grad)
    elif isinstance(module_or_param, torch.nn.Parameter):
        module_or_param.requires_grad_(requires_grad)
    else:
        raise TypeError(f"Unsupported type for set_requires_grad:
        ↪{type(module_or_param)}")

class PNNSimpleClassifier(nn.Module):
    """Fixed-column PNN-style modular baseline.

    One column is activated for each task; old columns are frozen after their
    ↪task.

    The global head reads the concatenation of all columns. This is a compact
    ↪Colab-safe
    approximation of a progressive neural network control.
```

```

"""
def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
    ↪latent_dim=LATENT_DIM, n_tasks=N_TASKS, n_classes=N_CLASSES):
    super().__init__()
    self.columns = nn.ModuleList([MLPBlock(input_dim, hidden_dim,
    ↪latent_dim) for _ in range(n_tasks)])
    self.head = nn.Linear(latent_dim * n_tasks, n_classes)

def set_trainable_column(self, task_id: int):
    for i, col in enumerate(self.columns):
        set_requires_grad(col, i == task_id)
    set_requires_grad(self.head, True)

def forward(self, x):
    zs = [col(x) for col in self.columns]
    return self.head(torch.cat(zs, dim=-1))

def baseline_forward(model, xb, method=None):
    if isinstance(model, SparseMoEClassifier):
        logits, gate = model(xb, return_gate=True)
        return logits, gate
    return model(xb), None

@torch.no_grad()
def evaluate_standard_model(model, tasks, upto_task, method, seed,
    ↪model_family, replay_memory_items=0):
    model.eval()
    rows = []
    seen = seen_classes_until(upto_task)
    trainable = count_parameters(model)
    total = total_parameters(model)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
    ↪shuffle=False)
        correct, total_items = 0, 0
        gate_entropies = []
        class2_margin_24, class5_margin_54 = [], []
        class2_rank, class5_rank = [], []
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits, gate = baseline_forward(model, xb, method=method)
            logits_eval = mask_logits_to_seen_classes(logits, seen)
            pred = logits_eval.argmax(dim=-1)
            correct += (pred == yb).sum().item()
            total_items += yb.numel()
            if gate is not None:
                gate_entropies.append(entropy_from_probs(gate).detach().cpu())

```

```

        if (yb == 2).any():
            m = (yb == 2)
            class2_margin_24.append((logits_eval[m, 2] - logits_eval[m, 4]).
↪detach()).cpu())
            class2_rank.append(((logits_eval[m] > logits_eval[m, 2].
↪unsqueeze(-1)).sum(dim=-1) + 1).float().detach()).cpu())
            if (yb == 5).any():
                m = (yb == 5)
                class5_margin_54.append((logits_eval[m, 5] - logits_eval[m, 4]).
↪detach()).cpu())
                class5_rank.append(((logits_eval[m] > logits_eval[m, 5].
↪unsqueeze(-1)).sum(dim=-1) + 1).float().detach()).cpu())
        rows.append({
            "method": method,
            "seed": seed,
            "model_family": model_family,
            "after_task": upto_task,
            "task_id": task_id,
            "task_name": tasks[task_id]["name"],
            "seen_classes": str(seen),
            "acc": correct / max(total_items, 1),
            "context_entropy": float(torch.cat(gate_entropies).mean()) if ↵
↪gate_entropies else np.nan,
            "context_margin": np.nan,
            "context_top1_acc": np.nan,
            "context_brier": np.nan,
            "context_ece": np.nan,
            "latent_frechet_margin": np.nan,
            "prototype_weight": 0.0,
            "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if ↵
↪class2_margin_24 else np.nan,
            "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if ↵
↪class5_margin_54 else np.nan,
            "class2_logit_rank": float(torch.cat(class2_rank).mean()) if ↵
↪class2_rank else np.nan,
            "class5_logit_rank": float(torch.cat(class5_rank).mean()) if ↵
↪class5_rank else np.nan,
            "trainable_params": trainable,
            "total_params": total,
            "backbone_params": total,
            "readout_trainable_params": np.nan,
            "readout_total_params": np.nan,
            "model_size_mb_total": total * 4 / (1024 ** 2),
            "uses_balanced_replay": method == "baseline_balanced_replay",
            "uses_output_distill": method == "baseline_lwf",
            "uses_local_mycorrhizal_head": False,

```

```

        "uses_global_head_refresh": False,
        "uses_context_selection": False,
        "uses_oracle_context_reference": False,
        "probe_row": False,
        "backbone_source": "baseline_sequential_or_joint",
        "probe_feature": "baseline_logits",
        "probe_context_mode": "none",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": False,
        "readout_variant": "baseline",
        "context_variant": "none",
    })
    return pd.DataFrame(rows)

@torch.no_grad()
def class_confusion_matrix_standard(model, tasks, upto_task, method, seed):
    """Class-confusion table for standard baselines, normalized like the MMALS_
    ↪ confusion table.

    Diagnostic patch:
    the original benchmark-harness smoke run wrote raw counts but did not write_
    ↪ the
    per-true-class `rate` column. `class_diagnostics_from_confusion` expects_
    ↪ this
    normalized field; without it, baseline class diagnostics become NaN/flat and
    cannot be compared fairly to MMALS rows.
    """
    model.eval()
    seen = seen_classes_until(upto_task)
    cm = torch.zeros((N_CLASSES, N_CLASSES), dtype=torch.long)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
        ↪ shuffle=False)
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits, _ = baseline_forward(model, xb, method=method)
            pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
            for t, p in zip(yb.detach().cpu(), pred.detach().cpu()):
                cm[int(t), int(p)] += 1

    rows = []
    for true_cls in range(N_CLASSES):
        row_total = int(cm[true_cls].sum().item())
        for pred_cls in range(N_CLASSES):
            count = int(cm[true_cls, pred_cls].item())

```

```

        rows.append({
            "true_class": true_cls,
            "pred_class": pred_cls,
            "count": count,
            "rate": float(count / max(row_total, 1)),
            "method": method,
            "seed": seed,
            "after_task": upto_task,
            "backbone_source": "baseline_sequential_or_joint",
            "readout_variant": "baseline",
            "context_variant": "none",
            "uses_oracle_context_reference": False,
        })
    return pd.DataFrame(rows)

def ewc_compute_fisher(model, dataset, max_items=512):
    model.eval()
    params = {n: p.detach().clone() for n, p in model.named_parameters() if p.
    ↪requires_grad}
    fisher = {n: torch.zeros_like(p, device=DEVICE) for n, p in model.
    ↪named_parameters() if p.requires_grad}
    X, y = dataset.tensors
    if len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        dataset = TensorDataset(X[idx], y[idx])
    loader = make_loader(dataset, batch_size=BATCH_SIZE, shuffle=False)
    n_batches = 0
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        model.zero_grad(set_to_none=True)
        logits, _ = baseline_forward(model, xb)
        loss = F.cross_entropy(logits, yb)
        loss.backward()
        for name, p in model.named_parameters():
            if p.grad is not None and name in fisher:
                fisher[name] += p.grad.detach().pow(2)
        n_batches += 1
    for name in fisher:
        fisher[name] = fisher[name] / max(n_batches, 1)
    return {"params": params, "fisher": fisher}

def ewc_penalty(model, ewc_snapshots):
    if not ewc_snapshots:
        return torch.zeros((), device=DEVICE)
    penalty = torch.zeros((), device=DEVICE)
    current = dict(model.named_parameters())
    for snap in ewc_snapshots:

```

```

        for name, old_param in snap["params"].items():
            if name in current:
                penalty = penalty + (snap["fisher"][name] * (current[name] -
old_param.to(DEVICE)).pow(2)).sum()
            return penalty

def train_sequential_baseline(method: str, seed: int):
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    if method == "baseline_sparse_moe":
        model = SparseMoEClassifier().to(DEVICE)
        model_family = "sparse_moe"
    elif method == "baseline_pnn":
        model = PNNSimpleClassifier().to(DEVICE)
        model_family = "pnn_fixed_columns"
    else:
        model = MLPBaseline().to(DEVICE)
        model_family = "mlp_baseline"
    class_memory: Dict[int, Dict[str, torch.Tensor]] = {}
    memory_buffers: Dict[int, TensorDataset] = {}
    teacher = None
    ewc_snapshots = []
    history_rows, loss_rows, confusion_rows = [], [], []
    for task_id, task in enumerate(tasks):
        if method == "baseline_pnn":
            model.set_trainable_column(task_id)
            opt = torch.optim.Adam([p for p in model.parameters() if p.
requires_grad], lr=LR)
            loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)
            for epoch in range(EPOCHS_PER_TASK):
                model.train()
                sums = {"loss": 0.0, "task": 0.0, "replay": 0.0, "distill": 0.0,
ewc": 0.0, "sparse": 0.0}
                n_batches = 0
                for xb, yb in loader:
                    xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                    opt.zero_grad(set_to_none=True)
                    logits, gate = baseline_forward(model, xb, method=method)
                    task_loss = F.cross_entropy(logits, yb)
                    replay_loss = torch.zeros(), device=DEVICE)
                    if method in ["baseline_experience_replay",
baseline_balanced_replay"] and class_memory:
                        xb_r, yb_r, _tb =
sample_balanced_class_memory(class_memory, batch_size=BALANCED_REPLAY_BATCH)
                        if xb_r is not None:

```

```

logits_r, _ = baseline_forward(model, xb_r,
↪method=method)
replay_loss = F.cross_entropy(logits_r, yb_r) *
↪BASELINE_REPLAY_WEIGHT
distill_loss = torch.zeros(), device=DEVICE)
if method == "baseline_lwf" and teacher is not None:
    with torch.no_grad():
        teacher_logits, _ = baseline_forward(teacher, xb,
↪method=method)
distill_loss = LWF_LAMBDA * kl_distill_loss(logits,
↪teacher_logits)
ewc_loss = torch.zeros(), device=DEVICE)
if method == "baseline_ewc" and ewc_snapshots:
    ewc_loss = EWC_LAMBDA * ewc_penalty(model, ewc_snapshots)
sparse_loss = torch.zeros(), device=DEVICE)
if method == "baseline_sparse_moe" and gate is not None:
    sparse_loss = SPARSE_MOE_ENTROPY_WEIGHT *
↪entropy_from_probs(gate).mean()
loss = task_loss + replay_loss + distill_loss + ewc_loss +
↪sparse_loss
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
opt.step()
sums["loss"] += float(loss.detach().cpu())
sums["task"] += float(task_loss.detach().cpu())
sums["replay"] += float(replay_loss.detach().cpu())
sums["distill"] += float(distill_loss.detach().cpu())
sums["ewc"] += float(ewc_loss.detach().cpu())
sums["sparse"] += float(sparse_loss.detach().cpu())
n_batches += 1
loss_rows.append({
    "method": method,
    "seed": seed,
    "task_id": task_id,
    "epoch": epoch,
    "loss": sums["loss"] / max(n_batches, 1),
    "task_loss": sums["task"] / max(n_batches, 1),
    "memory_loss": sums["distill"] / max(n_batches, 1),
    "context_loss": 0.0,
    "context_replay_loss": 0.0,
    "balanced_replay_loss": sums["replay"] / max(n_batches, 1),
    "fungal_loss": sums["sparse"] / max(n_batches, 1),
    "ewc_loss": sums["ewc"] / max(n_batches, 1),
})

```

```

        eval_rows = evaluate_standard_model(model, tasks, task_id, method,
↪seed, model_family, replay_memory_items=int(sum(len(v["y"]) for v in
↪class_memory.values()))))
        history_rows.append(eval_rows)
        print(f"[{method}] seed={seed}] after task {task_id}: {[round(a, 3) for
↪a in eval_rows['acc'].tolist()]}")
        class_memory = update_class_memory(class_memory, task["train"],
↪task_id, MEMORY_PER_CLASS, seed=seed + 3000 + task_id)
        memory_buffers[task_id] = build_memory_subset(task["train"],
↪MEMORY_PER_TASK, seed=seed + 4000 + task_id)
        if method == "baseline_lwf":
            teacher = clone_teacher(model)
        if method == "baseline_ewc":
            ewc_snapshots.append(ewc_compute_fisher(model, task["train"],
↪max_items=min(512, len(task["train"]))))
        clear_memory()
        confusion_rows.append(class_confusion_matrix_standard(model, tasks, N_TASKS,
↪- 1, method, seed))
        return pd.concat(history_rows, ignore_index=True), pd.DataFrame(loss_rows),
↪pd.concat(confusion_rows, ignore_index=True)

def train_joint_upper_bound_baseline(seed: int):
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    model = MLPBaseline().to(DEVICE)
    X_train = torch.cat([t["train"].tensors[0] for t in tasks], dim=0)
    y_train = torch.cat([t["train"].tensors[1] for t in tasks], dim=0)
    joint_ds = TensorDataset(X_train, y_train)
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    # Same total training budget as one pass over all sequential tasks.
    for epoch in range(EPOCHS_PER_TASK * N_TASKS):
        model.train()
        for xb, yb in make_loader(joint_ds, batch_size=BATCH_SIZE,
↪shuffle=True):
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            logits, _ = baseline_forward(model, xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()
        method = "baseline_joint_upper_bound"
        h = evaluate_standard_model(model, tasks, N_TASKS - 1, method, seed,
↪"joint_upper_bound", replay_memory_items=len(joint_ds))
        # Fill a final-only loss row; forgetting is not meaningful for joint upper
↪bound.

```

```

    l = pd.DataFrame([{"method": method, "seed": seed, "task_id": N_TASKS - 1,
↳ "epoch": EPOCHS_PER_TASK * N_TASKS, "loss": np.nan, "task_loss": np.nan}])
    c = class_confusion_matrix_standard(model, tasks, N_TASKS - 1, method, seed)
    print(f"[{method}] seed={seed} final joint eval: {[round(a, 3) for a in
↳ h['acc'].tolist()]}")
    return h, l, c

def run_benchmark_baselines_for_seed(seed: int):
    method_map = {
        "finetune": "baseline_finetune",
        "ewc": "baseline_ewc",
        "lwf": "baseline_lwf",
        "experience_replay": "baseline_experience_replay",
        "balanced_replay": "baseline_balanced_replay",
        "sparse_moe": "baseline_sparse_moe",
        "pnn": "baseline_pnn",
        "joint_upper_bound": "baseline_joint_upper_bound",
    }
    histories, losses, confusions = [], [], []
    for short_name in SELECTED_BASELINE_METHODS:
        method = method_map[short_name]
        if method == "baseline_joint_upper_bound":
            h, l, c = train_joint_upper_bound_baseline(seed)
        else:
            h, l, c = train_sequential_baseline(method, seed)
        histories.append(h)
        losses.append(l)
        confusions.append(c)
    return histories, losses, confusions

```

```

[ ]: start_persistent_logging()
log_event("experiment_started", payload={
    "rc2b_candidate_variants": RC2B_CANDIDATE_VARIANTS,
    "baseline_methods": SELECTED_BASELINE_METHODS,
    "strict_no_final_test_selection": RC2B_STRICT_NO_FINAL_TEST_SELECTION,
})
experiment_t0 = time.time()

all_history, all_losses, all_confusions, all_probe_losses, all_repair_logs,
↳ all_selection_logs = [], [], [], [], [], []

for seed in SEEDS:
    seed_t0 = time.time()
    log_event("seed_started", seed=seed,
↳ phase="rc2b_strict_base_proto_and_policy_selection")
    base_run = train_base_proto_first_checkpoints(seed)
    all_losses.append(base_run["loss_df"])

```

```

    for checkpoint in base_run["checkpoints"]:
        selected_h, selected_c, candidate_h, candidate_c, rlog, slog =
→ apply_rc2b_strict_policy_to_checkpoint(base_run, checkpoint)
        all_history.append(selected_h)
        if not candidate_h.empty:
            all_history.append(candidate_h)
        if not selected_c.empty:
            all_confusions.append(selected_c)
        if not candidate_c.empty:
            all_confusions.append(candidate_c)
        if not rlog.empty:
            all_repair_logs.append(rlog)
        if not slog.empty:
            all_selection_logs.append(slog)
        log_event(
            "rc2b_policy_selected",
            seed=int(checkpoint["seed"]),
            task_id=int(checkpoint["after_task"]),
            method=RC2B_METHOD,
            phase="policy_selection_before_final_test",
            payload=slog[slog.get("is_selected_candidate", False) == True].
→ to_dict("records")[:1] if not slog.empty else {},
        )

    if RUN_FINAL_PROBES:
        artifact = final_probe_artifact_from_base_run(base_run)
        ph, pc, pl = run_representation_audit(artifact)
        all_history.append(ph)
        all_confusions.append(pc)
        all_probe_losses.append(pl)
        del artifact
    del base_run
    clear_memory()

    # Comparative baseline suite on the same seed and task construction.
    base_histories, base_losses, base_confusions =
→ run_benchmark_baselines_for_seed(seed)
    all_history.extend(base_histories)
    all_losses.extend(base_losses)
    all_confusions.extend(base_confusions)
    clear_memory()
    log_event("seed_completed", seed=seed,
→ phase="rc2b_strict_base_proto_and_policy_selection", payload={"duration_sec":
→ time.time() - seed_t0})

```

```

history_df = pd.concat(all_history, ignore_index=True) if all_history else pd.
    ↪DataFrame()
loss_df = pd.concat(all_losses, ignore_index=True) if all_losses else pd.
    ↪DataFrame()
repair_log_df = pd.concat(all_repair_logs, ignore_index=True) if
    ↪all_repair_logs else pd.DataFrame()
selection_log_df = pd.concat(all_selection_logs, ignore_index=True) if
    ↪all_selection_logs else pd.DataFrame()
confusion_df = pd.concat(all_confusions, ignore_index=True) if all_confusions
    ↪else pd.DataFrame()
probe_loss_df = pd.concat(all_probe_losses, ignore_index=True) if
    ↪all_probe_losses else pd.DataFrame()

deployed_history = history_df[(history_df.probe_row == False)] if not
    ↪history_df.empty and "probe_row" in history_df.columns else history_df
forgetting_df = forgetting_from_history(deployed_history) if not
    ↪deployed_history.empty else pd.DataFrame()
class_diag_df = class_diagnostics_from_confusion(confusion_df) if not
    ↪confusion_df.empty else pd.DataFrame()

history_path = RESULTS_DIR / f"history_{RUN_MODE}.csv"
loss_path = RESULTS_DIR / f"losses_{RUN_MODE}.csv"
repair_log_path = RESULTS_DIR / f"context_gap_repair_logs_{RUN_MODE}.csv"
selection_log_path = RESULTS_DIR / f"rc2b_policy_selection_trace_{RUN_MODE}.csv"
probe_loss_path = RESULTS_DIR / f"probe_losses_{RUN_MODE}.csv"
forgetting_path = RESULTS_DIR / f"forgetting_{RUN_MODE}.csv"
confusion_path = RESULTS_DIR / f"class_confusion_{RUN_MODE}.csv"
class_diag_path = RESULTS_DIR / f"class_diagnostics_{RUN_MODE}.csv"

history_df.to_csv(history_path, index=False)
loss_df.to_csv(loss_path, index=False)
repair_log_df.to_csv(repair_log_path, index=False)
selection_log_df.to_csv(selection_log_path, index=False)
probe_loss_df.to_csv(probe_loss_path, index=False)
forgetting_df.to_csv(forgetting_path, index=False)
confusion_df.to_csv(confusion_path, index=False)
class_diag_df.to_csv(class_diag_path, index=False)

print("Saved:")
_saved_core_paths = [history_path, loss_path, repair_log_path,
    ↪selection_log_path, probe_loss_path, forgetting_path, confusion_path,
    ↪class_diag_path]
for p in _saved_core_paths:
    print(" -", p)
if not selection_log_df.empty:
    print("RC2b selected policy counts:")

```

```

        display(selection_log_df[selection_log_df["is_selected_candidate"] == True].
        ↳groupby(["selected_candidate_variant", "selected_policy_family"],
        ↳as_index=False).size())
log_event("experiment_core_results_saved", phase="export",
        ↳payload={"duration_sec": time.time() - experiment_t0, "files": [str(p) for p
        ↳in _saved_core_paths]})
# Keep logging active for interpretation/audit cells; it will be stopped in the
        ↳final packaging cell.

```

```

[base_proto seed=0] task=0 epoch=1/8 loss=1.9982 task=0.9697 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=2/8 loss=0.6132 task=0.3132 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=3/8 loss=0.5025 task=0.2561 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=4/8 loss=0.4533 task=0.2236 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=5/8 loss=0.4301 task=0.2097 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=6/8 loss=0.4155 task=0.2039 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=7/8 loss=0.4028 task=0.2001 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=8/8 loss=0.3904 task=0.1956 memory=0.0000
balanced=0.0000
[base_proto seed=0] calibrated context temperature after task 0: 0.25
[base_proto seed=0] task=1 epoch=1/8 loss=5.9971 task=1.6252 memory=0.6382
balanced=0.2744
[base_proto seed=0] task=1 epoch=2/8 loss=2.9186 task=0.5433 memory=0.2923
balanced=0.2405
[base_proto seed=0] task=1 epoch=3/8 loss=2.2425 task=0.3952 memory=0.1251
balanced=0.2533
[base_proto seed=0] task=1 epoch=4/8 loss=2.0118 task=0.3610 memory=0.0787
balanced=0.2713
[base_proto seed=0] task=1 epoch=5/8 loss=1.8401 task=0.3361 memory=0.0641
balanced=0.2382
[base_proto seed=0] task=1 epoch=6/8 loss=1.7256 task=0.3140 memory=0.0538
balanced=0.2066
[base_proto seed=0] task=1 epoch=7/8 loss=1.7098 task=0.2994 memory=0.0507
balanced=0.2413
[base_proto seed=0] task=1 epoch=8/8 loss=1.6227 task=0.2966 memory=0.0541
balanced=0.2014
[base_proto seed=0] calibrated context temperature after task 1: 0.80
[base_proto seed=0] task=2 epoch=1/8 loss=5.3338 task=1.6420 memory=0.4796
balanced=0.3281
[base_proto seed=0] task=2 epoch=2/8 loss=3.1753 task=0.6899 memory=0.3581
balanced=0.3122

```

```

[base_proto seed=0] task=2 epoch=3/8 loss=2.4895 task=0.4932 memory=0.1913
balanced=0.2910
[base_proto seed=0] task=2 epoch=4/8 loss=2.0783 task=0.3543 memory=0.1259
balanced=0.2568
[base_proto seed=0] task=2 epoch=5/8 loss=1.8385 task=0.3098 memory=0.0788
balanced=0.2311
[base_proto seed=0] task=2 epoch=6/8 loss=1.6558 task=0.2775 memory=0.0535
balanced=0.1757
[base_proto seed=0] task=2 epoch=7/8 loss=1.6173 task=0.2648 memory=0.0523
balanced=0.1784
[base_proto seed=0] task=2 epoch=8/8 loss=1.5575 task=0.2679 memory=0.0538
balanced=0.1815
[base_proto seed=0] calibrated context temperature after task 2: 0.60
[base_proto seed=0] task=3 epoch=1/8 loss=6.0587 task=2.1681 memory=1.0736
balanced=0.3271
[base_proto seed=0] task=3 epoch=2/8 loss=3.6216 task=1.0233 memory=0.7329
balanced=0.2342
[base_proto seed=0] task=3 epoch=3/8 loss=3.0602 task=0.7107 memory=0.6900
balanced=0.2503
[base_proto seed=0] task=3 epoch=4/8 loss=2.8869 task=0.5717 memory=0.7975
balanced=0.2071
[base_proto seed=0] task=3 epoch=5/8 loss=2.4093 task=0.5095 memory=0.4575
balanced=0.1914
[base_proto seed=0] task=3 epoch=6/8 loss=2.2695 task=0.4645 memory=0.4097
balanced=0.1946
[base_proto seed=0] task=3 epoch=7/8 loss=2.1135 task=0.4516 memory=0.3104
balanced=0.1933
[base_proto seed=0] task=3 epoch=8/8 loss=1.9416 task=0.4018 memory=0.2489
balanced=0.1720
[base_proto seed=0] calibrated context temperature after task 3: 0.60
[base_proto seed=0] task=4 epoch=1/8 loss=3.9757 task=1.3421 memory=0.3556
balanced=0.5376
[base_proto seed=0] task=4 epoch=2/8 loss=2.3971 task=0.3832 memory=0.1746
balanced=0.4670
[base_proto seed=0] task=4 epoch=3/8 loss=2.4329 task=0.3768 memory=0.3868
balanced=0.4598
[base_proto seed=0] task=4 epoch=4/8 loss=2.1288 task=0.3248 memory=0.2532
balanced=0.4398
[base_proto seed=0] task=4 epoch=5/8 loss=1.9261 task=0.3061 memory=0.1862
balanced=0.4003
[base_proto seed=0] task=4 epoch=6/8 loss=1.8328 task=0.2914 memory=0.2039
balanced=0.3756
[base_proto seed=0] task=4 epoch=7/8 loss=1.6287 task=0.2769 memory=0.0716
balanced=0.3605
[base_proto seed=0] task=4 epoch=8/8 loss=1.6770 task=0.2687 memory=0.1635
balanced=0.3570
[base_proto seed=0] calibrated context temperature after task 4: 0.60
[base_proto seed=0] completed in 230.5s

```

[RC2b seed=0] after task 0: selected proto_global_no_repair proxy_acc=0.8906
 score=1.0531
 [RC2b seed=0] after task 1: selected proto_global_no_repair proxy_acc=0.9271
 score=1.2990
 [RC2b seed=0] after task 2: selected proto_global_no_repair proxy_acc=0.9453
 score=1.3016
 [RC2b seed=0] after task 3: selected proto_global_head_ce_kl_guard_035
 proxy_acc=0.8781 score=1.0886
 [RC2b seed=0] after task 4: selected context_temp_blend_selected_head_guard_035
 proxy_acc=0.9271 score=1.2897
 [probe_proto__zf_proto seed=0] probe final task accs: [0.954, 0.94, 0.904,
 0.818, 0.892]
 [probe_proto__zf_oracle seed=0] probe final task accs: [0.96, 0.896, 0.95,
 0.899, 0.919]
 [probe_proto__transformed0_oracle seed=0] host probe accs: [0.767, 0.591, 0.315,
 0.554, 0.796]
 [probe_proto__transformed1_oracle seed=0] host probe accs: [0.73, 0.78, 0.757,
 0.691, 0.814]
 [probe_proto__transformed2_oracle seed=0] host probe accs: [0.564, 0.864, 0.715,
 0.268, 0.481]
 [probe_proto__transformed3_oracle seed=0] host probe accs: [0.9, 0.866, 0.799,
 0.859, 0.887]
 [probe_proto__transformed4_oracle seed=0] host probe accs: [0.787, 0.771, 0.344,
 0.246, 0.748]
 [baseline_finetune seed=0] after task 0: [0.981]
 [baseline_finetune seed=0] after task 1: [0.486, 0.99]
 [baseline_finetune seed=0] after task 2: [0.0, 0.487, 0.961]
 [baseline_finetune seed=0] after task 3: [0.0, 0.0, 0.477, 0.941]
 [baseline_finetune seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_ewc seed=0] after task 0: [0.981]
 [baseline_ewc seed=0] after task 1: [0.489, 0.989]
 [baseline_ewc seed=0] after task 2: [0.0, 0.487, 0.964]
 [baseline_ewc seed=0] after task 3: [0.0, 0.0, 0.485, 0.943]
 [baseline_ewc seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_lwf seed=0] after task 0: [0.981]
 [baseline_lwf seed=0] after task 1: [0.96, 0.515]
 [baseline_lwf seed=0] after task 2: [0.907, 0.974, 0.502]
 [baseline_lwf seed=0] after task 3: [0.253, 0.699, 0.92, 0.555]
 [baseline_lwf seed=0] after task 4: [0.007, 0.005, 0.388, 0.876, 0.985]
 [baseline_experience_replay seed=0] after task 0: [0.981]
 [baseline_experience_replay seed=0] after task 1: [0.96, 0.971]
 [baseline_experience_replay seed=0] after task 2: [0.886, 0.959, 0.92]
 [baseline_experience_replay seed=0] after task 3: [0.874, 0.825, 0.809, 0.892]
 [baseline_experience_replay seed=0] after task 4: [0.9, 0.756, 0.686, 0.856,
 0.968]
 [baseline_balanced_replay seed=0] after task 0: [0.981]
 [baseline_balanced_replay seed=0] after task 1: [0.96, 0.971]
 [baseline_balanced_replay seed=0] after task 2: [0.886, 0.959, 0.92]

[baseline_balanced_replay seed=0] after task 3: [0.874, 0.825, 0.809, 0.892]
 [baseline_balanced_replay seed=0] after task 4: [0.9, 0.756, 0.686, 0.856, 0.968]
 [baseline_sparse_moe seed=0] after task 0: [0.978]
 [baseline_sparse_moe seed=0] after task 1: [0.485, 0.99]
 [baseline_sparse_moe seed=0] after task 2: [0.0, 0.487, 0.964]
 [baseline_sparse_moe seed=0] after task 3: [0.0, 0.0, 0.474, 0.949]
 [baseline_sparse_moe seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_pnn seed=0] after task 0: [0.988]
 [baseline_pnn seed=0] after task 1: [0.481, 0.984]
 [baseline_pnn seed=0] after task 2: [0.0, 0.486, 0.96]
 [baseline_pnn seed=0] after task 3: [0.0, 0.0, 0.492, 0.924]
 [baseline_pnn seed=0] after task 4: [0.0, 0.0, 0.0, 0.496, 0.998]
 [baseline_joint_upper_bound seed=0] final joint eval: [0.924, 0.836, 0.795, 0.891, 0.915]
 [base_proto seed=1] task=0 epoch=1/8 loss=2.2393 task=0.9040 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=2/8 loss=0.6105 task=0.2822 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=3/8 loss=0.4889 task=0.2299 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=4/8 loss=0.4348 task=0.1996 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=5/8 loss=0.4193 task=0.1959 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=6/8 loss=0.3992 task=0.1845 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=7/8 loss=0.3903 task=0.1841 memory=0.0000 balanced=0.0000
 [base_proto seed=1] task=0 epoch=8/8 loss=0.3758 task=0.1762 memory=0.0000 balanced=0.0000
 [base_proto seed=1] calibrated context temperature after task 0: 0.25
 [base_proto seed=1] task=1 epoch=1/8 loss=6.4131 task=1.4879 memory=0.5950 balanced=0.2143
 [base_proto seed=1] task=1 epoch=2/8 loss=3.4552 task=0.5878 memory=0.2650 balanced=0.2296
 [base_proto seed=1] task=1 epoch=3/8 loss=2.7261 task=0.4016 memory=0.1468 balanced=0.1950
 [base_proto seed=1] task=1 epoch=4/8 loss=2.3917 task=0.3514 memory=0.0924 balanced=0.1760
 [base_proto seed=1] task=1 epoch=5/8 loss=2.1689 task=0.3279 memory=0.0589 balanced=0.1633
 [base_proto seed=1] task=1 epoch=6/8 loss=2.0627 task=0.3088 memory=0.0486 balanced=0.1656
 [base_proto seed=1] task=1 epoch=7/8 loss=1.9546 task=0.2907 memory=0.0390 balanced=0.1494
 [base_proto seed=1] task=1 epoch=8/8 loss=1.9031 task=0.2793 memory=0.0369 balanced=0.1377

```

[base_proto seed=1] calibrated context temperature after task 1: 0.80
[base_proto seed=1] task=2 epoch=1/8 loss=5.4834 task=1.4050 memory=0.4411
balanced=0.2619
[base_proto seed=1] task=2 epoch=2/8 loss=3.2329 task=0.5008 memory=0.2680
balanced=0.2641
[base_proto seed=1] task=2 epoch=3/8 loss=2.5083 task=0.3895 memory=0.1401
balanced=0.2197
[base_proto seed=1] task=2 epoch=4/8 loss=2.2068 task=0.3494 memory=0.1153
balanced=0.2033
[base_proto seed=1] task=2 epoch=5/8 loss=2.0123 task=0.3221 memory=0.0985
balanced=0.1881
[base_proto seed=1] task=2 epoch=6/8 loss=1.8730 task=0.3002 memory=0.0895
balanced=0.1724
[base_proto seed=1] task=2 epoch=7/8 loss=1.7574 task=0.2895 memory=0.0819
balanced=0.1617
[base_proto seed=1] task=2 epoch=8/8 loss=1.6584 task=0.2742 memory=0.0742
balanced=0.1426
[base_proto seed=1] calibrated context temperature after task 2: 0.60
[base_proto seed=1] task=3 epoch=1/8 loss=5.8071 task=2.0950 memory=0.8113
balanced=0.2983
[base_proto seed=1] task=3 epoch=2/8 loss=3.8773 task=1.0068 memory=0.7247
balanced=0.2882
[base_proto seed=1] task=3 epoch=3/8 loss=3.1464 task=0.7732 memory=0.5837
balanced=0.2866
[base_proto seed=1] task=3 epoch=4/8 loss=2.6342 task=0.6241 memory=0.3976
balanced=0.2547
[base_proto seed=1] task=3 epoch=5/8 loss=2.2963 task=0.5022 memory=0.3089
balanced=0.2198
[base_proto seed=1] task=3 epoch=6/8 loss=2.0697 task=0.4341 memory=0.2369
balanced=0.1916
[base_proto seed=1] task=3 epoch=7/8 loss=1.9027 task=0.3826 memory=0.2012
balanced=0.1604
[base_proto seed=1] task=3 epoch=8/8 loss=1.7993 task=0.3524 memory=0.1573
balanced=0.1778
[base_proto seed=1] calibrated context temperature after task 3: 0.60
[base_proto seed=1] task=4 epoch=1/8 loss=5.2031 task=1.9321 memory=0.7822
balanced=0.4425
[base_proto seed=1] task=4 epoch=2/8 loss=3.0526 task=0.8186 memory=0.2713
balanced=0.3212
[base_proto seed=1] task=4 epoch=3/8 loss=2.2470 task=0.3418 memory=0.1905
balanced=0.2888
[base_proto seed=1] task=4 epoch=4/8 loss=2.0519 task=0.3002 memory=0.1344
balanced=0.3078
[base_proto seed=1] task=4 epoch=5/8 loss=1.8827 task=0.2602 memory=0.1494
balanced=0.2663
[base_proto seed=1] task=4 epoch=6/8 loss=1.8056 task=0.2458 memory=0.1664
balanced=0.2639
[base_proto seed=1] task=4 epoch=7/8 loss=1.8055 task=0.2276 memory=0.1856

```

balanced=0.3248
 [base_proto seed=1] task=4 epoch=8/8 loss=1.5281 task=0.2165 memory=0.1127
 balanced=0.1789
 [base_proto seed=1] calibrated context temperature after task 4: 0.60
 [base_proto seed=1] completed in 241.7s
 [RC2b seed=1] after task 0: selected proto_global_no_repair proxy_acc=0.9219
 score=1.1000
 [RC2b seed=1] after task 1: selected proto_global_no_repair proxy_acc=0.9010
 score=1.2667
 [RC2b seed=1] after task 2: selected proto_global_no_repair proxy_acc=0.9375
 score=1.2906
 [RC2b seed=1] after task 3: selected proto_global_head_ce_kl_guard_035
 proxy_acc=0.9344 score=1.2949
 [RC2b seed=1] after task 4: selected context_blend_selected_global
 proxy_acc=0.9583 score=1.3784
 [probe_proto__zf_proto seed=1] probe final task accs: [0.979, 0.963, 0.896,
 0.848, 0.824]
 [probe_proto__zf_oracle seed=1] probe final task accs: [0.974, 0.958, 0.948,
 0.935, 0.943]
 [probe_proto__transformed0_oracle seed=1] host probe accs: [0.623, 0.632, 0.879,
 0.444, 0.5]
 [probe_proto__transformed1_oracle seed=1] host probe accs: [0.779, 0.886, 0.419,
 0.191, 0.656]
 [probe_proto__transformed2_oracle seed=1] host probe accs: [0.766, 0.64, 0.451,
 0.674, 0.899]
 [probe_proto__transformed3_oracle seed=1] host probe accs: [0.83, 0.838, 0.814,
 0.844, 0.912]
 [probe_proto__transformed4_oracle seed=1] host probe accs: [0.782, 0.601, 0.152,
 0.207, 0.703]
 [baseline_finetune seed=1] after task 0: [0.98]
 [baseline_finetune seed=1] after task 1: [0.487, 0.985]
 [baseline_finetune seed=1] after task 2: [0.0, 0.477, 0.961]
 [baseline_finetune seed=1] after task 3: [0.0, 0.0, 0.464, 0.943]
 [baseline_finetune seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_ewc seed=1] after task 0: [0.98]
 [baseline_ewc seed=1] after task 1: [0.492, 0.978]
 [baseline_ewc seed=1] after task 2: [0.0, 0.484, 0.956]
 [baseline_ewc seed=1] after task 3: [0.0, 0.0, 0.464, 0.948]
 [baseline_ewc seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_lwf seed=1] after task 0: [0.98]
 [baseline_lwf seed=1] after task 1: [0.97, 0.505]
 [baseline_lwf seed=1] after task 2: [0.925, 0.954, 0.52]
 [baseline_lwf seed=1] after task 3: [0.245, 0.62, 0.9, 0.614]
 [baseline_lwf seed=1] after task 4: [0.0, 0.0, 0.315, 0.814, 0.985]
 [baseline_experience_replay seed=1] after task 0: [0.98]
 [baseline_experience_replay seed=1] after task 1: [0.945, 0.966]
 [baseline_experience_replay seed=1] after task 2: [0.894, 0.927, 0.925]
 [baseline_experience_replay seed=1] after task 3: [0.87, 0.777, 0.802, 0.891]

[baseline_experience_replay seed=1] after task 4: [0.922, 0.765, 0.635, 0.816, 0.97]
 [baseline_balanced_replay seed=1] after task 0: [0.98]
 [baseline_balanced_replay seed=1] after task 1: [0.945, 0.966]
 [baseline_balanced_replay seed=1] after task 2: [0.894, 0.927, 0.925]
 [baseline_balanced_replay seed=1] after task 3: [0.87, 0.777, 0.802, 0.891]
 [baseline_balanced_replay seed=1] after task 4: [0.922, 0.765, 0.635, 0.816, 0.97]
 [baseline_sparse_moe seed=1] after task 0: [0.975]
 [baseline_sparse_moe seed=1] after task 1: [0.492, 0.985]
 [baseline_sparse_moe seed=1] after task 2: [0.0, 0.486, 0.956]
 [baseline_sparse_moe seed=1] after task 3: [0.0, 0.0, 0.46, 0.944]
 [baseline_sparse_moe seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_pnn seed=1] after task 0: [0.976]
 [baseline_pnn seed=1] after task 1: [0.487, 0.988]
 [baseline_pnn seed=1] after task 2: [0.0, 0.486, 0.956]
 [baseline_pnn seed=1] after task 3: [0.0, 0.0, 0.469, 0.948]
 [baseline_pnn seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_joint_upper_bound seed=1] final joint eval: [0.911, 0.904, 0.875, 0.869, 0.846]
 [base_proto seed=2] task=0 epoch=1/8 loss=2.0397 task=0.9444 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=2/8 loss=0.6053 task=0.3010 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=3/8 loss=0.4877 task=0.2358 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=4/8 loss=0.4408 task=0.2075 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=5/8 loss=0.4283 task=0.2043 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=6/8 loss=0.4158 task=0.2018 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=7/8 loss=0.3971 task=0.1905 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] task=0 epoch=8/8 loss=0.3865 task=0.1881 memory=0.0000
 balanced=0.0000
 [base_proto seed=2] calibrated context temperature after task 0: 0.25
 [base_proto seed=2] task=1 epoch=1/8 loss=6.5111 task=1.5935 memory=0.6193
 balanced=0.2447
 [base_proto seed=2] task=1 epoch=2/8 loss=3.2524 task=0.4515 memory=0.2993
 balanced=0.2131
 [base_proto seed=2] task=1 epoch=3/8 loss=2.3255 task=0.3479 memory=0.1581
 balanced=0.2302
 [base_proto seed=2] task=1 epoch=4/8 loss=1.9621 task=0.3313 memory=0.1143
 balanced=0.2331
 [base_proto seed=2] task=1 epoch=5/8 loss=1.8228 task=0.3098 memory=0.0754
 balanced=0.2259
 [base_proto seed=2] task=1 epoch=6/8 loss=1.7436 task=0.2865 memory=0.0674

balanced=0.2095
 [base_proto seed=2] task=1 epoch=7/8 loss=1.6431 task=0.2666 memory=0.0568
 balanced=0.2044
 [base_proto seed=2] task=1 epoch=8/8 loss=1.5738 task=0.2519 memory=0.0540
 balanced=0.1921
 [base_proto seed=2] calibrated context temperature after task 1: 0.80
 [base_proto seed=2] task=2 epoch=1/8 loss=5.1642 task=1.5283 memory=0.3634
 balanced=0.2751
 [base_proto seed=2] task=2 epoch=2/8 loss=3.2974 task=0.7620 memory=0.3398
 balanced=0.2502
 [base_proto seed=2] task=2 epoch=3/8 loss=2.6253 task=0.4739 memory=0.2383
 balanced=0.2683
 [base_proto seed=2] task=2 epoch=4/8 loss=2.2398 task=0.3885 memory=0.1647
 balanced=0.2367
 [base_proto seed=2] task=2 epoch=5/8 loss=1.9636 task=0.3088 memory=0.1118
 balanced=0.2171
 [base_proto seed=2] task=2 epoch=6/8 loss=1.7815 task=0.2729 memory=0.0816
 balanced=0.1952
 [base_proto seed=2] task=2 epoch=7/8 loss=1.6984 task=0.2527 memory=0.0660
 balanced=0.1896
 [base_proto seed=2] task=2 epoch=8/8 loss=1.6223 task=0.2455 memory=0.0554
 balanced=0.1871
 [base_proto seed=2] calibrated context temperature after task 2: 0.80
 [base_proto seed=2] task=3 epoch=1/8 loss=6.5100 task=2.3757 memory=1.1634
 balanced=0.3128
 [base_proto seed=2] task=3 epoch=2/8 loss=3.9558 task=1.1287 memory=0.8516
 balanced=0.2534
 [base_proto seed=2] task=3 epoch=3/8 loss=3.0964 task=0.7906 memory=0.5680
 balanced=0.2549
 [base_proto seed=2] task=3 epoch=4/8 loss=3.0956 task=0.6363 memory=0.8558
 balanced=0.2349
 [base_proto seed=2] task=3 epoch=5/8 loss=2.6420 task=0.5894 memory=0.5195
 balanced=0.2347
 [base_proto seed=2] task=3 epoch=6/8 loss=2.3708 task=0.5143 memory=0.3817
 balanced=0.2194
 [base_proto seed=2] task=3 epoch=7/8 loss=2.2239 task=0.4737 memory=0.3398
 balanced=0.2033
 [base_proto seed=2] task=3 epoch=8/8 loss=2.0686 task=0.4045 memory=0.3093
 balanced=0.1774
 [base_proto seed=2] calibrated context temperature after task 3: 0.60
 [base_proto seed=2] task=4 epoch=1/8 loss=5.7209 task=1.4460 memory=1.6991
 balanced=0.6096
 [base_proto seed=2] task=4 epoch=2/8 loss=3.1962 task=0.4697 memory=0.7557
 balanced=0.4623
 [base_proto seed=2] task=4 epoch=3/8 loss=2.6306 task=0.4175 memory=0.5424
 balanced=0.3565
 [base_proto seed=2] task=4 epoch=4/8 loss=2.7051 task=0.3660 memory=0.7190
 balanced=0.3633

```

[base_proto seed=2] task=4 epoch=5/8 loss=2.2111 task=0.3472 memory=0.4570
balanced=0.2739
[base_proto seed=2] task=4 epoch=6/8 loss=1.9362 task=0.3130 memory=0.2635
balanced=0.2935
[base_proto seed=2] task=4 epoch=7/8 loss=1.8509 task=0.2997 memory=0.2677
balanced=0.2827
[base_proto seed=2] task=4 epoch=8/8 loss=1.9189 task=0.2815 memory=0.3253
balanced=0.3328
[base_proto seed=2] calibrated context temperature after task 4: 1.00
[base_proto seed=2] completed in 247.0s
[RC2b seed=2] after task 0: selected proto_global_no_repair proxy_acc=0.9766
score=1.1672
[RC2b seed=2] after task 1: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9531 score=1.3324
[RC2b seed=2] after task 2: selected proto_global_no_repair proxy_acc=0.9336
score=1.2992
[RC2b seed=2] after task 3: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9219 score=1.2792
[RC2b seed=2] after task 4: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9479 score=1.3615
[probe_proto__zf_proto seed=2] probe final task accs: [0.964, 0.94, 0.843,
0.921, 0.909]
[probe_proto__zf_oracle seed=2] probe final task accs: [0.97, 0.955, 0.966,
0.93, 0.931]
[probe_proto__transformed0_oracle seed=2] host probe accs: [0.807, 0.83, 0.733,
0.398, 0.546]
[probe_proto__transformed1_oracle seed=2] host probe accs: [0.713, 0.599, 0.65,
0.82, 0.799]
[probe_proto__transformed2_oracle seed=2] host probe accs: [0.886, 0.876, 0.828,
0.818, 0.906]
[probe_proto__transformed3_oracle seed=2] host probe accs: [0.718, 0.69, 0.575,
0.634, 0.836]
[probe_proto__transformed4_oracle seed=2] host probe accs: [0.705, 0.824, 0.936,
0.465, 0.496]
[baseline_finetune seed=2] after task 0: [0.98]
[baseline_finetune seed=2] after task 1: [0.497, 0.983]
[baseline_finetune seed=2] after task 2: [0.0, 0.492, 0.97]
[baseline_finetune seed=2] after task 3: [0.0, 0.0, 0.468, 0.931]
[baseline_finetune seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
[baseline_ewc seed=2] after task 0: [0.98]
[baseline_ewc seed=2] after task 1: [0.495, 0.984]
[baseline_ewc seed=2] after task 2: [0.0, 0.489, 0.974]
[baseline_ewc seed=2] after task 3: [0.0, 0.0, 0.477, 0.946]
[baseline_ewc seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
[baseline_lwf seed=2] after task 0: [0.98]
[baseline_lwf seed=2] after task 1: [0.963, 0.516]
[baseline_lwf seed=2] after task 2: [0.89, 0.974, 0.521]
[baseline_lwf seed=2] after task 3: [0.31, 0.64, 0.916, 0.721]

```

[baseline_lwf seed=2] after task 4: [0.018, 0.0, 0.326, 0.826, 0.989]
 [baseline_experience_replay seed=2] after task 0: [0.98]
 [baseline_experience_replay seed=2] after task 1: [0.951, 0.978]
 [baseline_experience_replay seed=2] after task 2: [0.891, 0.956, 0.939]
 [baseline_experience_replay seed=2] after task 3: [0.895, 0.831, 0.839, 0.853]
 [baseline_experience_replay seed=2] after task 4: [0.894, 0.748, 0.726, 0.902, 0.97]
 [baseline_balanced_replay seed=2] after task 0: [0.98]
 [baseline_balanced_replay seed=2] after task 1: [0.951, 0.978]
 [baseline_balanced_replay seed=2] after task 2: [0.891, 0.956, 0.939]
 [baseline_balanced_replay seed=2] after task 3: [0.895, 0.831, 0.839, 0.853]
 [baseline_balanced_replay seed=2] after task 4: [0.894, 0.748, 0.726, 0.902, 0.97]
 [baseline_sparse_moe seed=2] after task 0: [0.984]
 [baseline_sparse_moe seed=2] after task 1: [0.495, 0.983]
 [baseline_sparse_moe seed=2] after task 2: [0.0, 0.482, 0.968]
 [baseline_sparse_moe seed=2] after task 3: [0.0, 0.0, 0.479, 0.943]
 [baseline_sparse_moe seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_pnn seed=2] after task 0: [0.98]
 [baseline_pnn seed=2] after task 1: [0.497, 0.984]
 [baseline_pnn seed=2] after task 2: [0.0, 0.491, 0.969]
 [baseline_pnn seed=2] after task 3: [0.0, 0.0, 0.499, 0.885]
 [baseline_pnn seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.991]
 [baseline_joint_upper_bound seed=2] final joint eval: [0.94, 0.899, 0.851, 0.818, 0.879]
 [base_proto seed=3] task=0 epoch=1/8 loss=1.7168 task=0.9251 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=2/8 loss=0.6411 task=0.3351 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=3/8 loss=0.5219 task=0.2716 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=4/8 loss=0.4611 task=0.2288 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=5/8 loss=0.4355 task=0.2164 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=6/8 loss=0.4179 task=0.2084 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=7/8 loss=0.4105 task=0.2095 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] task=0 epoch=8/8 loss=0.3897 task=0.1976 memory=0.0000
 balanced=0.0000
 [base_proto seed=3] calibrated context temperature after task 0: 0.25
 [base_proto seed=3] task=1 epoch=1/8 loss=5.3004 task=1.2692 memory=0.4542
 balanced=0.2134
 [base_proto seed=3] task=1 epoch=2/8 loss=2.6863 task=0.3874 memory=0.1556
 balanced=0.2098
 [base_proto seed=3] task=1 epoch=3/8 loss=2.1135 task=0.3177 memory=0.0605
 balanced=0.1991

```

[base_proto seed=3] task=1 epoch=4/8 loss=1.8606 task=0.2899 memory=0.0362
balanced=0.1978
[base_proto seed=3] task=1 epoch=5/8 loss=1.7236 task=0.2645 memory=0.0296
balanced=0.1838
[base_proto seed=3] task=1 epoch=6/8 loss=1.6693 task=0.2527 memory=0.0252
balanced=0.1789
[base_proto seed=3] task=1 epoch=7/8 loss=1.5905 task=0.2379 memory=0.0258
balanced=0.1617
[base_proto seed=3] task=1 epoch=8/8 loss=1.5140 task=0.2267 memory=0.0236
balanced=0.1555
[base_proto seed=3] calibrated context temperature after task 1: 0.80
[base_proto seed=3] task=2 epoch=1/8 loss=4.9913 task=1.3832 memory=0.5394
balanced=0.2704
[base_proto seed=3] task=2 epoch=2/8 loss=3.0276 task=0.6518 memory=0.3594
balanced=0.2442
[base_proto seed=3] task=2 epoch=3/8 loss=2.4574 task=0.4861 memory=0.2192
balanced=0.2257
[base_proto seed=3] task=2 epoch=4/8 loss=2.1622 task=0.4218 memory=0.1605
balanced=0.2065
[base_proto seed=3] task=2 epoch=5/8 loss=1.9097 task=0.3673 memory=0.1336
balanced=0.1953
[base_proto seed=3] task=2 epoch=6/8 loss=1.7711 task=0.3317 memory=0.1056
balanced=0.1798
[base_proto seed=3] task=2 epoch=7/8 loss=1.6853 task=0.3177 memory=0.1044
balanced=0.1736
[base_proto seed=3] task=2 epoch=8/8 loss=1.5633 task=0.2836 memory=0.0952
balanced=0.1399
[base_proto seed=3] calibrated context temperature after task 2: 0.60
[base_proto seed=3] task=3 epoch=1/8 loss=5.3704 task=1.9456 memory=0.6752
balanced=0.2698
[base_proto seed=3] task=3 epoch=2/8 loss=3.6252 task=0.9292 memory=0.7020
balanced=0.2487
[base_proto seed=3] task=3 epoch=3/8 loss=2.9037 task=0.7257 memory=0.4809
balanced=0.2040
[base_proto seed=3] task=3 epoch=4/8 loss=2.4155 task=0.5305 memory=0.3106
balanced=0.2035
[base_proto seed=3] task=3 epoch=5/8 loss=2.0381 task=0.4086 memory=0.2001
balanced=0.1564
[base_proto seed=3] task=3 epoch=6/8 loss=1.8397 task=0.3290 memory=0.1510
balanced=0.1543
[base_proto seed=3] task=3 epoch=7/8 loss=1.6912 task=0.3004 memory=0.1194
balanced=0.1209
[base_proto seed=3] task=3 epoch=8/8 loss=1.6148 task=0.2871 memory=0.1007
balanced=0.1212
[base_proto seed=3] calibrated context temperature after task 3: 0.60
[base_proto seed=3] task=4 epoch=1/8 loss=4.9632 task=1.6479 memory=0.9139
balanced=0.5925
[base_proto seed=3] task=4 epoch=2/8 loss=2.7452 task=0.5903 memory=0.2955

```

balanced=0.4873
 [base_proto seed=3] task=4 epoch=3/8 loss=2.2233 task=0.3175 memory=0.2431
 balanced=0.4652
 [base_proto seed=3] task=4 epoch=4/8 loss=2.0204 task=0.2791 memory=0.1994
 balanced=0.4631
 [base_proto seed=3] task=4 epoch=5/8 loss=1.8123 task=0.2578 memory=0.1488
 balanced=0.4124
 [base_proto seed=3] task=4 epoch=6/8 loss=1.7498 task=0.2532 memory=0.1560
 balanced=0.3997
 [base_proto seed=3] task=4 epoch=7/8 loss=1.6558 task=0.2373 memory=0.1494
 balanced=0.3795
 [base_proto seed=3] task=4 epoch=8/8 loss=1.6848 task=0.2364 memory=0.1825
 balanced=0.3962
 [base_proto seed=3] calibrated context temperature after task 4: 0.60
 [base_proto seed=3] completed in 207.2s
 [RC2b seed=3] after task 0: selected proto_global_no_repair proxy_acc=0.9297
 score=1.1016
 [RC2b seed=3] after task 1: selected proto_global_head_ce_kl_guard_035
 proxy_acc=0.9062 score=1.2668
 [RC2b seed=3] after task 2: selected context_temp_blend_selected_head_guard_035
 proxy_acc=0.9336 score=1.2868
 [RC2b seed=3] after task 3: selected context_blend_selected_global
 proxy_acc=0.9125 score=1.2678
 [RC2b seed=3] after task 4: selected proto_global_head_ce_kl_guard_035
 proxy_acc=0.9401 score=1.3412
 [probe_proto__zf_proto seed=3] probe final task accs: [0.909, 0.969, 0.909,
 0.799, 0.944]
 [probe_proto__zf_oracle seed=3] probe final task accs: [0.975, 0.98, 0.848,
 0.875, 0.98]
 [probe_proto__transformed0_oracle seed=3] host probe accs: [0.754, 0.912, 0.879,
 0.441, 0.484]
 [probe_proto__transformed1_oracle seed=3] host probe accs: [0.855, 0.854, 0.828,
 0.841, 0.902]
 [probe_proto__transformed2_oracle seed=3] host probe accs: [0.7, 0.752, 0.701,
 0.396, 0.579]
 [probe_proto__transformed3_oracle seed=3] host probe accs: [0.787, 0.823, 0.85,
 0.66, 0.708]
 [probe_proto__transformed4_oracle seed=3] host probe accs: [0.757, 0.705, 0.631,
 0.68, 0.806]
 [baseline_finetune seed=3] after task 0: [0.973]
 [baseline_finetune seed=3] after task 1: [0.496, 0.984]
 [baseline_finetune seed=3] after task 2: [0.0, 0.486, 0.964]
 [baseline_finetune seed=3] after task 3: [0.0, 0.0, 0.477, 0.931]
 [baseline_finetune seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
 [baseline_ewc seed=3] after task 0: [0.973]
 [baseline_ewc seed=3] after task 1: [0.494, 0.986]
 [baseline_ewc seed=3] after task 2: [0.0, 0.486, 0.965]
 [baseline_ewc seed=3] after task 3: [0.0, 0.0, 0.482, 0.936]

[baseline_ewc seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_lwf seed=3] after task 0: [0.973]
 [baseline_lwf seed=3] after task 1: [0.984, 0.5]
 [baseline_lwf seed=3] after task 2: [0.938, 0.929, 0.468]
 [baseline_lwf seed=3] after task 3: [0.312, 0.362, 0.86, 0.748]
 [baseline_lwf seed=3] after task 4: [0.172, 0.0, 0.385, 0.877, 0.98]
 [baseline_experience_replay seed=3] after task 0: [0.973]
 [baseline_experience_replay seed=3] after task 1: [0.955, 0.958]
 [baseline_experience_replay seed=3] after task 2: [0.891, 0.949, 0.936]
 [baseline_experience_replay seed=3] after task 3: [0.877, 0.781, 0.775, 0.876]
 [baseline_experience_replay seed=3] after task 4: [0.877, 0.821, 0.764, 0.844, 0.946]
 [baseline_balanced_replay seed=3] after task 0: [0.973]
 [baseline_balanced_replay seed=3] after task 1: [0.955, 0.958]
 [baseline_balanced_replay seed=3] after task 2: [0.891, 0.949, 0.936]
 [baseline_balanced_replay seed=3] after task 3: [0.877, 0.781, 0.775, 0.876]
 [baseline_balanced_replay seed=3] after task 4: [0.877, 0.821, 0.764, 0.844, 0.946]
 [baseline_sparse_moe seed=3] after task 0: [0.991]
 [baseline_sparse_moe seed=3] after task 1: [0.494, 0.983]
 [baseline_sparse_moe seed=3] after task 2: [0.0, 0.477, 0.968]
 [baseline_sparse_moe seed=3] after task 3: [0.0, 0.0, 0.49, 0.943]
 [baseline_sparse_moe seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
 [baseline_pnn seed=3] after task 0: [0.973]
 [baseline_pnn seed=3] after task 1: [0.496, 0.983]
 [baseline_pnn seed=3] after task 2: [0.0, 0.486, 0.963]
 [baseline_pnn seed=3] after task 3: [0.0, 0.0, 0.454, 0.924]
 [baseline_pnn seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_joint_upper_bound seed=3] final joint eval: [0.939, 0.892, 0.84, 0.805, 0.881]
 [base_proto seed=4] task=0 epoch=1/8 loss=1.8377 task=0.8274 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=2/8 loss=0.5534 task=0.2754 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=3/8 loss=0.4730 task=0.2277 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=4/8 loss=0.4159 task=0.1908 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=5/8 loss=0.3981 task=0.1821 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=6/8 loss=0.3867 task=0.1790 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=7/8 loss=0.3819 task=0.1819 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] task=0 epoch=8/8 loss=0.3773 task=0.1844 memory=0.0000
 balanced=0.0000
 [base_proto seed=4] calibrated context temperature after task 0: 0.25
 [base_proto seed=4] task=1 epoch=1/8 loss=5.4183 task=1.4637 memory=0.4375

balanced=0.2131
 [base_proto seed=4] task=1 epoch=2/8 loss=2.9962 task=0.6040 memory=0.1977
 balanced=0.1871
 [base_proto seed=4] task=1 epoch=3/8 loss=2.3804 task=0.3889 memory=0.1252
 balanced=0.1923
 [base_proto seed=4] task=1 epoch=4/8 loss=2.0680 task=0.3454 memory=0.0844
 balanced=0.1764
 [base_proto seed=4] task=1 epoch=5/8 loss=1.9159 task=0.3204 memory=0.0601
 balanced=0.1639
 [base_proto seed=4] task=1 epoch=6/8 loss=1.8193 task=0.2982 memory=0.0518
 balanced=0.1547
 [base_proto seed=4] task=1 epoch=7/8 loss=1.7361 task=0.2844 memory=0.0498
 balanced=0.1421
 [base_proto seed=4] task=1 epoch=8/8 loss=1.7106 task=0.2808 memory=0.0596
 balanced=0.1391
 [base_proto seed=4] calibrated context temperature after task 1: 0.60
 [base_proto seed=4] task=2 epoch=1/8 loss=5.1953 task=1.4983 memory=0.3977
 balanced=0.2978
 [base_proto seed=4] task=2 epoch=2/8 loss=3.2786 task=0.8136 memory=0.2887
 balanced=0.3217
 [base_proto seed=4] task=2 epoch=3/8 loss=2.5224 task=0.4726 memory=0.1686
 balanced=0.2891
 [base_proto seed=4] task=2 epoch=4/8 loss=2.1573 task=0.3806 memory=0.1073
 balanced=0.2611
 [base_proto seed=4] task=2 epoch=5/8 loss=1.8892 task=0.3272 memory=0.0793
 balanced=0.2052
 [base_proto seed=4] task=2 epoch=6/8 loss=1.7000 task=0.2999 memory=0.0679
 balanced=0.1787
 [base_proto seed=4] task=2 epoch=7/8 loss=1.6482 task=0.2822 memory=0.0776
 balanced=0.1789
 [base_proto seed=4] task=2 epoch=8/8 loss=1.6045 task=0.2745 memory=0.0799
 balanced=0.1763
 [base_proto seed=4] calibrated context temperature after task 2: 0.80
 [base_proto seed=4] task=3 epoch=1/8 loss=5.7505 task=2.1065 memory=0.8068
 balanced=0.3040
 [base_proto seed=4] task=3 epoch=2/8 loss=3.2543 task=0.7429 memory=0.4585
 balanced=0.2736
 [base_proto seed=4] task=3 epoch=3/8 loss=2.4797 task=0.4893 memory=0.2020
 balanced=0.2673
 [base_proto seed=4] task=3 epoch=4/8 loss=2.1952 task=0.3860 memory=0.1390
 balanced=0.2542
 [base_proto seed=4] task=3 epoch=5/8 loss=2.0103 task=0.3400 memory=0.1117
 balanced=0.2364
 [base_proto seed=4] task=3 epoch=6/8 loss=1.8794 task=0.3111 memory=0.0938
 balanced=0.2124
 [base_proto seed=4] task=3 epoch=7/8 loss=1.8278 task=0.2907 memory=0.0898
 balanced=0.2223
 [base_proto seed=4] task=3 epoch=8/8 loss=1.7703 task=0.2844 memory=0.0819

```

balanced=0.2199
[base_proto seed=4] calibrated context temperature after task 3: 0.80
[base_proto seed=4] task=4 epoch=1/8 loss=8.0603 task=1.9888 memory=2.9831
balanced=1.1856
[base_proto seed=4] task=4 epoch=2/8 loss=3.5411 task=0.6399 memory=0.7924
balanced=0.5625
[base_proto seed=4] task=4 epoch=3/8 loss=3.4593 task=0.3135 memory=1.0801
balanced=0.7081
[base_proto seed=4] task=4 epoch=4/8 loss=2.2786 task=0.2834 memory=0.2962
balanced=0.4461
[base_proto seed=4] task=4 epoch=5/8 loss=2.5325 task=0.2338 memory=0.5335
balanced=0.5983
[base_proto seed=4] task=4 epoch=6/8 loss=1.8937 task=0.2221 memory=0.1835
balanced=0.4162
[base_proto seed=4] task=4 epoch=7/8 loss=2.0464 task=0.2054 memory=0.3986
balanced=0.4461
[base_proto seed=4] task=4 epoch=8/8 loss=2.1608 task=0.2207 memory=0.5008
balanced=0.4855
[base_proto seed=4] calibrated context temperature after task 4: 0.80
[base_proto seed=4] completed in 290.8s
[RC2b seed=4] after task 0: selected proto_global_no_repair proxy_acc=0.9062
score=1.0750
[RC2b seed=4] after task 1: selected proto_global_no_repair proxy_acc=0.9271
score=1.2958
[RC2b seed=4] after task 2: selected proto_global_no_repair proxy_acc=0.9258
score=1.2789
[RC2b seed=4] after task 3: selected context_temp_blend_selected_global
proxy_acc=0.9563 score=1.3334
[RC2b seed=4] after task 4: selected context_temp_blend_selected_head_guard_035
proxy_acc=0.9375 score=1.3157
[probe_proto__zf_proto seed=4] probe final task accs: [0.945, 0.953, 0.741,
0.851, 0.91]
[probe_proto__zf_oracle seed=4] probe final task accs: [0.958, 0.92, 0.943,
0.907, 0.964]
[probe_proto__transformed0_oracle seed=4] host probe accs: [0.84, 0.759, 0.666,
0.782, 0.846]
[probe_proto__transformed1_oracle seed=4] host probe accs: [0.738, 0.671, 0.316,
0.445, 0.739]
[probe_proto__transformed2_oracle seed=4] host probe accs: [0.574, 0.456, 0.608,
0.81, 0.828]
[probe_proto__transformed3_oracle seed=4] host probe accs: [0.844, 0.839, 0.825,
0.579, 0.681]
[probe_proto__transformed4_oracle seed=4] host probe accs: [0.704, 0.762, 0.736,
0.263, 0.496]
[baseline_finetune seed=4] after task 0: [0.98]
[baseline_finetune seed=4] after task 1: [0.495, 0.989]
[baseline_finetune seed=4] after task 2: [0.0, 0.491, 0.964]
[baseline_finetune seed=4] after task 3: [0.0, 0.0, 0.476, 0.934]

```

```

[baseline_finetune seed=4] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
[baseline_ewc seed=4] after task 0: [0.98]
[baseline_ewc seed=4] after task 1: [0.492, 0.986]
[baseline_ewc seed=4] after task 2: [0.0, 0.492, 0.958]
[baseline_ewc seed=4] after task 3: [0.0, 0.0, 0.489, 0.94]
[baseline_ewc seed=4] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
[baseline_lwf seed=4] after task 0: [0.98]
[baseline_lwf seed=4] after task 1: [0.983, 0.499]
[baseline_lwf seed=4] after task 2: [0.926, 0.96, 0.482]
[baseline_lwf seed=4] after task 3: [0.044, 0.357, 0.836, 0.741]
[baseline_lwf seed=4] after task 4: [0.001, 0.003, 0.367, 0.866, 0.985]
[baseline_experience_replay seed=4] after task 0: [0.98]
[baseline_experience_replay seed=4] after task 1: [0.948, 0.976]
[baseline_experience_replay seed=4] after task 2: [0.885, 0.966, 0.932]
[baseline_experience_replay seed=4] after task 3: [0.836, 0.801, 0.825, 0.884]
[baseline_experience_replay seed=4] after task 4: [0.914, 0.777, 0.681, 0.851,
0.984]
[baseline_balanced_replay seed=4] after task 0: [0.98]
[baseline_balanced_replay seed=4] after task 1: [0.948, 0.976]
[baseline_balanced_replay seed=4] after task 2: [0.885, 0.966, 0.932]
[baseline_balanced_replay seed=4] after task 3: [0.836, 0.801, 0.825, 0.884]
[baseline_balanced_replay seed=4] after task 4: [0.914, 0.777, 0.681, 0.851,
0.984]
[baseline_sparse_moe seed=4] after task 0: [0.989]
[baseline_sparse_moe seed=4] after task 1: [0.495, 0.994]
[baseline_sparse_moe seed=4] after task 2: [0.0, 0.486, 0.966]
[baseline_sparse_moe seed=4] after task 3: [0.0, 0.0, 0.48, 0.945]
[baseline_sparse_moe seed=4] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
[baseline_pnn seed=4] after task 0: [0.986]
[baseline_pnn seed=4] after task 1: [0.492, 0.989]
[baseline_pnn seed=4] after task 2: [0.0, 0.49, 0.961]
[baseline_pnn seed=4] after task 3: [0.0, 0.0, 0.484, 0.94]
[baseline_pnn seed=4] after task 4: [0.0, 0.0, 0.0, 0.499, 0.998]

```

1.18 13. Scorecards and diagnostic tables

The central comparison is paired delta versus paired_proto_global_no_repair and versus the corrected paired_proto_global_head_ce_kl_guard_035, not independent model-vs-model training.

```

[ ]: def final_scorecard(history_df, forgetting_df, confusion_df=None):
    final_task = int(history_df.after_task.max())
    final_hist = history_df[history_df.after_task == final_task]
    final_forgetting = forgetting_df[forgetting_df.after_task == final_task] if
↪not forgetting_df.empty else pd.DataFrame(columns=["method", "seed",
↪"avg_forgetting"])
    # Ensure optional columns exist.

```

```

    for col in ["backbone_params", "readout_trainable_params",
↪ "readout_total_params"]:
        if col not in final_hist.columns:
            final_hist[col] = np.nan
    acc_summary = final_hist.groupby(["method", "seed"], as_index=False).agg(
        model_family=("model_family", "first"),
        final_avg_acc=("acc", "mean"),
        min_task_acc=("acc", "min"),
        context_entropy_mean=("context_entropy", "mean"),
        context_margin_mean=("context_margin", "mean"),
        context_top1_acc_mean=("context_top1_acc", "mean"),
        latent_frechet_margin_mean=("latent_frechet_margin", "mean"),
        prototype_weight_mean=("prototype_weight", "mean"),
        class2_margin_vs4_mean=("class2_margin_vs4", "mean"),
        class5_margin_vs4_mean=("class5_margin_vs4", "mean"),
        class2_logit_rank_mean=("class2_logit_rank", "mean"),
        class5_logit_rank_mean=("class5_logit_rank", "mean"),
        trainable_params=("trainable_params", "max"),
        total_params=("total_params", "max"),
        backbone_params=("backbone_params", "max"),
        readout_trainable_params=("readout_trainable_params", "max"),
        readout_total_params=("readout_total_params", "max"),
        model_size_mb_total=("model_size_mb_total", "max"),
        probe_row=("probe_row", "max"),
        backbone_source=("backbone_source", "first"),
        probe_feature=("probe_feature", "first"),
        probe_context_mode=("probe_context_mode", "first"),
    )
    mid = final_hist[final_hist.task_id.isin([2, 3])].groupby(["method",
↪ "seed"], as_index=False).agg(
        middle_avg_acc=("acc", "mean"), middle_min_acc=("acc", "min")
    )
    merged = acc_summary.merge(mid, on=["method", "seed"], how="left")
    merged = merged.merge(final_forgetting[["method", "seed",
↪ "avg_forgetting"]], on=["method", "seed"], how="left")
    merged.loc[merged["probe_row"].astype(bool), "avg_forgetting"] = np.nan
    class_diag = class_diagnostics_from_confusion(confusion_df) if confusion_df
↪ is not None and not confusion_df.empty else pd.DataFrame()
    if not class_diag.empty:
        merged = merged.merge(class_diag, on=["method", "seed"], how="left")
    else:
        for col in ["old_class_acc", "recent_class_acc", "class0_acc",
↪ "class1_acc", "class2_acc", "class3_acc", "class4_acc", "class5_acc",
↪ "min_class_acc", "pred4_bias"]:
            merged[col] = np.nan
    summary = merged.groupby("method", as_index=False).agg(
        model_family=("model_family", "first"),

```

```

final_avg_acc_mean=("final_avg_acc", "mean"),
final_avg_acc_std=("final_avg_acc", "std"),
min_task_acc_mean=("min_task_acc", "mean"),
avg_forgetting_mean=("avg_forgetting", "mean"),
middle_avg_acc_mean=("middle_avg_acc", "mean"),
middle_min_acc_mean=("middle_min_acc", "mean"),
old_class_acc_mean=("old_class_acc", "mean"),
recent_class_acc_mean=("recent_class_acc", "mean"),
class0_acc_mean=("class0_acc", "mean"),
class1_acc_mean=("class1_acc", "mean"),
class2_acc_mean=("class2_acc", "mean"),
class3_acc_mean=("class3_acc", "mean"),
class4_acc_mean=("class4_acc", "mean"),
class5_acc_mean=("class5_acc", "mean"),
min_class_acc_mean=("min_class_acc", "mean"),
pred4_bias_mean=("pred4_bias", "mean"),
class2_margin_vs4_mean=("class2_margin_vs4_mean", "mean"),
class5_margin_vs4_mean=("class5_margin_vs4_mean", "mean"),
class2_logit_rank_mean=("class2_logit_rank_mean", "mean"),
class5_logit_rank_mean=("class5_logit_rank_mean", "mean"),
trainable_params_mean=("trainable_params", "mean"),
total_params_mean=("total_params", "mean"),
backbone_params_mean=("backbone_params", "mean"),
readout_total_params_mean=("readout_total_params", "mean"),
model_size_mb_total_mean=("model_size_mb_total", "mean"),
probe_row=("probe_row", "max"),
backbone_source=("backbone_source", "first"),
probe_feature=("probe_feature", "first"),
probe_context_mode=("probe_context_mode", "first"),
)
return summary.sort_values("final_avg_acc_mean", ascending=False), merged

scorecard_df, per_seed_df = final_scorecard(history_df, forgetting_df,
↳confusion_df)
scorecard_path = RESULTS_DIR / f"scorecard_{RUN_MODE}.csv"
per_seed_path = RESULTS_DIR / f"per_seed_scorecard_{RUN_MODE}.csv"
scorecard_df.to_csv(scorecard_path, index=False)
per_seed_df.to_csv(per_seed_path, index=False)
print("Saved scorecards:")
print(" -", scorecard_path)
print(" -", per_seed_path)
display(scorecard_df)

# -----
# Diagnostic patch: separate deployable methods from diagnostic ceilings/probes.
# -----
def method_role(method: str, probe_row: bool = False):

```

```

m = str(method)
if bool(probe_row):
    return "diagnostic_probe"
if "joint_upper_bound" in m:
    return "diagnostic_upper_bound"
if "oracle" in m:
    return "diagnostic_oracle_reference"
if m.startswith("paired_"):
    return "deployable_mmals_paired"
if m.startswith("baseline_"):
    return "deployable_continual_baseline"
return "other"

scorecard_df["method_role"] = [
    method_role(row.method, row.probe_row)
    for row in scorecard_df.itertuples(index=False)
]
per_seed_df["method_role"] = [
    method_role(row.method, row.probe_row)
    for row in per_seed_df.itertuples(index=False)
]

scorecard_path = RESULTS_DIR / f"scorecard_{RUN_MODE}.csv"
per_seed_path = RESULTS_DIR / f"per_seed_scorecard_{RUN_MODE}.csv"
scorecard_df.to_csv(scorecard_path, index=False)
per_seed_df.to_csv(per_seed_path, index=False)

compact_cols = [
    "method_role", "method", "model_family", "final_avg_acc_mean",
    "min_task_acc_mean", "avg_forgetting_mean", "class2_acc_mean",
    "class5_acc_mean", "pred4_bias_mean", "total_params_mean",
    "probe_row"
]
compact_cols = [c for c in compact_cols if c in scorecard_df.columns]
benchmark_groups_df = scorecard_df[compact_cols].sort_values(
    ["method_role", "final_avg_acc_mean"], ascending=[True, False]
)
benchmark_groups_path = RESULTS_DIR / f"benchmark_method_groups_{RUN_MODE}.csv"
benchmark_groups_df.to_csv(benchmark_groups_path, index=False)
print("Saved benchmark method groups:", benchmark_groups_path)
display(benchmark_groups_df)

```

1.19 14. Plots

Quick visual debugging plots for the paired RC1b candidate-freeze audit. These are not publication figures.

```

[ ]: if not scorecard_df.empty:
    for metric, title in [
        ("final_avg_acc_mean", "Final average accuracy"),
        ("middle_avg_acc_mean", "Middle tasks average accuracy"),
        ("class2_acc_mean", "Class 2 accuracy"),
        ("class5_acc_mean", "Class 5 accuracy"),
        ("class2_margin_vs4_mean", "Class 2 logit margin vs class 4"),
        ("class5_margin_vs4_mean", "Class 5 logit margin vs class 4"),
        ("pred4_bias_mean", "Predicted class-4 bias"),
        ("readout_total_params_mean", "Readout parameters"),
        ("total_params_mean", "Total parameters"),
    ]:
        if metric not in scorecard_df.columns:
            continue
        plt.figure(figsize=(11, 4))
        plt.bar(scorecard_df["method"], scorecard_df[metric])
        plt.xticks(rotation=35, ha="right")
        plt.ylabel(metric)
        plt.title(f"MMALS v1.0-RC1b - {title}")
        plt.tight_layout()
        plt.show()

if 'confusion_df' in globals() and not confusion_df.empty:
    for method in confusion_df.method.unique():
        cm = confusion_df[confusion_df.method == method].pivot_table(
            index="true_class", columns="pred_class", values="rate",
            fill_value=0.0
        ).reindex(index=range(N_CLASSES), columns=range(N_CLASSES),
            fill_value=0.0)
        plt.figure(figsize=(5, 4))
        plt.imshow(cm.values, aspect="auto")
        plt.xticks(range(N_CLASSES), range(N_CLASSES))
        plt.yticks(range(N_CLASSES), range(N_CLASSES))
        plt.xlabel("Predicted class")
        plt.ylabel("True class")
        plt.title(f"Class confusion - {method}")
        plt.colorbar(label="rate")
        plt.tight_layout()
        plt.show()

```

1.20 15. Automatic interpretation and RC1b validation gate

This interpretation is deliberately conservative. It separates deployable inferred-context variants from oracle references, compares the RC1b candidate against the true global guard, computes probe gaps, writes a pass/fail validation-gate table, and keeps the original RC1 row only as a rejected diagnostic.

```

[ ]: def get_row(scorecard_df, method):
    sub = scorecard_df[scorecard_df.method == method]
    return sub.iloc[0] if not sub.empty else None

def metric(row, name):
    if row is None or name not in row:
        return np.nan
    return float(row[name])

def method_role(method: str, probe_row: bool = False):
    """Classify methods so diagnostics are not mistaken for deployable models.
    ↪ """
    m = str(method)
    if bool(probe_row):
        return "diagnostic_probe"
    if "joint_upper_bound" in m:
        return "diagnostic_upper_bound"
    if "oracle" in m:
        return "diagnostic_oracle_reference"
    if m.startswith("paired_"):
        return "deployable_mmals_paired"
    if m.startswith("baseline_"):
        return "deployable_continual_baseline"
    return "other"

def non_diagnostic_scorecard(scorecard_df):
    if scorecard_df.empty:
        return scorecard_df
    if "method_role" not in scorecard_df.columns:
        tmp = scorecard_df.copy()
        tmp["method_role"] = [method_role(r.method, r.probe_row) for r in tmp.
↪ itertuples(index=False)]
    else:
        tmp = scorecard_df
        return tmp[tmp["method_role"].isin(["deployable_mmals_paired",
↪ "deployable_continual_baseline"])]

def paired_context_delta_table(scorecard_df):
    base = get_row(scorecard_df, "paired_proto_global_no_repair")
    global_guard = get_row(scorecard_df,
↪ "paired_proto_global_head_ce_kl_guard_035")
    rows = []
    for variant in SELECTED_CONTEXT_HEAD_VARIANTS:
        method = method_name_for_context_variant(variant)
        row = get_row(scorecard_df, method)
        if row is None or base is None:
            continue

```

```

        rows.append({
            "method": method,
            "is_oracle_reference": "oracle" in method,
            "delta_acc_vs_proto_no_repair": metric(row, "final_avg_acc_mean") -
↪metric(base, "final_avg_acc_mean"),
            "delta_acc_vs_proto_global_guard": metric(row,
↪"final_avg_acc_mean") - metric(global_guard, "final_avg_acc_mean") if
↪global_guard is not None else np.nan,
            "delta_min_task_vs_no_repair": metric(row, "min_task_acc_mean") -
↪metric(base, "min_task_acc_mean"),
            "delta_forgetting_vs_no_repair": metric(base,
↪"avg_forgetting_mean") - metric(row, "avg_forgetting_mean"),
            "delta_class2_vs_no_repair": metric(row, "class2_acc_mean") -
↪metric(base, "class2_acc_mean"),
            "delta_class5_vs_no_repair": metric(row, "class5_acc_mean") -
↪metric(base, "class5_acc_mean"),
            "context_temperature": metric(row, "context_entropy_mean"),
            "readout_params": metric(row, "readout_total_params_mean"),
        })
    return pd.DataFrame(rows).sort_values("delta_acc_vs_proto_no_repair",
↪ascending=False)

delta_df = paired_context_delta_table(scorecard_df)
delta_path = RESULTS_DIR / f"paired_context_deltas_{RUN_MODE}.csv"
delta_df.to_csv(delta_path, index=False)
print("Saved paired context deltas:", delta_path)
display(delta_df)

base = get_row(scorecard_df, "paired_proto_global_no_repair")
noop = get_row(scorecard_df, "paired_proto_kl_only_true_noop")
global_guard = get_row(scorecard_df, "paired_proto_global_head_ce_kl_guard_035")
oracle_ref = get_row(scorecard_df, "paired_oracle_global_no_repair_reference")
oracle_guard = get_row(scorecard_df,
↪"paired_oracle_global_head_ce_kl_guard_035_reference")

non_probe = scorecard_df[scorecard_df.probe_row == False] if not scorecard_df.
↪empty else pd.DataFrame()
deployable = non_diagnostic_scorecard(scorecard_df)
deployable_mmals = deployable[deployable.method.str.startswith("paired_",
↪na=False)] if not deployable.empty else pd.DataFrame()
deployable_baselines = deployable[deployable.method.str.startswith("baseline_",
↪na=False)] if not deployable.empty else pd.DataFrame()
context_selected = deployable_mmals[deployable_mmals.method.str.
↪contains("context_", case=False, na=False)] if not deployable_mmals.empty
↪else pd.DataFrame()

```

```

best_deployable = deployable.sort_values("final_avg_acc_mean", ascending=False).
    ↪iloc[0] if not deployable.empty else None
best_deployable_mmals = deployable_mmals.sort_values("final_avg_acc_mean",
    ↪ascending=False).iloc[0] if not deployable_mmals.empty else None
best_continual_baseline = deployable_baselines.
    ↪sort_values("final_avg_acc_mean", ascending=False).iloc[0] if not
    ↪deployable_baselines.empty else None
best_context = context_selected.sort_values("final_avg_acc_mean",
    ↪ascending=False).iloc[0] if not context_selected.empty else None

print("\n=== MMALS v1.0-RC1b automatic interpretation ===")
if base is not None:
    print(f"paired_proto_global_no_repair final_avg_acc = {metric(base,
    ↪'final_avg_acc_mean'):.4f}")
if noop is not None and base is not None:
    noop_delta = metric(noop, "final_avg_acc_mean") - metric(base,
    ↪"final_avg_acc_mean")
    print(f"proto_kl_only_true_noop delta vs proto_no_repair = {noop_delta:+.
    ↪6f}")
    if abs(noop_delta) < 1e-6:
        print("No-op control is clean: proto_kl_only_true_noop matches
    ↪proto_global_no_repair.")
    else:
        print("Warning: no-op differs from no_repair. Check deterministic
    ↪evaluation or audit implementation.")
if global_guard is not None:
    print(f"TRUE proto global CE+KL guard: {global_guard.method}
    ↪acc={global_guard.final_avg_acc_mean:.4f}")
    if base is not None:
        true_guard_delta = global_guard.final_avg_acc_mean - metric(base,
    ↪"final_avg_acc_mean")
        print(f"TRUE guard delta vs no-repair: {true_guard_delta:+.4f}")
        if abs(true_guard_delta) < 1e-6:
            print("Warning: TRUE guard still equals no-repair. Inspect the
    ↪guarded-head training logs.")
        else:
            print("Control-fix signal: TRUE guard differs from no-repair.")
if best_context is not None:
    print(f"Best deployable inferred-context variant: {best_context.method}
    ↪acc={best_context.final_avg_acc_mean:.4f}")
    if base is not None:
        print(f"Delta vs proto no-repair: {best_context.final_avg_acc_mean -
    ↪metric(base, 'final_avg_acc_mean'):.4f}")
    if global_guard is not None:
        print(f"Delta vs TRUE proto global guarded head: {best_context.
    ↪final_avg_acc_mean - global_guard.final_avg_acc_mean:.4f}")

```

```

    print(f"Class-2 delta vs no-repair: {best_context.class2_acc_mean -
↳metric(base, 'class2_acc_mean'):+.4f}")
    print(f"Min-task delta vs no-repair: {best_context.min_task_acc_mean -
↳metric(base, 'min_task_acc_mean'):+.4f}")
    print(f"Forgetting reduction vs no-repair: {metric(base,
↳'avg_forgetting_mean') - best_context.avg_forgetting_mean:+.4f}")
if best_deployable is not None:
    print(f"Best deployable method overall, excluding probes/oracle/joint upper
↳bound: {best_deployable.method} acc={best_deployable.final_avg_acc_mean:.
↳4f}")
if best_deployable_mmals is not None:
    print(f"Best deployable MMALS paired row: {best_deployable_mmals.method}
↳acc={best_deployable_mmals.final_avg_acc_mean:.4f}")
if best_continual_baseline is not None:
    print(f"Best deployable continual-learning baseline:
↳{best_continual_baseline.method} acc={best_continual_baseline.
↳final_avg_acc_mean:.4f}")
if oracle_ref is not None:
    print(f"Oracle context reference: {oracle_ref.method} acc={oracle_ref.
↳final_avg_acc_mean:.4f}")
if oracle_guard is not None:
    print(f"Oracle context + guarded head reference: {oracle_guard.method}
↳acc={oracle_guard.final_avg_acc_mean:.4f}")

probe_rows = scorecard_df[(scorecard_df.probe_row == True) & (scorecard_df.
↳probe_feature == "zf")] if not scorecard_df.empty else pd.DataFrame()
if not probe_rows.empty and best_deployable is not None:
    proto_probe = probe_rows[probe_rows.probe_context_mode == "proto"].
↳sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    oracle_probe = probe_rows[probe_rows.probe_context_mode == "oracle"].
↳sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    head_gap = proto_probe.final_avg_acc_mean - best_deployable.
↳final_avg_acc_mean
    context_gap = oracle_probe.final_avg_acc_mean - proto_probe.
↳final_avg_acc_mean
    total_gap = oracle_probe.final_avg_acc_mean - best_deployable.
↳final_avg_acc_mean
    print(f"Best proto-context z_f probe: {proto_probe.method} acc={proto_probe.
↳final_avg_acc_mean:.4f}")
    print(f"Best oracle-context z_f probe: {oracle_probe.method}
↳acc={oracle_probe.final_avg_acc_mean:.4f}")
    print(f"Remaining deployed-to-proto-probe head gap: {head_gap:+.4f}")
    print(f"Remaining proto-to-oracle context gap: {context_gap:+.4f}")
    print(f"Total deployed-to-oracle-probe gap: {total_gap:+.4f}")

```

```

# Explicit context-effect isolation: context-only vs context+guard.
context_only = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_global")
context_guard = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_head_guard_035")
if context_only is not None and context_guard is not None:
    print("\nContext effect isolation:")
    print(f"Context-only temp+blend: {context_only.final_avg_acc_mean:.4f}")
    print(f"Context+guard temp+blend: {context_guard.final_avg_acc_mean:.4f}")
    print(f"Added value of guarded readout after context selection:
    ↪{context_guard.final_avg_acc_mean - context_only.final_avg_acc_mean:+.4f}")
    if global_guard is not None:
        print(f"Context+guard vs TRUE global guard: {context_guard.
    ↪final_avg_acc_mean - global_guard.final_avg_acc_mean:+.4f}")

if RUN_MODE == "smoke":
    print("\nSmoke-mode note:")
    print("- Smoke is an execution and diagnostics check only.")
    print("- Do not freeze or reject RC1b from smoke-mode benchmark numbers.")
    print("- Evidence/robust/benchmark modes are required before scientific
    ↪ranking versus baselines.")

print("\nDecision rule:")
print("- If deployable context-selected variants beat the TRUE proto global
    ↪guard and reduce total gap, v1.0-RC1b supports freezing temp/blend
    ↪context-gap selected guarded MMALS.")
print("- If they only match the TRUE global guard, context selection is
    ↪diagnostic but not yet a sufficient architecture change.")
print("- If oracle reference remains much higher, the next branch should target
    ↪learned context posterior / context memory, not more readout-only variants.")
print("- If proto and oracle probes converge, remaining work returns to readout/
    ↪probe-gap closure.")
print("- Do not claim oracle rows as deployable task-free MMALS.")

# Compact benchmark separation table for interpretation.
if "method_role" in scorecard_df.columns:
    role_summary = scorecard_df.groupby("method_role", as_index=False).agg(
        best_acc=("final_avg_acc_mean", "max"),
        n_methods=("method", "count"),
    ).sort_values("best_acc", ascending=False)
    role_summary_path = RESULTS_DIR / f"benchmark_role_summary_{RUN_MODE}.csv"
    role_summary.to_csv(role_summary_path, index=False)
    print("\nSaved benchmark role summary:", role_summary_path)
    display(role_summary)

```

```

# -----
# v1.0-RC1b robust validation gate
# -----
PRIMARY_RC1 = "paired_context_gap_selected"
SECONDARY_RC1 = "paired_context_temp_blend_selected_head_guard_035"
REJECTED_ORIGINAL_RC1 = "paired_context_blend_selected_head_guard_035"
TRUE_GUARD = "paired_proto_global_head_ce_kl_guard_035"
BASELINE = "paired_proto_global_no_repair"
NOOP = "paired_proto_kl_only_true_noop"

def row_or_nan(method):
    return get_row(scorecard_df, method)

rc1 = row_or_nan(PRIMARY_RC1)
rc1_secondary = row_or_nan(SECONDARY_RC1)
true_guard = row_or_nan(TRUE_GUARD)
base = row_or_nan(BASELINE)
noop = row_or_nan(NOOP)

# Per-seed win-rate against the corrected true global guard.
per_seed_wins = np.nan
per_seed_ties_or_wins = np.nan
if rc1 is not None and true_guard is not None and not per_seed_df.empty:
    p = per_seed_df[per_seed_df.method == PRIMARY_RC1][["seed",
    ↪ "final_avg_acc", "min_task_acc", "avg_forgetting", "class2_acc",
    ↪ "class5_acc"]]
    g = per_seed_df[per_seed_df.method == TRUE_GUARD][["seed", "final_avg_acc",
    ↪ "min_task_acc", "avg_forgetting", "class2_acc", "class5_acc"]]
    pg = p.merge(g, on="seed", suffixes=("_rc1", "_guard"))
    if not pg.empty:
        pg["acc_delta_rc1_vs_guard"] = pg["final_avg_acc_rc1"] -
    ↪ pg["final_avg_acc_guard"]
        pg["min_task_delta_rc1_vs_guard"] = pg["min_task_acc_rc1"] -
    ↪ pg["min_task_acc_guard"]
        pg["forgetting_reduction_rc1_vs_guard"] = pg["avg_forgetting_guard"] -
    ↪ pg["avg_forgetting_rc1"]
        pg["class2_delta_rc1_vs_guard"] = pg["class2_acc_rc1"] -
    ↪ pg["class2_acc_guard"]
        per_seed_wins = float((pg["acc_delta_rc1_vs_guard"] > 0.0).mean())
        per_seed_ties_or_wins = float((pg["acc_delta_rc1_vs_guard"] >= -0.0025).
    ↪ mean())
        per_seed_cmp_path = RESULTS_DIR /
    ↪ f"rc1b_per_seed_vs_true_guard_{RUN_MODE}.csv"
        pg.to_csv(per_seed_cmp_path, index=False)
        print("Saved RC1b per-seed comparison:", per_seed_cmp_path)
        display(pg)

```

```

# Probe gaps.
probe_rows = scorecard_df[(scorecard_df.probe_row == True) & (scorecard_df.
    ↪probe_feature == "zf")] if not scorecard_df.empty else pd.DataFrame()
proto_probe = None
oracle_probe = None
if not probe_rows.empty:
    proto_probe = probe_rows[probe_rows.probe_context_mode == "proto"].
    ↪sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    oracle_probe = probe_rows[probe_rows.probe_context_mode == "oracle"].
    ↪sort_values("final_avg_acc_mean", ascending=False).iloc[0]

checks = []

def add_check(name, value, threshold_text, passed, details=""):
    checks.append({
        "check": name,
        "value": value,
        "threshold": threshold_text,
        "passed": bool(passed),
        "details": details,
    })

if base is not None and noop is not None:
    noop_delta = metric(noop, "final_avg_acc_mean") - metric(base,
    ↪"final_avg_acc_mean")
    add_check("no_op_equals_no_repair", noop_delta, "abs(delta) <= 1e-6",
    ↪abs(noop_delta) <= 1e-6,
        "No-op must be an evaluation no-op, not a hidden training variant.
    ↪")

if base is not None and true_guard is not None:
    guard_delta = metric(true_guard, "final_avg_acc_mean") - metric(base,
    ↪"final_avg_acc_mean")
    add_check("true_guard_active", guard_delta, "delta != 0 and preferably > +0.
    ↪005", abs(guard_delta) > 1e-6 and guard_delta > 0.005,
        "Corrected global CE+KL guard should not behave like no-repair.")

if rc1 is not None and base is not None:
    rc1_delta_base = metric(rc1, "final_avg_acc_mean") - metric(base,
    ↪"final_avg_acc_mean")
    add_check("rc1_beats_no_repair_accuracy", rc1_delta_base, ">= +0.020",
    ↪rc1_delta_base >= 0.020,
        "Primary RC1b candidate must materially beat no-repair.")
    rc1_min_delta_base = metric(rc1, "min_task_acc_mean") - metric(base,
    ↪"min_task_acc_mean")

```

```

    add_check("rc1_improves_min_task_vs_no_repair", rc1_min_delta_base, ">= +0.
↪050", rc1_min_delta_base >= 0.050,
        "RC1b candidate must improve weakest-task robustness.")
    rc1_forget_red_base = metric(base, "avg_forgetting_mean") - metric(rc1,
↪"avg_forgetting_mean")
    add_check("rc1_reduces_forgetting_vs_no_repair", rc1_forget_red_base, ">=
↪+0.020", rc1_forget_red_base >= 0.020,
        "RC1b candidate should reduce forgetting, not only improve the
↪last task.")

if rc1 is not None and true_guard is not None:
    rc1_delta_guard = metric(rc1, "final_avg_acc_mean") - metric(true_guard,
↪"final_avg_acc_mean")
    rc1_min_delta_guard = metric(rc1, "min_task_acc_mean") - metric(true_guard,
↪"min_task_acc_mean")
    rc1_forget_red_guard = metric(true_guard, "avg_forgetting_mean") -
↪metric(rc1, "avg_forgetting_mean")
    rc1_class2_delta_guard = metric(rc1, "class2_acc_mean") -
↪metric(true_guard, "class2_acc_mean")
    rc1_class5_delta_guard = metric(rc1, "class5_acc_mean") -
↪metric(true_guard, "class5_acc_mean")
    robustness_preferable = (rc1_min_delta_guard >= 0.010 and
↪rc1_forget_red_guard >= 0.005 and rc1_class2_delta_guard >= -0.005)
    add_check("rc1_vs_true_guard_accuracy_or_robustness", rc1_delta_guard,
        "accuracy >= +0.005 OR robustness-preferable", rc1_delta_guard >=
↪0.005 or robustness_preferable,
        f"min_task_delta={rc1_min_delta_guard:+.4f},
↪forgetting_reduction={rc1_forget_red_guard:+.4f},
↪class2_delta={rc1_class2_delta_guard:+.4f}")
    add_check("rc1_preserves_class5_vs_true_guard", rc1_class5_delta_guard, ">=
↪-0.010", rc1_class5_delta_guard >= -0.010,
        "Class 5 preservation must not be sacrificed.")

if not np.isnan(per_seed_ties_or_wins):
    add_check("rc1_seed_stability_vs_true_guard", per_seed_ties_or_wins, ">= 0.
↪60 ties-or-wins", per_seed_ties_or_wins >= 0.60,
        "RC1b candidate should not be a one-seed artifact.")

if proto_probe is not None and oracle_probe is not None and rc1 is not None:
    head_gap = proto_probe.final_avg_acc_mean - metric(rc1,
↪"final_avg_acc_mean")
    context_gap = oracle_probe.final_avg_acc_mean - proto_probe.
↪final_avg_acc_mean
    add_check("probe_gap_interpretable", head_gap, "head_gap <= 0.025
↪preferred", head_gap <= 0.025,

```

```

        f"proto_to_oracle_context_gap={context_gap:+.4f}; larger context_
        ↪gap suggests context frontier.")

rejected_original = row_or_nan(REJECTED_ORIGINAL_RC1)
secondary = row_or_nan(SECONDARY_RC1)

if rc1 is not None and rejected_original is not None:
    rc1b_delta_original = metric(rc1, "final_avg_acc_mean") -_
    ↪metric(rejected_original, "final_avg_acc_mean")
    add_check("rc1b_beats_rejected_original_rc1", rc1b_delta_original, ">= +0.
    ↪005 preferred", rc1b_delta_original >= 0.005,
        "RC1b should improve over the original blend-only guarded row_
        ↪that failed robust freeze.")

validation_gate_df = pd.DataFrame(checks)
validation_gate_path = RESULTS_DIR / f"rc1b_validation_gate_{RUN_MODE}.csv"
validation_gate_df.to_csv(validation_gate_path, index=False)
print("\n=== v1.0-RC1b validation gate ===")
print("Saved validation gate:", validation_gate_path)
display(validation_gate_df)

if not validation_gate_df.empty:
    passed_count = int(validation_gate_df["passed"].sum())
    total_count = int(len(validation_gate_df))
    print(f"Validation checks passed: {passed_count}/{total_count}")
    if RUN_MODE == "smoke":
        print("Decision: smoke passed only if the harness executed and_
        ↪diagnostics are coherent. Do not freeze/reject RC1b from smoke.")
        elif passed_count == total_count:
            print("Decision: RC1b passes this validation mode. Next step: benchmark_
            ↪campaign, not a new architecture branch.")
            elif passed_count >= max(1, total_count - 1):
                print("Decision: RC1b is nearly validated. Inspect the failed check_
                ↪before freezing.")
            else:
                print("Decision: do not freeze RC1b yet. Diagnose failed checks before_
                ↪benchmark campaign.")

```

1.21 15.2 Benchmark leaderboard and comparative gate

This cell compares the frozen RC1b candidate against the strongest non-oracle baseline rows. The decision rule is stricter than the internal paired audit:

- RC1b should beat fine-tuning, EWC, LwF and sparse MoE on average accuracy and robustness.
- Replay/PNN may remain strong controls; RC1b is benchmark-interesting if it is competitive

while preserving the MMALS interpretability/probe-gap story.

- Joint upper bound is diagnostic, not a deployable continual-learning baseline.

```
[ ]: # Benchmark-level leaderboard: exclude probes and oracle references.
leaderboard_df = scorecard_df.copy()
if not leaderboard_df.empty:
    leaderboard_df["is_probe"] = leaderboard_df["probe_row"].astype(bool)
    leaderboard_df["is_oracle_reference"] = leaderboard_df["method"].str.
    ↪contains("oracle", case=False, na=False)
    benchmark_leaderboard = leaderboard_df[(~leaderboard_df["is_probe"]) &
    ↪(~leaderboard_df["is_oracle_reference"])] .copy()
    benchmark_leaderboard = benchmark_leaderboard.
    ↪sort_values("final_avg_acc_mean", ascending=False)
    bench_path = RESULTS_DIR / f"benchmark_leaderboard_{RUN_MODE}.csv"
    benchmark_leaderboard.to_csv(bench_path, index=False)
    print("Saved benchmark leaderboard:", bench_path)
    display_cols = [
        "method", "final_avg_acc_mean", "final_avg_acc_std",
    ↪"min_task_acc_mean", "avg_forgetting_mean",
        "class2_acc_mean", "class5_acc_mean", "pred4_bias_mean",
    ↪"total_params_mean", "replay_memory_items_mean"
    ]
    display(benchmark_leaderboard[[c for c in display_cols if c in
    ↪benchmark_leaderboard.columns]])

    rc1b = get_row(scorecard_df, "paired_context_gap_selected")
    rc1b_equiv = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_head_guard_035")
    true_guard = get_row(scorecard_df,
    ↪"paired_proto_global_head_ce_kl_guard_035")
    joint = get_row(scorecard_df, "baseline_joint_upper_bound")
    deployable_competitors = benchmark_leaderboard[~benchmark_leaderboard.
    ↪method.isin([
        "paired_context_gap_selected",
        "paired_context_temp_blend_selected_head_guard_035",
        "baseline_joint_upper_bound",
    ])]
    best_competitor = deployable_competitors.iloc[0] if not
    ↪deployable_competitors.empty else None

    print("\n=== Benchmark comparative interpretation ===")
    if rc1b is not None:
        print(f"RC1b primary candidate: {rc1b.method} acc={rc1b.
    ↪final_avg_acc_mean:.4f}, min_task={rc1b.min_task_acc_mean:.4f},
    ↪forgetting={rc1b.avg_forgetting_mean:.4f}")
        if rc1b_equiv is not None:
```

```

    print(f"RC1b equivalent diagnostic: {rc1b_equiv.method} acc={rc1b_equiv.
↪final_avg_acc_mean:.4f}")
    if true_guard is not None and rc1b is not None:
        print(f"Delta RC1b vs true global guard: {rc1b.final_avg_acc_mean -
↪true_guard.final_avg_acc_mean:+.4f} acc, {rc1b.min_task_acc_mean -
↪true_guard.min_task_acc_mean:+.4f} min-task")
        if best_competitor is not None and rc1b is not None:
            print(f"Best non-oracle, non-joint competitor: {best_competitor.method}
↪acc={best_competitor.final_avg_acc_mean:.4f}")
            print(f"Delta RC1b vs best competitor: {rc1b.final_avg_acc_mean -
↪best_competitor.final_avg_acc_mean:+.4f}")
            if joint is not None and rc1b is not None:
                print(f"Joint upper bound: {joint.final_avg_acc_mean:.4f}; RC1b gap to
↪joint={joint.final_avg_acc_mean - rc1b.final_avg_acc_mean:+.4f}")

    benchmark_gate_rows = []
    if rc1b is not None:
        def add_gate(name, passed, detail):
            benchmark_gate_rows.append({"check": name, "passed": bool(passed),
↪"detail": detail})
        add_gate("rc1b_above_true_guard", true_guard is not None and rc1b.
↪final_avg_acc_mean >= true_guard.final_avg_acc_mean - 0.002,
            f"RC1b={rc1b.final_avg_acc_mean:.4f}; true_guard={true_guard.
↪final_avg_acc_mean if true_guard is not None else np.nan:.4f}")
        add_gate("rc1b_above_finetune", get_row(scorecard_df,
↪"baseline_finetune") is not None and rc1b.final_avg_acc_mean >
↪get_row(scorecard_df, "baseline_finetune").final_avg_acc_mean,
            "RC1b should beat naive fine-tuning.")
        add_gate("rc1b_min_task_strong", rc1b.min_task_acc_mean >= 0.80 if not
↪np.isnan(rc1b.min_task_acc_mean) else False,
            f"min_task={rc1b.min_task_acc_mean:.4f}")
        add_gate("rc1b_forgetting_controlled", rc1b.avg_forgetting_mean <= 0.08
↪if not np.isnan(rc1b.avg_forgetting_mean) else False,
            f"forgetting={rc1b.avg_forgetting_mean:.4f}")
        add_gate("rc1b_class2_not_collapsed", rc1b.class2_acc_mean >= 0.80 if
↪not np.isnan(rc1b.class2_acc_mean) else False,
            f"class2={rc1b.class2_acc_mean:.4f}")
        add_gate("rc1b_class5_preserved", rc1b.class5_acc_mean >= 0.95 if not
↪np.isnan(rc1b.class5_acc_mean) else False,
            f"class5={rc1b.class5_acc_mean:.4f}")
        if best_competitor is not None:
            add_gate("rc1b_competitive_with_best_baseline", rc1b.
↪final_avg_acc_mean >= best_competitor.final_avg_acc_mean - 0.010,
            f"RC1b={rc1b.final_avg_acc_mean:.4f};
↪best_competitor={best_competitor.final_avg_acc_mean:.4f}")
    benchmark_gate_df = pd.DataFrame(benchmark_gate_rows)

```

```

gate_path = RESULTS_DIR / f"benchmark_gate_{RUN_MODE}.csv"
benchmark_gate_df.to_csv(gate_path, index=False)
print("Saved benchmark gate:", gate_path)
display(benchmark_gate_df)
else:
    print("Scorecard is empty; run the experiment cells first.")

```

1.22 16. Observability audit package and weak-seed diagnosis

This cell converts the benchmark results into auditable artifacts. It does not change scores. It produces seed-level, context-level, head-level, memory-level, and OTEL-style trace tables so that weak runs can be inspected instead of treated as black-box noise.

```

[ ]: # -----
# Observability post-processing: seed, context, head, memory, and trace audits.
# -----
def _first_available(df, cols):
    return [c for c in cols if c in df.columns]

def build_seed_diagnostics(per_seed_df, class_diag_df):
    if per_seed_df.empty:
        return pd.DataFrame()
    keep = _first_available(per_seed_df, [
        "method", "seed", "method_role", "model_family", "final_avg_acc",
        ↪ "min_task_acc",
        "avg_forgetting", "class2_acc", "class5_acc", "pred4_bias",
        ↪ "total_params", "replay_memory_items"
    ])
    out = per_seed_df[keep].copy()
    out["is_rc1b_primary"] = out["method"].eq("paired_context_gap_selected")
    out["is_true_guard"] = out["method"].
    ↪ eq("paired_proto_global_head_ce_kl_guard_035")
    return out.sort_values(["seed", "final_avg_acc"], ascending=[True, False])

def build_rc1b_vs_guard_audit(per_seed_df):
    if per_seed_df.empty:
        return pd.DataFrame()
    rc = per_seed_df[per_seed_df.method == "paired_context_gap_selected"].copy()
    gd = per_seed_df[per_seed_df.method ==
    ↪ "paired_proto_global_head_ce_kl_guard_035"].copy()
    if rc.empty or gd.empty:
        return pd.DataFrame()
    cols = ["seed", "final_avg_acc", "min_task_acc", "avg_forgetting",
    ↪ "class2_acc", "class5_acc", "pred4_bias"]
    rc = rc[[c for c in cols if c in rc.columns]].rename(columns={c:
    ↪ f"rc1b_{c}" for c in cols if c != "seed"})

```

```

    gd = gd[[c for c in cols if c in gd.columns]].rename(columns={c:
↪f"guard_{c}" for c in cols if c != "seed"})
    out = rc.merge(gd, on="seed", how="outer")
    for m in ["final_avg_acc", "min_task_acc", "class2_acc", "class5_acc"]:
        if f"rc1b_{m}" in out and f"guard_{m}" in out:
            out[f"delta_{m}_rc1b_minus_guard"] = out[f"rc1b_{m}"] -
↪out[f"guard_{m}"]
        if "rc1b_avg_forgetting" in out and "guard_avg_forgetting" in out:
            out["delta_forgetting_reduction_rc1b_vs_guard"] =
↪out["guard_avg_forgetting"] - out["rc1b_avg_forgetting"]
        out["weak_seed_flag"] = False
        if "delta_final_avg_acc_rc1b_minus_guard" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] |
↪(out["delta_final_avg_acc_rc1b_minus_guard"] < 0)
        if "rc1b_class2_acc" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] | (out["rc1b_class2_acc"]
↪< 0.80)
        if "rc1b_min_task_acc" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] |
↪(out["rc1b_min_task_acc"] < 0.80)
        return out.sort_values("seed")

def build_context_selection_audit(repair_log_df):
    if repair_log_df.empty:
        return pd.DataFrame()
    mask = repair_log_df.get("repair_type", pd.Series(dtype=str)).astype(str).
↪str.contains("context_config", na=False)
    cols = _first_available(repair_log_df, [
        "method", "seed", "after_task", "task_id", "variant", "repair_type",
↪"selection_mode",
        "candidate", "selected_candidate", "candidate_temp", "selected_temp",
↪"candidate_blend_name",
        "selected_blend_name", "candidate_blend_single",
↪"candidate_blend_latent", "candidate_blend_feature",
        "selected_blend_single", "selected_blend_latent",
↪"selected_blend_feature", "proxy_acc",
        "proxy_context_top1", "proxy_context_entropy", "proxy_context_margin",
↪"proxy_class2",
        "proxy_class5", "proxy_min_class", "proxy_score",
↪"selected_proxy_score", "selected_proxy_acc",
        "selected_proxy_context_top1", "selected_proxy_class2",
↪"selected_proxy_min_class"
    ])
    return repair_log_df.loc[mask, cols].copy()

def build_head_repair_audit(repair_log_df):

```

```

    if repair_log_df.empty:
        return pd.DataFrame()
    rt = repair_log_df.get("repair_type", pd.Series(dtype=str)).astype(str)
    mask = rt.str.contains("guard|head|readout|ce_kl", case=False, na=False) |
    ↪ repair_log_df.get("uses_guarded_global_head", pd.Series(False,
    ↪ index=repair_log_df.index)).fillna(False).astype(bool)
    cols = _first_available(repair_log_df, [
        "method", "seed", "after_task", "task_id", "variant", "repair_type",
    ↪ "readout_train_loss",
        "readout_train_ce", "readout_train_kl", "readout_train_margin",
    ↪ "kl_weight", "margin_weight",
        "uses_guarded_global_head", "true_guard_control_fixed",
    ↪ "paired_same_backbone"
    ])
    return repair_log_df.loc[mask, cols].copy()

def build_memory_audit(history_df):
    if history_df.empty:
        return pd.DataFrame()
    final_task = int(history_df.after_task.max()) if "after_task" in history_df
    ↪ else N_TASKS - 1
    sub = history_df[(history_df.after_task == final_task) & (history_df.
    ↪ task_id == final_task)].copy()
    cols = _first_available(sub, [
        "method", "seed", "model_family", "replay_memory_items",
    ↪ "memory_per_class", "memory_per_task",
        "total_params", "readout_total_params", "model_size_mb_total",
    ↪ "uses_balanced_replay",
        "uses_context_selection", "uses_global_head_refresh",
    ↪ "context_temperature_used",
        "batch_blend_single_used", "batch_blend_latent_used",
    ↪ "batch_blend_feature_used"
    ])
    return sub[cols].sort_values(["seed", "method"]).copy()

def build_probe_gap_audit(scorecard_df):
    if scorecard_df.empty:
        return pd.DataFrame()
    def row(method):
        s = scorecard_df[scorecard_df.method == method]
        return s.iloc[0] if not s.empty else None
    rc = row("paired_context_gap_selected")
    proto = row("probe_proto__zf_proto")
    oracle = row("probe_proto__zf_oracle")
    rows = []
    if rc is not None and proto is not None:

```

```

        rows.append({
            "gap_name": "deployed_to_proto_zf_probe",
            "from_method": "paired_context_gap_selected",
            "to_method": "probe_proto__zf_proto",
            "gap": float(proto.final_avg_acc_mean - rc.final_avg_acc_mean),
        })
    if proto is not None and oracle is not None:
        rows.append({
            "gap_name": "proto_to_oracle_context_gap",
            "from_method": "probe_proto__zf_proto",
            "to_method": "probe_proto__zf_oracle",
            "gap": float(oracle.final_avg_acc_mean - proto.final_avg_acc_mean),
        })
    if rc is not None and oracle is not None:
        rows.append({
            "gap_name": "deployed_to_oracle_zf_probe",
            "from_method": "paired_context_gap_selected",
            "to_method": "probe_proto__zf_oracle",
            "gap": float(oracle.final_avg_acc_mean - rc.final_avg_acc_mean),
        })
    return pd.DataFrame(rows)

seed_diag_df = build_seed_diagnostics(per_seed_df, class_diag_df)
rc1b_guard_audit_df = build_rc1b_vs_guard_audit(per_seed_df)
context_selection_audit_df = build_context_selection_audit(repair_log_df)
head_repair_audit_df = build_head_repair_audit(repair_log_df)
memory_audit_df = build_memory_audit(history_df)
probe_gap_audit_df = build_probe_gap_audit(scorecard_df)

exports = {
    "seed_diagnostics": seed_diag_df,
    "rc1b_vs_true_guard_seed_audit": rc1b_guard_audit_df,
    "context_selection_trace": context_selection_audit_df,
    "head_repair_trace": head_repair_audit_df,
    "memory_audit": memory_audit_df,
    "probe_gap_audit": probe_gap_audit_df,
}
for name, df in exports.items():
    path = AUDIT_DIR / f"{name}_{RUN_MODE}.csv"
    df.to_csv(path, index=False)
    print("Saved audit artifact:", path)

# Create OTEL-style span rows: one span per seed/method final summary and per_
# weak-seed diagnosis.
span_rows = []
for _, r in seed_diag_df.iterrows():
    span_rows.append({

```

```

        "trace_id": TRACE_ID,
        "span_id": hashlib.sha256(f"{RUN_ID}:{r.get('seed')}:{r.get('method')}".
↳ encode()).hexdigest()[:16],
        "parent_span_id": TRACE_ID,
        "span_name": "method_seed_final_score",
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "seed": safe_jsonable(r.get("seed")),
        "method": safe_jsonable(r.get("method")),
        "final_avg_acc": safe_jsonable(r.get("final_avg_acc")),
        "min_task_acc": safe_jsonable(r.get("min_task_acc")),
        "avg_forgetting": safe_jsonable(r.get("avg_forgetting")),
        "class2_acc": safe_jsonable(r.get("class2_acc")),
        "class5_acc": safe_jsonable(r.get("class5_acc")),
        "pred4_bias": safe_jsonable(r.get("pred4_bias")),
    })
TRACE_SPANS.extend(span_rows)
span_df = pd.DataFrame(TRACE_SPANS)
span_df.to_csv(TRACE_SPANS_PATH, index=False)
print("Saved OTEL-style spans:", TRACE_SPANS_PATH)

# Add compact JSONL events for key audit rows.
for _, r in rc1b_guard_audit_df.iterrows():
    log_event("rc1b_vs_true_guard_seed_audit", seed=safe_jsonable(r.
↳ get("seed")), method="paired_context_gap_selected", phase="post_run_audit",
↳ payload=r.to_dict(), level="WARN" if bool(r.get("weak_seed_flag")) else
↳ "INFO")
for _, r in probe_gap_audit_df.iterrows():
    log_event("probe_gap_audit", method=safe_jsonable(r.get("from_method")),
↳ phase="post_run_audit", payload=r.to_dict())

weak_rows = rc1b_guard_audit_df[rc1b_guard_audit_df.get("weak_seed_flag",
↳ False) == True] if not rc1b_guard_audit_df.empty else pd.DataFrame()
weak_seed_report = {
    "n_weak_seeds": int(len(weak_rows)),
    "weak_seeds": safe_jsonable(weak_rows.to_dict("records")) if not weak_rows.
↳ empty else [],
    "diagnostic_interpretation": "Weak seeds are candidates for localized
↳ inspection: check context_selection_trace, head_repair_trace, memory_audit,
↳ class_diagnostics, and probe_gap_audit.",
}
weak_seed_report_path = AUDIT_DIR / f"weak_seed_report_{RUN_MODE}.json"
with open(weak_seed_report_path, "w", encoding="utf-8") as f:
    json.dump(weak_seed_report, f, indent=2, ensure_ascii=False)
print("Saved weak seed report:", weak_seed_report_path)

```

```

log_event("observability_audit_completed", phase="post_run_audit",
    payload={"audit_files": [str(AUDIT_DIR / f"{k}_{RUN_MODE}.csv") for k in
    exports] + [str(weak_seed_report_path)]})

# Write/update manifest after audit outputs are available.
manifest = write_manifest(extra={
    "audit_exports": list(exports.keys()),
    "weak_seed_report": str(weak_seed_report_path),
    "trace_jsonl": str	TRACE_JSONL_PATH),
    "trace_spans": str	TRACE_SPANS_PATH),
})
print("Saved manifest:", MANIFEST_PATH)

print("\n=== Weak seed audit preview ===")
display(rc1b_guard_audit_df)
print("\n=== Probe-gap audit ===")
display(probe_gap_audit_df)

```

1.23 16. Export results

This cell zips all CSV outputs for upload and external analysis. The ZIP includes history, losses, repair logs, probe losses, forgetting, confusion, scorecards, paired deltas, per-seed RC1b comparisons, and the validation-gate table.

1.24 16b. Notebook/PDF evidence archival

This cell stores notebook-level evidence inside the same Google Drive results directory.

It always creates a compact **run-summary PDF** from the tables already produced by the notebook.

It also tries to copy the source `.ipynb` and any already-generated full notebook PDF from Google Drive into `notebook_exports/`.

Optional full notebook PDF export is supported, but disabled by default because LaTeX/Pandoc installation can be slow in Colab.

```

[ ]: # -----
# Notebook/PDF evidence archival helper.
# -----
# By default this cell creates a compact run-summary PDF that is always stored
# in RESULTS_DIR/notebook_exports and included in the final ZIP package.
#
# If you want to export the full notebook to PDF from inside Colab, set both:
#   INSTALL_FULL_PDF_EXPORT_DEPS = True
#   TRY_FULL_NOTEBOOK_PDF_EXPORT = True
# The dependency installation can take several minutes.
# -----
from matplotlib.backends.backend_pdf import PdfPages
import subprocess

```

```

INSTALL_FULL_PDF_EXPORT_DEPS = False
TRY_FULL_NOTEBOOK_PDF_EXPORT = False
COPY_EXISTING_NOTEBOOK_PDF_IF_FOUND = True

NOTEBOOK_EXPORT_DIR.mkdir(parents=True, exist_ok=True)
EXPECTED_FULL_NOTEBOOK_PDF = □
    ↪f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}_{RUN_ID}.pdf"
RUN_SUMMARY_PDF_PATH = NOTEBOOK_EXPORT_DIR / □
    ↪f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}_{RUN_ID}_summary.pdf"

def find_file_by_name(filename: str, max_hits: int = 5):
    roots = []
    for p in [Path("/content/drive/MyDrive"), Path("/content"), Path(".")]:
        if p.exists():
            roots.append(p)
    hits = []
    for root in roots:
        try:
            hits.extend(list(root.rglob(filename))[:max_hits])
        except Exception:
            pass
    # Prefer files outside RESULTS_DIR to avoid copying the same file onto □
    ↪itself.
    unique = []
    seen = set()
    for h in hits:
        try:
            hp = h.resolve()
            if str(hp) not in seen:
                seen.add(str(hp))
                unique.append(h)
        except Exception:
            pass
    return unique[:max_hits]

def copy_if_exists(src: Path, dst: Path):
    try:
        if src.exists():
            dst.parent.mkdir(parents=True, exist_ok=True)
            if src.resolve() != dst.resolve():
                shutil.copy2(src, dst)
            return dst
    except Exception as e:
        print(f"Copy skipped for {src}: {repr(e)}")
    return None

```

```

def text_page(pdf, title, lines, fontsize=10):
    fig = plt.figure(figsize=(8.27, 11.69)) # A4 portrait
    fig.text(0.06, 0.96, title, fontsize=15, weight="bold", va="top")
    y = 0.91
    for line in lines:
        if y < 0.06:
            pdf.savefig(fig, bbox_inches="tight")
            plt.close(fig)
            fig = plt.figure(figsize=(8.27, 11.69))
            y = 0.96
            fig.text(0.06, y, str(line), fontsize=fontsize, va="top",
family="monospace")
            y -= 0.026
        pdf.savefig(fig, bbox_inches="tight")
        plt.close(fig)

def write_run_summary_pdf():
    lines = [
        f"Notebook: {NOTEBOOK_FILENAME}",
        f"Run ID: {RUN_ID}",
        f"Dataset: {DATASET_NAME}",
        f"Pair mode: {PAIR_MODE}",
        f"Rotation angles: {ROTATION_ANGLES if DATASET_NAME == 'RotatedMNIST'
else 'n/a'}",
        f"Permutation seeds: {PERMUTATION_SEEDS if DATASET_NAME ==
'PermutedMNIST' else 'n/a'}",
        f"RC2b candidates: {RC2B_CANDIDATE_VARIANTS}",
        f"Run mode: {RUN_MODE}",
        f"Seeds: {SEEDS}",
        f"Results dir: {RESULTS_DIR}",
        f"Trace dir: {TRACE_DIR}",
        f"Audit dir: {AUDIT_DIR}",
        f"Config hash: {CONFIG_HASH}",
        "",
        "Strict selected candidate: paired_rc2b_validation_selected_policy",
        "Historical fixed candidate: paired_context_gap_selected",
        "Architecture: proto-first MMALS + validation-memory-selected
deployable policy; no final-test leakage for policy selection",
        "",
        "This PDF is a compact evidence summary generated by the notebook.",
        "For the full executed notebook PDF, use the optional nbconvert export
or Colab browser PDF export.",
    ]

    top_methods = []
    try:

```

```

        if "scorecard_df" in globals() and isinstance(scorecard_df, pd.
↪DataFrame) and not scorecard_df.empty:
            sort_col = "final_avg_acc_mean" if "final_avg_acc_mean" in
↪scorecard_df.columns else None
            cols = [
                "method_role", "method", "final_avg_acc_mean",
↪"min_task_acc_mean",
                "avg_forgetting_mean", "class2_acc_mean", "class5_acc_mean"
            ]
            cols = [c for c in cols if c in scorecard_df.columns]
            if sort_col:
                top = scorecard_df[cols].sort_values(sort_col, ascending=False).
↪head(15)
            else:
                top = scorecard_df[cols].head(15)
                top_methods = top.to_string(index=False).splitlines()
        except Exception as e:
            top_methods = [f"Could not render scorecard table: {repr(e)}"]

        gate_lines = []
        try:
            if "validation_gate_df" in globals() and isinstance(validation_gate_df,
↪pd.DataFrame) and not validation_gate_df.empty:
                gate_lines += ["", "RC1b validation gate:"]
                gate_lines += validation_gate_df.to_string(index=False).splitlines()
            if "benchmark_gate_df" in globals() and isinstance(benchmark_gate_df,
↪pd.DataFrame) and not benchmark_gate_df.empty:
                gate_lines += ["", "Benchmark gate:"]
                gate_lines += benchmark_gate_df.to_string(index=False).splitlines()
        except Exception as e:
            gate_lines += [f"Could not render gate tables: {repr(e)}"]

        artifact_lines = []
        try:
            artifact_lines = [str(p.relative_to(RESULTS_DIR)) for p in
↪sorted(RESULTS_DIR.rglob("*")) if p.is_file()]
        except Exception:
            artifact_lines = []

        with PdfPages(RUN_SUMMARY_PDF_PATH) as pdf:
            text_page(pdf, "MMALS v1.0-RC1b Rotated-MNIST Cross-Benchmark - Run
↪Summary", lines)
            text_page(pdf, "Top scorecard rows", top_methods if top_methods else
↪["Scorecard not available yet."])
            text_page(pdf, "Validation / benchmark gates", gate_lines if gate_lines
↪else ["Gate tables not available yet."])

```

```

        text_page(pdf, "Evidence artifacts stored in this run folder",
        ↪artifact_lines if artifact_lines else ["No artifacts listed yet."],
        ↪fontsize=8)

    return RUN_SUMMARY_PDF_PATH

# Always create a compact summary PDF inside the evidence folder.
summary_pdf_path = write_run_summary_pdf()
print("Run-summary PDF stored at:", summary_pdf_path)

# Try to archive the source notebook if it is saved in Drive or /content.
archived_notebook_path = None
notebook_hits = find_file_by_name(NOTEBOOK_FILENAME)
if notebook_hits:
    archived_notebook_path = copy_if_exists(notebook_hits[0],
    ↪NOTEBOOK_EXPORT_DIR / NOTEBOOK_FILENAME)
    print("Notebook source archived:", archived_notebook_path)
else:
    print("Notebook source not found automatically. Save the notebook in Google
    ↪Drive with this exact filename if you want it archived:")
    print(" -", NOTEBOOK_FILENAME)

# Optionally copy an existing full notebook PDF into the evidence folder.
archived_full_pdf_path = None
if COPY_EXISTING_NOTEBOOK_PDF_IF_FOUND:
    candidate_pdf_names = [
        EXPECTED_FULL_NOTEBOOK_PDF,
        f"{NOTEBOOK_BASENAME}.pdf",
        f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}.pdf",
    ]
    for pdf_name in candidate_pdf_names:
        hits = find_file_by_name(pdf_name)
        if hits:
            archived_full_pdf_path = copy_if_exists(hits[0],
            ↪NOTEBOOK_EXPORT_DIR / pdf_name)
            print("Existing full notebook PDF archived:",
            ↪archived_full_pdf_path)
            break
    if archived_full_pdf_path is None:
        print("No existing full notebook PDF found automatically. Expected one
        ↪of:")
        for pdf_name in candidate_pdf_names:
            print(" -", pdf_name)

# Optional: install dependencies and try full notebook PDF export from inside
↪Colab.

```

```

nbconvert_status = {"attempted": False, "success": False, "message": "not_
↳attempted"}
if INSTALL_FULL_PDF_EXPORT_DEPS:
    print("Installing full PDF export dependencies. This can take several_
↳minutes.")
    subprocess.run("apt-get update -qq && apt-get install -y texlive_
↳texlive-xetex texlive-latex-extra pandoc", shell=True, check=False)
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "nbformat"],_
↳check=False)

if TRY_FULL_NOTEBOOK_PDF_EXPORT:
    nbconvert_status["attempted"] = True
    source_for_nbconvert = archived_notebook_path
    if source_for_nbconvert is None and notebook_hits:
        source_for_nbconvert = notebook_hits[0]
    if source_for_nbconvert is None:
        nbconvert_status["message"] = "No notebook source found for nbconvert."
    else:
        try:
            cmd = [
                "jupyter", "nbconvert",
                "--to", "pdf",
                "--output-dir", str(NOTEBOOK_EXPORT_DIR),
                "--PDFExporter.latex_command=['xelatex', '{filename}'],_
↳'-interaction=nonstopmode']",
                str(source_for_nbconvert),
            ]
            print("Running:", " ".join(cmd))
            proc = subprocess.run(cmd, text=True, capture_output=True,_
↳check=False)
            nbconvert_status["returncode"] = proc.returncode
            nbconvert_status["stdout_tail"] = proc.stdout[-2000:]
            nbconvert_status["stderr_tail"] = proc.stderr[-2000:]
            nbconvert_status["success"] = proc.returncode == 0
            nbconvert_status["message"] = "nbconvert completed" if proc.
↳returncode == 0 else "nbconvert failed"
        except Exception as e:
            nbconvert_status["message"] = repr(e)

with open(NOTEBOOK_EXPORT_DIR / "notebook_pdf_export_status.json", "w",_
↳encoding="utf-8") as f:
    json.dump({
        "notebook_filename": NOTEBOOK_FILENAME,
        "expected_full_notebook_pdf": EXPECTED_FULL_NOTEBOOK_PDF,
        "summary_pdf_path": str(summary_pdf_path),

```

```

        "archived_notebook_path": str(archived_notebook_path) if
↪ archived_notebook_path else None,
        "archived_full_pdf_path": str(archived_full_pdf_path) if
↪ archived_full_pdf_path else None,
        "nbconvert_status": nbconvert_status,
    }, f, indent=2)

log_event("notebook_pdf_evidence_archived", phase="export", payload={
    "summary_pdf_path": str(summary_pdf_path),
    "archived_notebook_path": str(archived_notebook_path) if
↪ archived_notebook_path else None,
    "archived_full_pdf_path": str(archived_full_pdf_path) if
↪ archived_full_pdf_path else None,
    "nbconvert_status": nbconvert_status,
})
write_manifest(extra={"notebook_pdf_evidence": str(NOTEBOOK_EXPORT_DIR)})

```

1.25 15.3 RC2b strict no-leakage policy-selection gate

This gate evaluates the selected policy row `paired_rc2b_validation_selected_policy`.

It checks both scientific performance and the strict process claim: policy selection must be performed from validation memory before final-test evaluation.

```

[ ]: # -----
# v1.1-RC2b strict no-leakage policy-selection gate.
# -----
RC2B_PRIMARY = RC2B_METHOD
RC1B_PRIMARY = "paired_context_gap_selected"

def _score_row(method):
    return get_row(scorecard_df, method)

rc2b_row = _score_row(RC2B_PRIMARY)
rc1b_row = _score_row(RC1B_PRIMARY)
balanced_replay_row = _score_row("baseline_balanced_replay")
experience_replay_row = _score_row("baseline_experience_replay")

replay_candidates = [r for r in [balanced_replay_row, experience_replay_row] if
↪ r is not None]
best_replay_row = max(replay_candidates, key=lambda r: metric(r,
↪ "final_avg_acc_mean")) if replay_candidates else None

non_diag = non_diagnostic_scorecard(scorecard_df)
mmals_deployable = non_diag[non_diag.method.str.startswith("paired_",
↪ na=False)] if not non_diag.empty else pd.DataFrame()

```

```

# Best deployable MMALS row, but exclude oracle references and probes by
↳construction.
best_mmals_row = mmals_deployable.sort_values("final_avg_acc_mean",
↳ascending=False).iloc[0] if not mmals_deployable.empty else None

selection_expected_rows = len(SEEDS) * N_TASKS * len(RC2B_CANDIDATE_VARIANTS)
selected_rows = selection_log_df[selection_log_df.get("is_selected_candidate",
↳pd.Series(dtype=bool)).astype(bool)] if "selection_log_df" in globals() and
↳not selection_log_df.empty else pd.DataFrame()
selected_per_checkpoint = selected_rows.groupby(["seed", "after_task"]).size().
↳reset_index(name="n_selected") if not selected_rows.empty else pd.DataFrame()

rc2b_gate_rows = []
def add_rc2b_gate(name, passed, detail):
    rc2b_gate_rows.append({"check": name, "passed": bool(passed), "detail":
↳str(detail)})

add_rc2b_gate(
    "policy_level_validation_proxy_available_all_candidates",
    (not selection_log_df.empty) and len(selection_log_df) >=
↳selection_expected_rows and selection_log_df["proxy_acc"].notna().all(),
    f"selection_rows={len(selection_log_df) if 'selection_log_df' in globals()
↳else 0}; expected>={selection_expected_rows}",
)
add_rc2b_gate(
    "exactly_one_policy_selected_per_seed_task",
    (not selected_per_checkpoint.empty) and
↳bool((selected_per_checkpoint["n_selected"] == 1).all()) and
↳len(selected_per_checkpoint) == len(SEEDS) * N_TASKS,
    f"selected_checkpoints={len(selected_per_checkpoint)}; expected={len(SEEDS)
↳* N_TASKS}",
)
add_rc2b_gate(
    "no_final_test_used_for_selection_declared",
    (not selection_log_df.empty) and
↳bool(selection_log_df["no_final_test_used_for_selection"].astype(bool).
↳all()),
    "selection trace marks no_final_test_used_for_selection=True for every
↳candidate row.",
)
if rc2b_row is not None and rc1b_row is not None:
    add_rc2b_gate(
        "rc2b_at_least_fixed_rc1b_with_tolerance",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(rc1b_row,
↳"final_avg_acc_mean") - RC2B_SELECTION_TIE_TOL,

```

```

        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
    ↪RC1b={metric(rc1b_row, 'final_avg_acc_mean'):.4f}; delta={metric(rc2b_row,
    ↪'final_avg_acc_mean') - metric(rc1b_row, 'final_avg_acc_mean'):+.4f}",
    )
if rc2b_row is not None and best_replay_row is not None:
    add_rc2b_gate(
        "rc2b_beats_best_replay_margin",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_replay_row,
    ↪"final_avg_acc_mean") + 0.005,
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
    ↪best_replay={metric(best_replay_row, 'final_avg_acc_mean'):.4f}",
    )
if rc2b_row is not None and best_mmals_row is not None:
    add_rc2b_gate(
        "rc2b_within_tolerance_of_best_deployable_mmals",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_mmals_row,
    ↪"final_avg_acc_mean") - max(0.005, RC2B_SELECTION_TIE_TOL),
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
    ↪best_mmals={metric(best_mmals_row, 'final_avg_acc_mean'):.4f};
    ↪best={best_mmals_row.method}",
    )
if rc2b_row is not None:
    add_rc2b_gate("rc2b_min_task_floor", metric(rc2b_row, "min_task_acc_mean")
    ↪>= 0.80, f"min_task={metric(rc2b_row, 'min_task_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_class2_floor", metric(rc2b_row, "class2_acc_mean") >= 0.
    ↪80, f"class2={metric(rc2b_row, 'class2_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_class5_floor", metric(rc2b_row, "class5_acc_mean") >= 0.
    ↪90, f"class5={metric(rc2b_row, 'class5_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_forgetting_controlled", metric(rc2b_row,
    ↪"avg_forgetting_mean") <= 0.08, f"forgetting={metric(rc2b_row,
    ↪'avg_forgetting_mean'):.4f}")

rc2b_gate_df = pd.DataFrame(rc2b_gate_rows)
rc2b_gate_path = RESULTS_DIR / f"rc2b_strict_policy_gate_{RUN_MODE}.csv"
rc2b_gate_df.to_csv(rc2b_gate_path, index=False)
print("Saved RC2b strict policy gate:", rc2b_gate_path)
display(rc2b_gate_df)

RC2B_STRICT_PASSED = bool(rc2b_gate_df["passed"].all()) if not rc2b_gate_df.
    ↪empty else False
print("RC2B_STRICT_PASSED:", RC2B_STRICT_PASSED)

if not selected_rows.empty:
    selected_policy_summary_df = selected_rows.groupby([
        "selected_candidate_variant", "selected_candidate_method",
    ↪"selected_policy_family", "selected_policy_label"

```

```

], as_index=False).agg(
    n_selected=("is_selected_candidate", "sum"),
    mean_selected_proxy_acc=("selected_proxy_acc", "mean"),
    mean_selected_proxy_score=("selected_proxy_score", "mean"),
).sort_values("n_selected", ascending=False)
selected_policy_summary_path = RESULTS_DIR /
↳f"rc2b_selected_policy_summary_{RUN_MODE}.csv"
selected_policy_summary_df.to_csv(selected_policy_summary_path, index=False)
print("Saved selected policy summary:", selected_policy_summary_path)
display(selected_policy_summary_df)
else:
    selected_policy_summary_df = pd.DataFrame()

```

```

[ ]: # Finalize persistent logs, manifest, and ZIP package.
log_event("packaging_started", phase="export", payload={"results_dir":
↳str(RESULTS_DIR)})
write_manifest(extra={"packaging_started": True})
stop_persistent_logging()

output_base = RESULTS_DIR.parent / f"{RESULTS_DIR.name}_package"
zip_path = Path(shutil.make_archive(str(output_base), "zip", RESULTS_DIR))
zip_sha = file_sha256(zip_path)

package_sha_path = zip_path.with_name(zip_path.stem + "_sha256.txt")
package_sha_path.write_text(f"{zip_sha} {zip_path.name}\n", encoding="utf-8")

release_manifest_path = zip_path.with_name(zip_path.stem + "_release_manifest.
↳json")
release_manifest = {
    "run_id": RUN_ID,
    "trace_id": TRACE_ID,
    "dataset": DATASET_NAME,
    "run_mode": RUN_MODE,
    "pair_mode": PAIR_MODE,
    "rotation_angles": ROTATION_ANGLES if DATASET_NAME == "RotatedMNIST" else
↳None,
    "permutation_seeds": PERMUTATION_SEEDS if DATASET_NAME == "PermutedMNIST"
↳else None,
    "rc2b_method": RC2B_METHOD,
    "rc2b_candidate_variants": RC2B_CANDIDATE_VARIANTS,
    "rc2b_strict_passed": bool(globals().get("RC2B_STRICT_PASSED", False)),
    "notebook_filename": NOTEBOOK_FILENAME,
    "results_dir": str(RESULTS_DIR),
    "zip_package": str(zip_path),
    "zip_sha256": zip_sha,
    "package_sha256_file": str(package_sha_path),
    "run_manifest": str(MANIFEST_PATH),

```

```

    "notebook_export_dir": str(NOTEBOOK_EXPORT_DIR),
    "created_utc": datetime.now(timezone.utc).isoformat(),
}
release_manifest_path.write_text(json.dumps(safe_jsonable(release_manifest),
    ↪indent=2, ensure_ascii=False), encoding="utf-8")

print(f"Results zipped to: {zip_path}")
print(f"ZIP SHA-256: {zip_sha}")
print(f"Package SHA file: {package_sha_path}")
print(f"Release manifest: {release_manifest_path}")

# Add ZIP metadata to the run manifest after the ZIP exists.
write_manifest(extra={
    "zip_package": str(zip_path),
    "zip_sha256": zip_sha,
    "package_sha256_file": str(package_sha_path),
    "release_manifest": str(release_manifest_path),
})
print("Final manifest:", MANIFEST_PATH)

```

1.26 17. Next scientific step after RC2b

If RC2b strict passes across the four MNIST-family benchmarks, the next step is:

MMALS v1.2 - Split-CIFAR-10 Visual Benchmark

Split-CIFAR-10 should not reuse the MLP backbone as-is. It should introduce a small CNN or tiny ResNet-style encoder and explicitly separate:

- visual representation quality;
- continual-learning memory stability;
- context inference;
- readout/head policy selection;
- parameter and compute cost.

If RC2b fails, do not move to CIFAR yet. Diagnose whether the failure comes from the validation proxy, the selected policy family, or the backbone/context-memory mechanism.

1.27 17. Explanation for a 12-year-old kid

Before, the mushroom model was tested on normal images.

Now we can make the game harder in two ways:

1. rotate the handwritten digits (`RotatedMNIST`);
2. scramble the pixels differently for each lesson (`PermutedMNIST`).

The lessons still overlap:

- lesson 0: digits 0 and 1;
- lesson 1: digits 1 and 2;

- lesson 2: digits 2 and 3;
- lesson 3: digits 3 and 4;
- lesson 4: digits 4 and 5.

For `RotatedMNIST`, each lesson is turned a little differently:

- -30 degrees;
- -15 degrees;
- 0 degrees;
- +15 degrees;
- +30 degrees.

For `PermutedMNIST`, each lesson uses a different secret pixel shuffle:

- seed 100;
- seed 101;
- seed 102;
- seed 103;
- seed 104.

So the mushroom must not only remember old digits. It must also understand that the situation changed.

In this observability patch, we keep a black box recorder, like in an airplane.

It records:

- which situation the mushroom thought it was in;
- which memory/prototype it used;
- whether the head/readout was stable;
- whether one seed was weaker than the others;
- whether the model beat replay and other baselines.

The important idea is:

We are not changing the mushroom again. We are testing whether the same mushroom stays reliable on harder tracks.

1.28 Resources

This notebook is intended to be saved as:

`MMALS_Prototype_1_1_RC2b_Strict_No_Leakage_Policy_Selection_Harness_Colab.ipynb`

Expected evidence files generated by the run:

```
history_<mode>.csv
scorecard_<mode>.csv
per_seed_scorecard_<mode>.csv
rc2b_policy_selection_trace_<mode>.csv
rc2b_strict_policy_gate_<mode>.csv
rc2b_selected_policy_summary_<mode>.csv
benchmark_leaderboard_<mode>.csv
benchmark_gate_<mode>.csv
```

```
traces/otel_style_events_<mode>.jsonl
traces/otel_style_spans_<mode>.csv
audit/*.csv
notebook_exports/*summary.pdf
run_manifest_<mode>.json
*_package.zip
```

Run order:

1. RUN_MODE = "smoke" on one dataset.
2. RUN_MODE = "robust" on the same dataset.
3. Repeat robust for FashionMNIST, MNIST, RotatedMNIST and PermutedMNIST.
4. Consolidate results into an RC2b cross-benchmark package.

Publication-safe claim if robust passes:

RC2b demonstrates strict, validation-memory-selected deployable policy selection without final-test leakage on the evaluated benchmark.

Non-claims:

- not certification;
- not production readiness;
- not boundary-free streaming;
- not open-world class-incremental learning;
- not Split-CIFAR-10 evidence.

1.29 Resources

This is a placeholder for any additional resources or references.

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

```
[ ]: import os

!apt-get update
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc
!pip install py pandoc
!pip -q install nbformat
```

```
[ ]: # Explicitly setting the notebook filename for this execution,
# as there isn't a robust automatic detection method in Colab.
notebook_filename =
    ↪ "MMALS_Prototype_1_1_RC2b_Strict_No_Leakage_Policy_Selection_Harness_Colab."
    ↪ "ipynb"
notebook_name_without_ext = os.path.splitext(notebook_filename)[0]

# Define the full path to the notebook in Google Drive
full_notebook_path_drive = f"/content/drive/MyDrive/Colab Notebooks/
    ↪ {notebook_filename}"
```

```

local_notebook_path = f"/content/{notebook_filename}"

# Copy the notebook to the local /content/ directory to avoid path issues with
↳ nbconvert
print(f"Executing: cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\"")
!bash -c "cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\""

```

```

[ ]: !jupyter nbconvert \
    --to pdf \
    --output-dir='/content' \
    --PDFExporter.latex_command=["'xelatex', '{filename}', \
    ↳ '-interaction=nonstopmode']" \
    "/content/
    ↳ MMALS_Prototype_1_1_RC2b_Strict_No_Leakage_Policy_Selection_Harness_Colab.
    ↳ ipynb"

```